

Raccu



Sistema Móvil Gamificado para la Autorregulación Digital

Integrantes del grupo

- Fernanda Arraño
- Javier Vieytes
- Benjamín Castillo

Índice

1. Introducción	5
2. Contexto del proyecto	6
3. Descripción del problema u oportunidad	11
3.1. Descripción del Problema	12
3.2. Análisis de causas y efectos	12
3.3. Representación Gráfica del Problema	13
Diagrama de Ishikawa	13
Árbol del problema	14
4. Descripción del mercado objetivo	15
4.1. Definición de los Usuarios y Clientes	15
4.2. Necesidades y Dolores del Mercado	15
Perfil de usuario 1	16
Perfil de usuario 2	16
Stakeholders Internos	17
Stakeholders Directos	17
Stakeholders Indirectos	17
5. Objetivo general y objetivos específicos	18
5.1. Objetivo General	18
5.2. Objetivos Específicos	18
6. Situación inicial del cliente (AS-IS)	19
6.1. Descripción del Proceso Actual	19
6.2. Puntos de Dolor Identificados	19
6.3. Diagrama de Flujo del Proceso Actual (AS-IS)	20
7. Planificación	21
7.1. Metodología de Gestión: SCRUM y Trello	21
7.2. Organización del Equipo y Roles Críticos	21
7.3. Cronograma de Actividades (Carta Gantt)	22
8. Servicios Cloud a utilizar (IaaS / PaaS / SaaS)	23
9. Conceptualización (Solución propuesta – Versión preliminar)	24
Funcionalidades Principales:	24
1. Módulo de Intercepción y Bloqueo Inteligente:	24
2. Motor de Desafíos Multimodal:	25
3. Sistema de Economía de Fichas (Puntos y Recompensas):	27
4. Panel de Control Parental (Gestor de Reglas):	28
5. Sistema de Notificaciones y Reportes:	29
6. Arquitectura de Sincronización Cloud:	30
10. Alcance, supuestos y restricciones	31
Alcance del Proyecto	31
Exclusiones del Proyecto	31
Entregables	32
Supuestos	32

Restricciones	32
11. Metodología de desarrollo y gestión del proyecto	34
Metodología Scrum	34
Planificación, Seguimiento y Control Semanal	34
12. Documentos y diagramas de diseño de la solución	40
Modelo de datos (MER/DER) o esquema de BD (tablas, relaciones)	41
Mockups de la app: seleccione foto de perfil, revisión de evidencias enviadas	45
Diagrama de arquitectura (alto nivel).	46
13. Arquitectura de software y patrones de diseño	47
13.1. Arquitectura de N-Capas (Monolito Modular)	47
13.2. Patrones de Diseño Implementados	47
13.3. Justificación de la Arquitectura	47
13.4. Representación Visual de la Arquitectura	48
C4 Nivel 1 - Contexto del Sistema	48
C4 Nivel 2 - Contenedores	49
14. Tecnologías, lenguaje y dependencias (stack definitivo)	50
Stack Tecnológico Definitivo	50
Dependencias del Backend (requirements.txt)	50
Estándares de Código	50
15. Configuración de servidor de producción (paso a paso)	53
Plataforma Cloud	53
Paso a Paso — Backend en Render	53
Paso a Paso — Base de Datos en Supabase	53
16. Estado de avance del desarrollo (evidencia de producto funcional)	55
Funcionalidades Implementadas	55
Funcionalidades En Progreso	56
Calidad y Seguridad	56
17. Integraciones necesarias	57
17.1. Detalle de Integraciones Existentes y Planificadas	57
17.2. Consideraciones de Seguridad y Protección de Datos	58
17.3. Flujo de Datos y Comunicación entre Servicios	58
17.4. Evidencias de Pruebas de Integración	59
18. Estrategia y enfoque de calidad	61
19. Plan de pruebas	63
20. Tipos de pruebas aplicadas	65
20.1a pruebas unitarias (lógica) · Flutter	65
20.1b pruebas unitarias (UI / widgets) · Flutter	65
20.2 Pruebas de humo (smoke)	65
20.3 Pruebas de integración	66
20.4 Pruebas funcionales (E2E)	66
20.5 Pruebas de regresión	66
20.6 Pruebas de rendimiento / carga	67
20.7 Pruebas de seguridad	67
20.8 Pruebas unitarias de backend	68

21. Herramientas y software utilizados	69
22. Ejecución de pruebas y resultados (Coverage)	71
Coverage total del Frontend	71
Coverage del Backend - 95%	73
Casos representativos:	74
Pruebas de humo en dispositivo (9 casos)	83
Pruebas Unitarias - Lógica (modelos, providers y servicios)	84
Pruebas Unitarias — Interfaz (pantallas / widgets)	85
Pruebas de Integración	86
Pruebas Funcionales (E2E)	87
Pruebas de Regresión	88
Pruebas de Seguridad	89
Pruebas de carga / rendimiento (4 casos)	91
Pruebas de Humo Backend	93
Pruebas unitarias backend (105 casos)	93
Pruebas de integración	94
Pruebas de Regresión	95
Prueba de Seguridad: Auditoría de dependencias	95
Prueba de seguridad: Validación de entradas	96
Prueba de Carga: k6	96
Conclusión 1	97
Referencias Bibliográficas	100
Anexos	101
Análisis estático de código — flutter analyze (Linter)	101

1. Introducción

En la actualidad, los dispositivos electrónicos forman parte central de la vida cotidiana de niños y jóvenes. Si bien estas tecnologías ofrecen oportunidades de aprendizaje y entretenimiento, su uso desmedido representa un desafío creciente para padres, educadores y profesionales de la salud. Las plataformas digitales están diseñadas para maximizar el tiempo que los usuarios pasan en ellas mediante estímulos visuales constantes, lo que dificulta que los niños desarrollen hábitos de uso equilibrado de manera autónoma.

En este contexto, se identificó una problemática concreta: los niños pueden llegar a pasar hasta 10 horas diarias frente a una pantalla, generando dependencia y limitando el tiempo dedicado a actividades esenciales para su desarrollo, como la lectura, el ejercicio físico o los hábitos de higiene. Frente a esto, surge *MapacheSecure*, una aplicación móvil Android que bloquea de forma inteligente las aplicaciones de entretenimiento de alto consumo, reemplazando ese tiempo con actividades interactivas y saludables. Al completarlas, los niños acumulan puntos que pueden canjear por recompensas definidas por sus padres o responsables, convirtiendo el autocontrol en una experiencia motivadora.

El presente informe describe el desarrollo de esta solución en detalle. Se expone la problemática detectada, los objetivos del proyecto, el alcance definido, los requerimientos funcionales y no funcionales, el diseño de la aplicación y las conclusiones del trabajo. El objetivo es ofrecer una visión completa del proceso de construcción de *MapacheSecure*, desde su concepción hasta su implementación técnica.

2. Contexto del proyecto

La masificación de los dispositivos móviles ha transformado profundamente la infancia. Hoy en día, los niños tienen acceso a smartphones y tablets desde edades muy tempranas, y aunque esto les abre puertas al aprendizaje y la creatividad, también los expone a un entorno diseñado para mantenerlos enganchados el mayor tiempo posible. Las plataformas de entretenimiento digital utilizan algoritmos de recomendación y sistemas de recompensa inmediata que resultan especialmente efectivos en cerebros en desarrollo, generando patrones de uso difíciles de controlar sin ayuda externa.

La idea detrás de MapacheSecure surgió de una observación cotidiana: en redes sociales es común ver videos donde niños reaccionan con crisis emocionales intensas al momento de que se les retira un dispositivo, lo que evidencia una dependencia que va más allá del entretenimiento. Esta situación motivó al equipo a buscar una solución que no solo limite el tiempo de pantalla, sino que convierta ese tiempo de desconexión en una experiencia positiva y formativa para el niño.

El público objetivo son niños de entre 5 y 15 años cuyos padres o tutores buscan herramientas concretas para establecer límites saludables sin generar conflictos. A diferencia de un simple bloqueo, MapacheSecure propone un sistema de desafíos y recompensas que hace que el niño participe activamente en la gestión de su propio tiempo.

En cuanto al estado del arte, existen aplicaciones que abordan parcialmente este problema. Forest permite establecer temporizadores de concentración usando la metáfora de plantar árboles. Freedom y Opal bloquean aplicaciones específicas por períodos definidos. StayFocusd limita el tiempo en sitios web. Sin embargo, ninguna de estas soluciones combina el bloqueo efectivo con actividades de reemplazo, un sistema de recompensas personalizable por los padres y una experiencia gamificada orientada específicamente a niños. La siguiente tabla resume esta comparación:

Forest

Es una aplicación de productividad que usa la metáfora de plantar un árbol para mantener el enfoque.



Imagen 1 "Interfaz de usuario y mecánica de gamificación ambiental de la aplicación Forest"

- **Funcionalidad:** Actúa como un temporizador de concentración donde el usuario "planta" un árbol que muere si sale de la aplicación.
- **Ventajas:** Posee una interfaz sumamente simple, intuitiva y visualmente atractiva, lo que facilita su adopción inicial.
- **Desventajas y Limitaciones:** Su principal debilidad es que no ejerce un bloqueo real sobre el dispositivo. Es una herramienta diseñada para la autogestión de adultos; un niño puede simplemente cerrar la app y el "castigo" de ver el árbol morir no es suficiente para frenar la dependencia.
- **Diferencia de MapacheSecure:** Nosotros aplicamos un bloqueo real del sistema y obligamos a completar actividades para desbloquearlo, no solo un aviso visual.

Freedom

Es una herramienta robusta diseñada para eliminar distracciones bloqueando el acceso a internet y aplicaciones específicas en múltiples dispositivos.



Imagen 2: "Interfaz de configuración de bloqueos y sesiones en Freedom"

Funcionalidad: Permite programar sesiones de bloqueo total para aplicaciones y sitios web seleccionados.

Ventajas: Es una solución multiplataforma (funciona en PC, Mac y móviles).

Desventajas: Es puramente restrictiva; al bloquear, no ofrece ninguna actividad alternativa al usuario, lo que genera frustración en niños.

Diferencia de MapacheSecure: A diferencia del "muro ciego" de Freedom, MapacheSecure propone actividades de reemplazo lúdicas que transforman la restricción en una oportunidad de aprendizaje.

Opal

Es una herramienta moderna enfocada en la productividad y en reducir el tiempo de exposición a pantallas mediante un enfoque de bienestar digital.



Imagen 3: "Dashboard de productividad y gestión de tiempo de la aplicación Opal"

Funcionalidad: Implementa bloqueos de aplicaciones específicos basados en horarios programados por el usuario.

Ventajas: Cuenta con una interfaz de usuario sumamente amigable, estética y moderna.

Desventajas: No está orientada al público infantil; su diseño y lenguaje están pensados para adultos que buscan mejorar su productividad laboral.

Diferencia de MapacheSecure: A diferencia de Opal, que es una herramienta de autogestión, MapacheSecure incluye un sistema de recompensas diseñado específicamente para motivar a los niños, haciendo que el cumplimiento de metas sea un juego y no solo una estadística de tiempo.

Stay Focused

Es una herramienta clásica de productividad que busca reducir la navegación excesiva limitando el tiempo que el usuario puede pasar en sitios web distractores.



Imagen 4: "Interfaz de configuración de límites de navegación en la herramienta Stay Focused"

Funcionalidad: Establece límites de tiempo diarios para sitios web específicos; una vez agotado el tiempo, el acceso se bloquea automáticamente.

Ventajas: Es extremadamente fácil de configurar y muy directa en su función.

Desventajas: Su enfoque principal es el navegador; no tiene un control integral sobre las aplicaciones instaladas en el sistema móvil, lo que permite al niño simplemente cambiar de app.

Diferencia de MapacheSecure: MapacheSecure ofrece un control total del dispositivo, no solo del navegador. Actúa sobre cualquier aplicación de entretenimiento, garantizando que el límite no se pueda evadir saltando de la web a una aplicación.

MapacheSecure

Tras analizar las soluciones actuales, se identifica una oportunidad para crear una herramienta que no sea puramente restrictiva ni fácilmente eludible por un menor.



Imagen 5: *"Identidad visual y concepto central del sistema MapacheSecure"*

- Qué hace: Implementa un sistema de bloqueo inteligente integrado con desafíos interactivos (cognitivos, físicos y de orden) y un catálogo de recompensas configurables.
- Ventajas: Es una solución integral y gamificada que no solo limita el tiempo, sino que educa y motiva al niño a realizar actividades saludables.
- Desventajas: Actualmente se encuentra en fase de desarrollo y prototipado técnico.
- Diferenciador principal: MapacheSecure transforma el concepto de "control parental" de una prohibición a un intercambio de valor positivo, donde el acceso al entretenimiento es una recompensa obtenida mediante el esfuerzo personal.

3. Descripción del problema u oportunidad

3.1. Descripción del Problema

Actualmente, los niños en edad escolar enfrentan una crisis de sobreexposición digital. El uso de dispositivos electrónicos ha dejado de ser una actividad complementaria para convertirse en el eje central del día a día, con promedios que superan las 10 horas diarias frente a una pantalla. Esta situación provoca una dependencia progresiva que desplaza actividades vitales como el juego físico, la lectura y la interacción social presencial.

El núcleo del problema no es solo el tiempo, sino la naturaleza del contenido: las aplicaciones de entretenimiento y redes sociales operan bajo una lógica de gratificación instantánea. Mediante algoritmos de retención diseñados para explotar la liberación de dopamina, los dispositivos capturan la atención del menor de forma compulsiva, dificultando su capacidad de autorregulación y generando una resistencia activa frente a cualquier intento de mediación parental.

3.2. Análisis de causas y efectos

Para comprender la profundidad de este fenómeno, es necesario desglosar los factores que lo alimentan y las consecuencias sistémicas que genera en el desarrollo infantil.

Análisis de Causas (Raíces del problema):

- **Diseño Afectivo y Adictivo:** Las plataformas digitales utilizan mecánicas de juego y recompensas variables que resultan especialmente efectivas en cerebros en desarrollo, los cuales aún no poseen la madurez cortical necesaria para el control de impulsos.
- **Ineficacia de herramientas actuales:** La mayoría de las soluciones de control parental son punitivas o meramente restrictivas. Al no ofrecer una alternativa atractiva, el bloqueo se percibe como una agresión, lo que fomenta el conflicto familiar.
- **Erosión de las rutinas analógicas:** La facilidad de acceso a la entretención digital ha desplazado las rutinas de hogar, haciendo que el esfuerzo requerido para leer un libro o hacer ejercicio parezca "aburrido" en comparación con la estimulación rápida de una pantalla.

Análisis de Efectos (Impacto en el menor):

- **Deterioro de la Salud Física:** El sedentarismo prolongado aumenta el riesgo de obesidad infantil y problemas posturales. Además, la luz azul de las pantallas altera los ciclos circadianos, provocando trastornos del sueño y fatiga crónica.

- **Dispersión Cognitiva:** La exposición a estímulos fragmentados y rápidos reduce la capacidad de atención sostenida, lo que se traduce en un bajo rendimiento escolar y dificultades para procesar información compleja.
- **Tensión en el Clima Familiar:** La falta de una herramienta mediadora convierte el límite digital en una fuente constante de peleas, debilitando la autoridad parental y el bienestar emocional del hogar.

3.3. Representación Gráfica del Problema

A continuación, se presentan los diagramas técnicos que sintetizan la relación entre los factores antes mencionados.

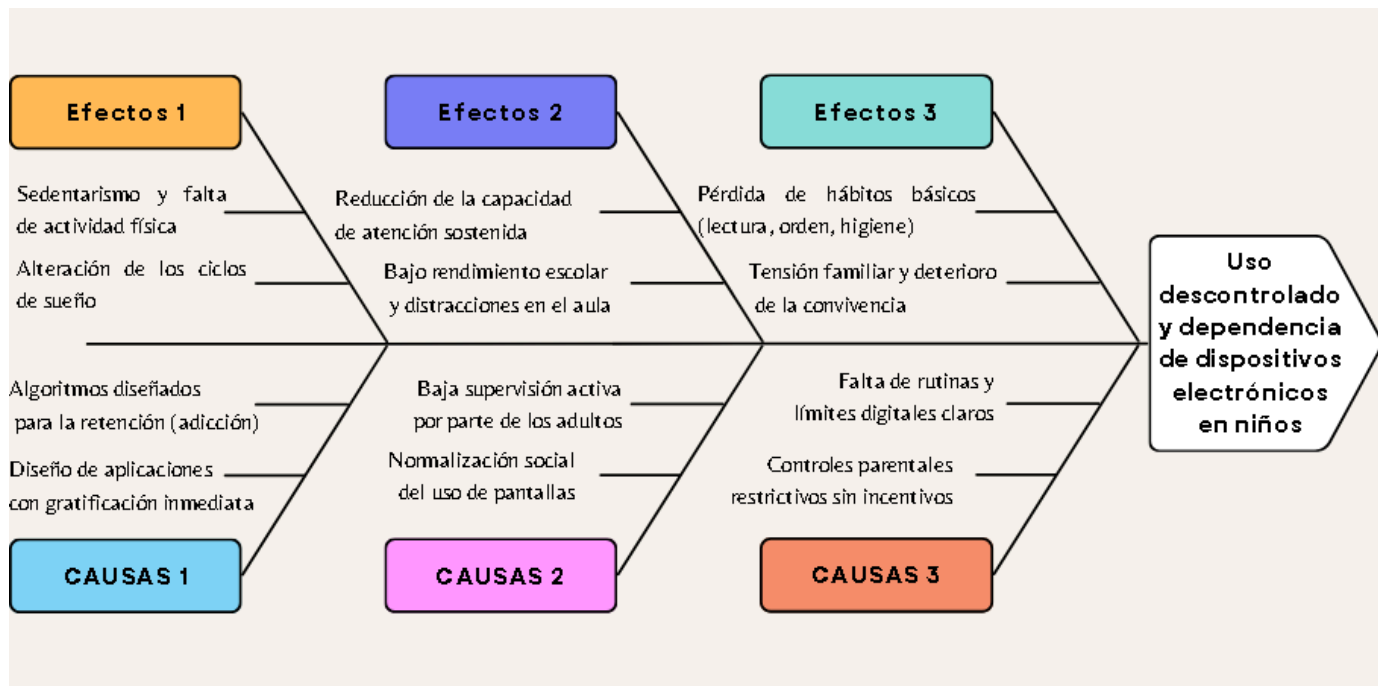
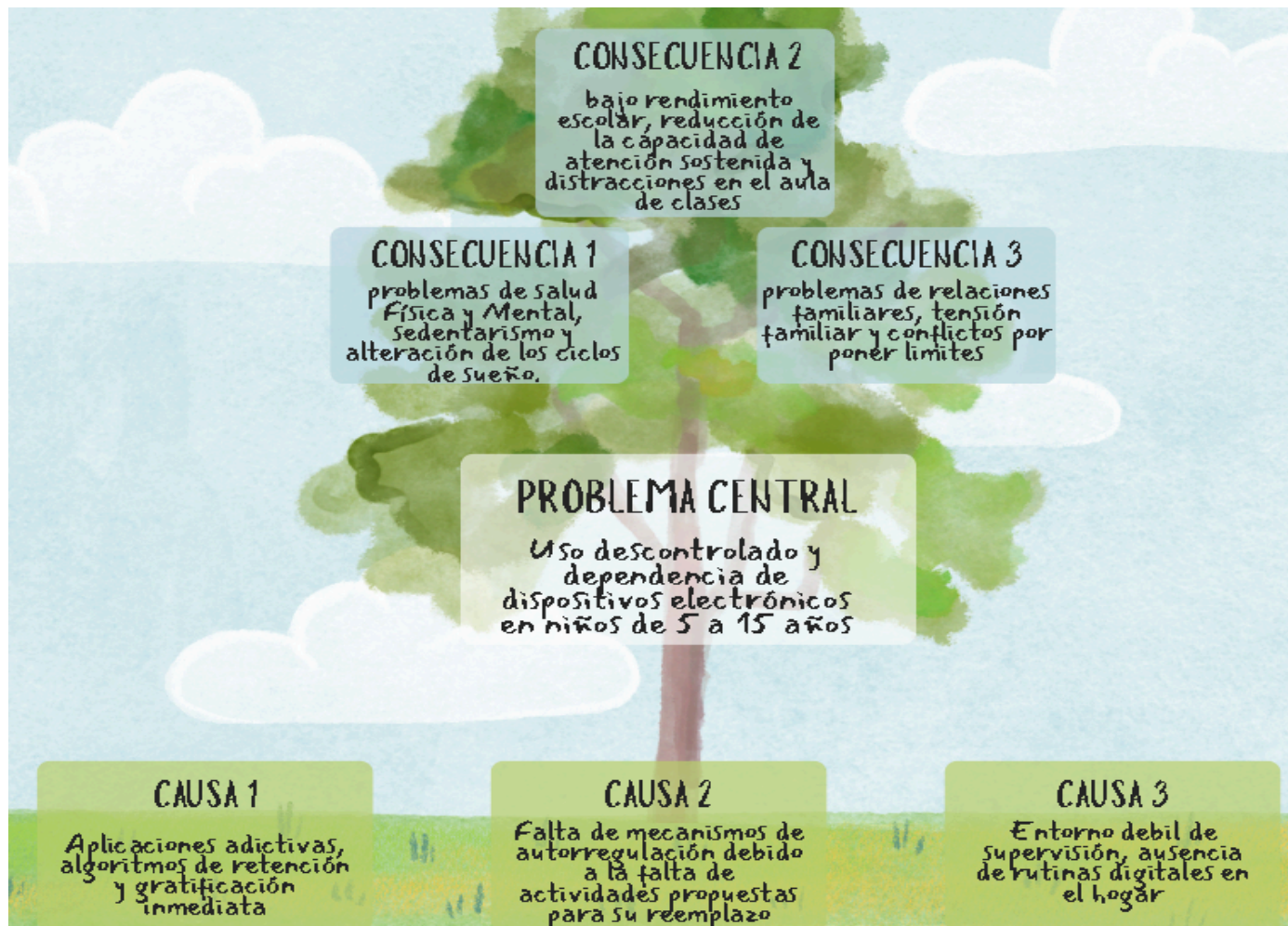


Diagrama de Ishikawa

El diagrama resume el corazón del problema que queremos resolver con MapacheSecure. No se trata solo de niños usando pantallas, sino de un círculo vicioso: las aplicaciones están diseñadas para atrapar su atención, y al no haber rutinas claras o una supervisión constante, esa distracción se convierte rápidamente en una dependencia.

Esto pega fuerte en tres áreas: la salud (por el sedentarismo y el mal sueño), el colegio (porque pierden la capacidad de concentrarse) y la familia (por las peleas al intentar poner límites).

Aquí es donde vemos nuestra oportunidad: el problema no es solo que falta control, sino que falta motivación. En lugar de solo prohibir, el análisis nos muestra que necesitamos transformar ese tiempo perdido en una experiencia positiva de autocontrol y premios.



Árbol del problema

El Árbol de Problemas revela que la dependencia infantil no es un evento aislado, sino un ciclo de retroalimentación negativa. Este ecosistema de conflicto se sostiene sobre tres pilares críticos:

Origen Técnico (Raíces): Aplicaciones diseñadas con algoritmos de retención y gratificación inmediata que anulan la capacidad de autorregulación del menor.

Impacto Sistémico (Copa): La carencia de límites técnicos deriva en sedentarismo, dispersión cognitiva en el aula y un constante desgaste de la autoridad parental.

Oportunidad de Intervención (Tronco): El problema central no es la falta de prohibición, sino la falta de motivación. Actualmente, el bloqueo es un "muro ciego" que genera fricción familiar.

MapacheSecure rompe este ciclo sustituyendo la restricción rígida por un intercambio de valor. Transformamos el tiempo de pantalla en una recompensa obtenida mediante desafíos, convirtiendo el límite en un logro personal en lugar de un castigo.

4. Descripción del mercado objetivo

4.1. Definición de los Usuarios y Clientes

MapacheSecure opera en un entorno familiar donde coexisten dos figuras clave con motivaciones distintas pero unidas por un problema común: el uso desmedido de la tecnología.

1. Usuario Final (Niños y Adolescentes): Menores de entre 5 y 15 años que han crecido en un entorno digital nativo. Para ellos, el dispositivo móvil es su principal ventana al ocio, la socialización y la información. Sin embargo, carecen de la madurez necesaria para gestionar el tiempo de uso, lo que genera frustración inmediata cuando se les impone un límite externo.
2. Cliente Decisor (Padres y Tutores): Adultos de entre 28 y 50 años, generalmente con jornadas laborales intensas, que valoran la tecnología pero se sienten sobrepasados por la gestión del tiempo de pantalla de sus hijos. Buscan una solución que no los obligue a ser el "policía" constante de la casa.

4.2. Necesidades y Dolores del Mercado

El mercado actual demanda una transición desde el control punitivo hacia la mediación tecnológica.

- Necesidades: El niño necesita estímulos que no sean frustrantes y que validen su esfuerzo. El padre necesita "paz mental", una herramienta automatizada que aplique reglas justas y preestablecidas sin necesidad de una confrontación verbal diaria.
- Dolores: La fricción familiar constante, el sentimiento de culpa en los padres por "solo prohibir", y el aburrimiento reactivo en los niños (no saber qué hacer una vez bloqueada la pantalla).

4.3. Escenario de Uso y Perfiles de Usuario

El escenario principal es el hogar, generalmente en ese bloque crítico que va desde que llegan del colegio hasta la hora de dormir. Es el momento donde los padres están cocinando o descansando y los niños buscan el refugio de las pantallas.

MapacheSecure aparece justo en ese momento de fricción. En lugar de que el padre tenga que ir a quitar el dispositivo físicamente, la aplicación toma el relevo de forma automática. El niño se encuentra con un desafío que lo saca del "trance" del video o el juego y lo invita a pensar. Si lo logra, siente la satisfacción de haber cumplido un objetivo y desbloquea sus recompensas (como puntos o más minutos), transformando un momento que solía ser de pelea en un momento de logro personal.



Mateo
10 años · Usuario principal

Hijo

INTERESES

- YouTube
- Retos
- Videojuegos

COMPORTAMIENTO

Podría pasar horas viendo contenido sin límite. No percibe el tiempo al estar en pantalla.

NECESIDAD PRINCIPAL

Que el bloqueo no se sienta como un muro, sino como un nivel de juego que puede superar.

PUNTO DE DOLOR

Los conflictos diarios con su mamá por el uso del celular le generan estrés y tensión.

LO QUE MÁS QUIERE

"Que mi mamá no se enoje conmigo todos los días por lo mismo."

Motivación

Frustración

Autonomía digital

Perfil de usuario 1



Elena
36 años · Usuario secundariol

Mamá

CONTEXTO

Trabaja mucho y llega a casa buscando tranquilidad. Poco tiempo para gestionar conflictos digitales.

HERRAMIENTA CLAVE

- Control parental
- Límites de tiempo
- Retos educativos

NECESIDAD PRINCIPAL

Una herramienta que le permita delegar la decisión del tiempo de pantalla sin discutir cada día.

PUNTO DE DOLOR

Los conflictos diarios por el celular rompen la armonía familiar justo cuando más necesita descansar.

LO QUE MÁS QUIERE

"Hijo, ya sabes que MapacheSecure decide el tiempo de hoy." — y que eso sea suficiente.

META DE FONDO

Saber que si su hijo usa el celular, al menos está pasando por retos que le aportan algo. Recuperar la armonía en la mesa.

Estrés diario

Confianza en el tech

Motivación de control

Perfil de usuario 2



Mapa de Stakeholders

Stakeholders Internos

Corresponden a los actores principales que interactúan directamente con el sistema Raccu y participan activamente en el funcionamiento de la aplicación. Este grupo está compuesto por los padres/tutores y los niños o adolescentes, quienes utilizan diariamente la plataforma para gestionar el control parental, los desafíos y el sistema de recompensas.

Stakeholders Directos

Son actores técnicos y operativos que permiten el funcionamiento del sistema y participan directamente en su desarrollo e infraestructura tecnológica. Incluye al equipo de desarrollo y a los servicios tecnológicos utilizados por la aplicación, como Android/Google, Firebase, Supabase y Render.

Stakeholders Indirectos

Corresponden a actores externos que no interactúan directamente con la aplicación, pero que influyen en el contexto educativo, social y de bienestar asociado al uso responsable de dispositivos digitales. En este grupo se encuentran colegios, educadores, psicólogos infantiles, pediatras y organismos relacionados con la educación.

5. Objetivo general y objetivos específicos

5.1. Objetivo General

Desarrollar una aplicación móvil para la plataforma Android que permita la autorregulación digital de niños y adolescentes entre 5 y 15 años, mediante un sistema de bloqueo inteligente de aplicaciones de alto consumo de entretenimiento, integrando desafíos interactivos y una economía de fichas (recompensas personalizables) para fomentar hábitos digitales saludables y reducir la dependencia a las pantallas.

5.2. Objetivos Específicos

Para lograr el objetivo general, el equipo se propone cumplir con los siguientes hitos técnicos y funcionales:

- Implementación del Módulo de Intercepción: Desarrollar un servicio en segundo plano que utilice la *Usage Stats API* de Android para detectar aplicaciones en primer plano y activar la pantalla de desafío de forma automática y funcional antes de la semana 8.
- Desarrollo del Motor de Desafíos: Crear un sistema que integre al menos tres categorías de actividades (cognitiva, física y de orden/hogar), con un mínimo de dos actividades funcionales por categoría, disponible para el usuario final antes de la semana 10.
- Gestión de Panel Parental y Autenticación: Diseñar e integrar un panel de control para el tutor que permita la configuración de horarios, selección de retos y definición de premios, protegido mediante *Supabase Auth*, operativo antes de la semana 10.
- Infraestructura de Backend y Cloud: Construir y desplegar una API funcional utilizando FastAPI y Supabase (PostgreSQL) para la persistencia de perfiles, logs de uso y sincronización de puntos, antes de la semana 9.
- Sistema de Notificaciones Push: Integrar *Firebase Cloud Messaging* (FCM) para el envío de alertas de logros desbloqueados al menor y resúmenes diarios de actividad al padre, verificado en dispositivos físicos antes de la semana 11.

6. Situación inicial del cliente (AS-IS)

6.1. Descripción del Proceso Actual

En la actualidad, la gestión del tiempo de pantalla en el hogar es un proceso 100% manual, reactivo y generador de conflictos. El ciclo habitual comienza cuando el niño accede al dispositivo móvil sin restricciones técnicas previas. Al no existir un sistema de alerta o bloqueo automático, el menor pierde la noción del tiempo debido al diseño de gratificación inmediata de las aplicaciones de entretenimiento.

El control depende exclusivamente de la vigilancia visual del adulto. El proceso se rompe en los siguientes puntos críticos:

- **Detección Tardía:** El padre suele notar el exceso de uso cuando ya han pasado varias horas, lo que genera una reacción de culpa en el adulto y frustración en el niño.
- **Intervención Física:** La restricción ocurre mediante la retirada física del dispositivo o el apagado abrupto, lo que se percibe como una agresión arbitraria.
- **Negociación Incesante:** Se inicia un ciclo de "cinco minutos más" que desgasta la autoridad parental y no educa en la autorregulación.
- **Falta de Registro:** No existe un dato real de cuánto tiempo se consumió ni en qué aplicaciones, imposibilitando la toma de decisiones informadas.

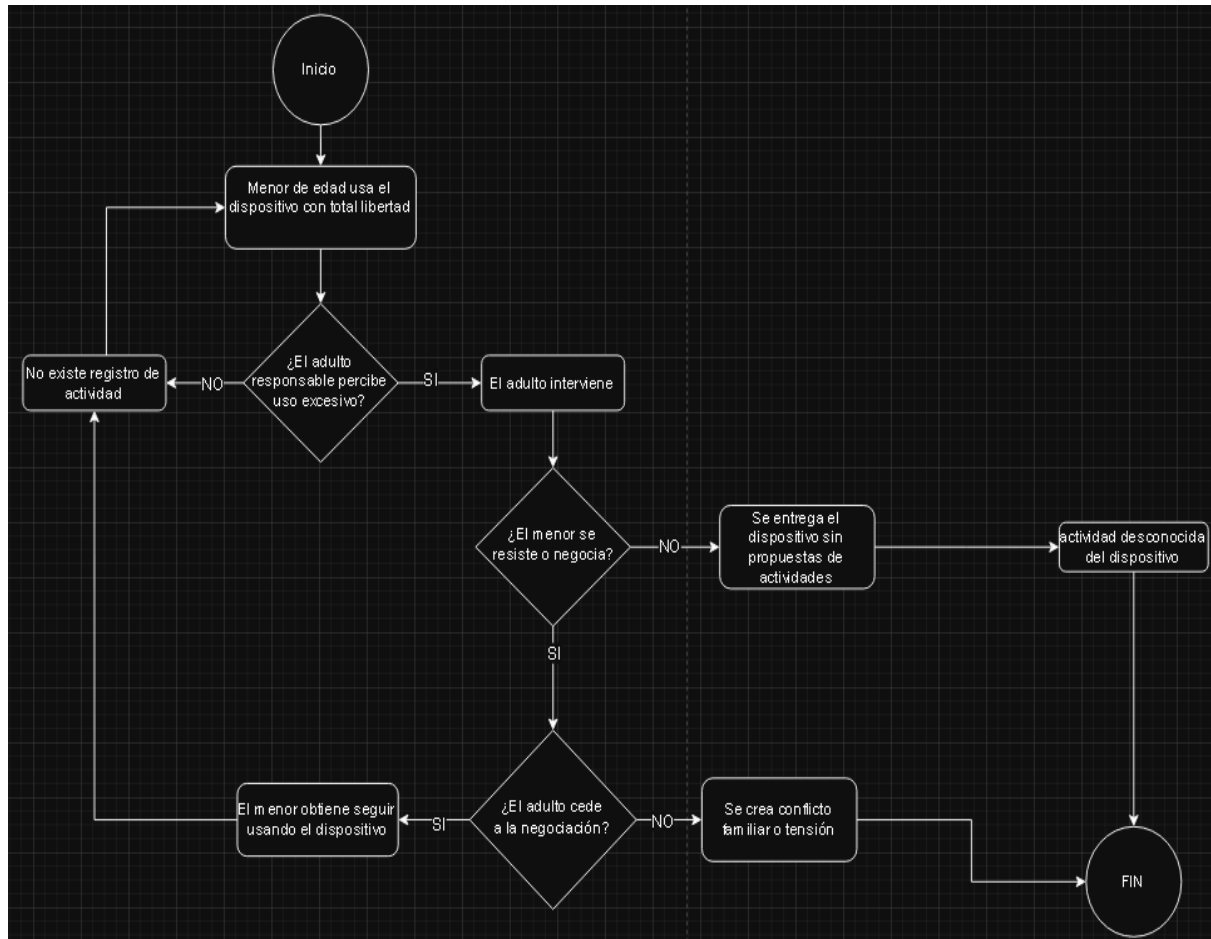
6.2. Puntos de Dolor Identificados

Este proceso "analógico" genera cuatro problemas graves:

1. **Fricción Familiar:** El clima del hogar se tensa diariamente por la "guerra de las pantallas".
2. **Inexistencia de Alternativas:** Al quitar el celular, el niño queda en un vacío de actividad, lo que dispara la irritabilidad.
3. **Carencia de Incentivos:** La única herramienta es la prohibición; no hay un premio por dejar el teléfono voluntariamente.
4. **Dependencia del Adulto:** El sistema falla si el padre está ocupado, trabajando o descansando.

6.3. Diagrama de Flujo del Proceso Actual (AS-IS)

A continuación, se representa gráficamente la secuencia de eventos y las decisiones que hoy en día terminan en conflicto familiar.



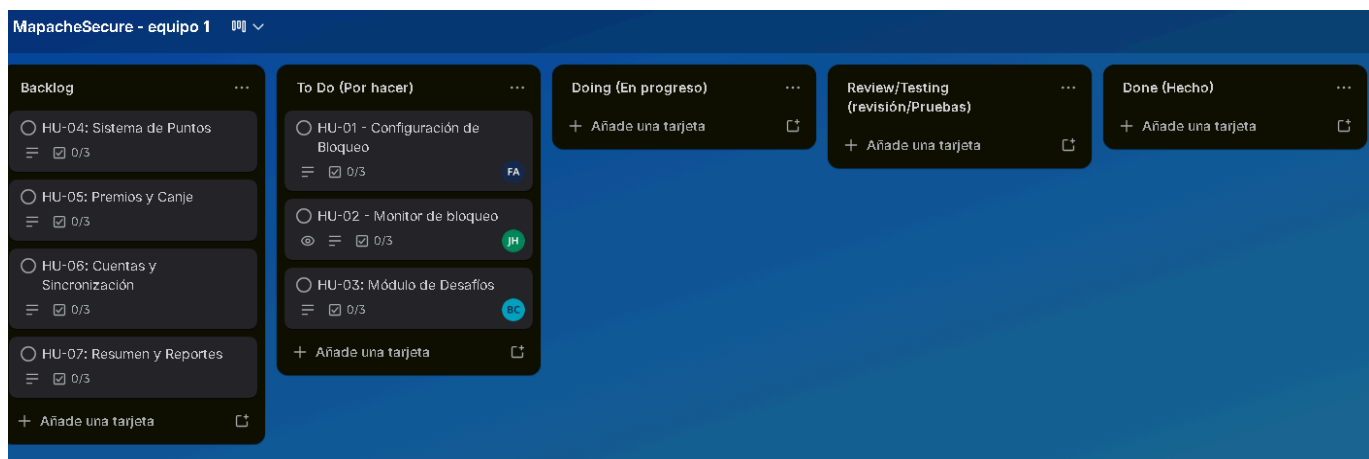
Flujo de interacción manual y puntos de fricción en el control parental tradicional.

7. Planificación

7.1. Metodología de Gestión: SCRUM y Trello

Para asegurar el cumplimiento de los hitos en el plazo establecido, el equipo ha implementado un marco de trabajo Ágil (Scrum). La gestión visual y el control de flujos se centralizan en un tablero de Trello, el cual permite organizar el trabajo mediante Historias de Usuario (HU) distribuidas en columnas de estado: *Backlog*, *To Do* (Por hacer), *Doing* (En progreso), *Review/Testing* (Revisión) y *Done* (Hecho).

Como se observa en la Imagen, el equipo ha desglosado las funcionalidades críticas (HU-01 a HU-07). Actualmente, el foco está en la configuración del monitor de bloqueo y el módulo de desafíos, permitiendo una trazabilidad total del avance y una rápida detección de cuellos de botella técnicos.



<https://trello.com/invite/b/69ca8b5f9fe047ba0b854111/ATTI62d76a73cad5160a7aaff5eeef843617690313D6/mapachesequre-equipo-1>

7.2. Organización del Equipo y Roles Críticos

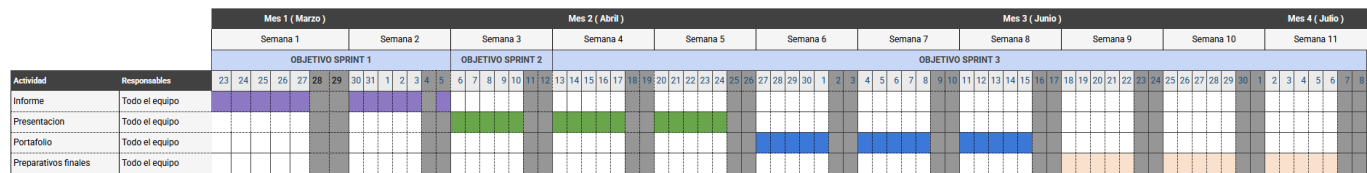
La distribución de responsabilidades asegura que cada capa de la aplicación (Mobile, API y Datos) tenga un responsable directo, aunque la ejecución de los entregables principales es colaborativa:

- Benjamín Castillo (Líder de Proyecto, DBA y Analista Funcional): Responsable de la arquitectura de datos en Supabase, el diseño de la lógica de negocio y la administración del tablero Trello.
- Javier Vieytes (Desarrollador Backend y Especialista Técnico): Encargado de la construcción de la API en FastAPI, la integración con Firebase y el despliegue en la plataforma Render.
- Fernanda Arraño (QA, Testing y Desarrolladora Frontend Mobile): Responsable de la interfaz de usuario en Android, la lógica de intercepción de aplicaciones y el aseguramiento de la calidad.

7.3. Cronograma de Actividades (Carta Gantt)

El proyecto se desarrolla a lo largo de 11 semanas, organizadas en tres Sprints principales de trabajo. A continuación, se presenta la representación gráfica de la planificación temporal para los entregables clave del semestre:

MapacheSecure



<https://docs.google.com/spreadsheets/d/1Re5Rkkl2143U-21JFFkwfK4Rm3SaJ8x169OaGsRKOZk/edit?gid=207134840#gid=207134840>

8. Servicios Cloud a utilizar (IaaS / PaaS / SaaS)

Para garantizar la disponibilidad y escalabilidad de **MapacheSecure**, se ha seleccionado un stack tecnológico basado en servicios en la nube que permiten un despliegue ágil y sin costos de infraestructura inicial:

- **Render (PaaS):** Se utiliza para el hosting del backend desarrollado en FastAPI. Render permite la integración continua con GitHub, desplegando automáticamente cada cambio en el código y proporcionando certificados SSL (HTTPS) de forma nativa.
- **Supabase (DBaaS/PaaS):** Actúa como nuestra base de datos. Además de almacenar perfiles y puntos, se utiliza su servicio de autenticación para proteger el panel de control parental mediante tokens seguros.
- **Firestore Cloud Messaging (SaaS):** Es el servicio encargado de la mensajería push. Permite enviar alertas en tiempo real desde el backend hacia el dispositivo Android del niño (logros) y del padre (reportes de uso).

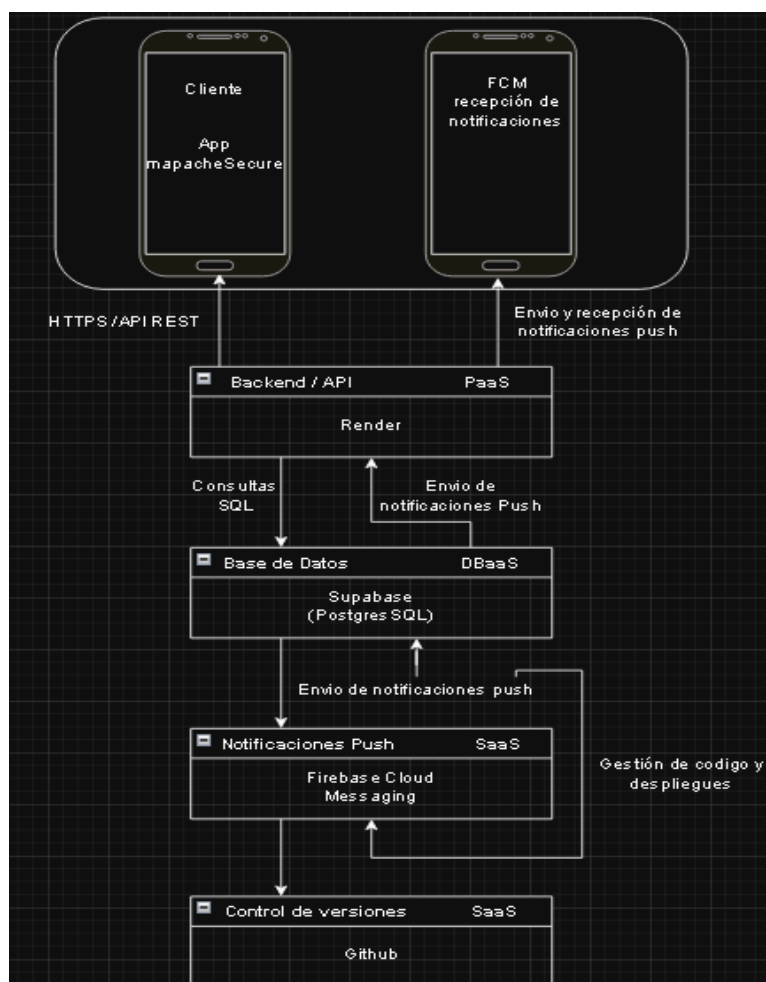


Diagrama de arquitectura servicio cloud

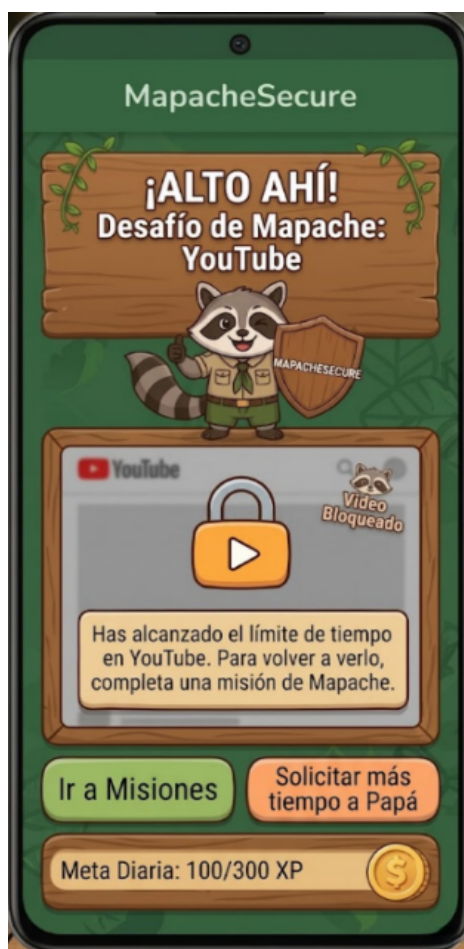
9. Conceptualización (Solución propuesta – Versión preliminar)

Con **MapacheSecure** instalado, el proceso de uso de dispositivos cambia completamente. Cuando el niño intenta abrir una aplicación de entretenimiento bloqueada, el sistema intercepta ese intento en tiempo real y muestra una pantalla de desafío en su lugar. El niño debe completar una actividad para poder acceder. Al terminarla, gana puntos que acumula en su perfil. Esos puntos los puede canjear por las recompensas que su padre configuró previamente.

Funcionalidades Principales:

1. Módulo de Intercepción y Bloqueo Inteligente:

El sistema utiliza un servicio en segundo plano que monitorea el "Usage Stats API" de Android. Cuando detecta que una aplicación está marcada como "entretenimiento" (ej. TikTok, YouTube, Roblox) pasa a primer plano, la app superpone inmediatamente una pantalla de bloqueo personalizada.



Bloqueo inteligente

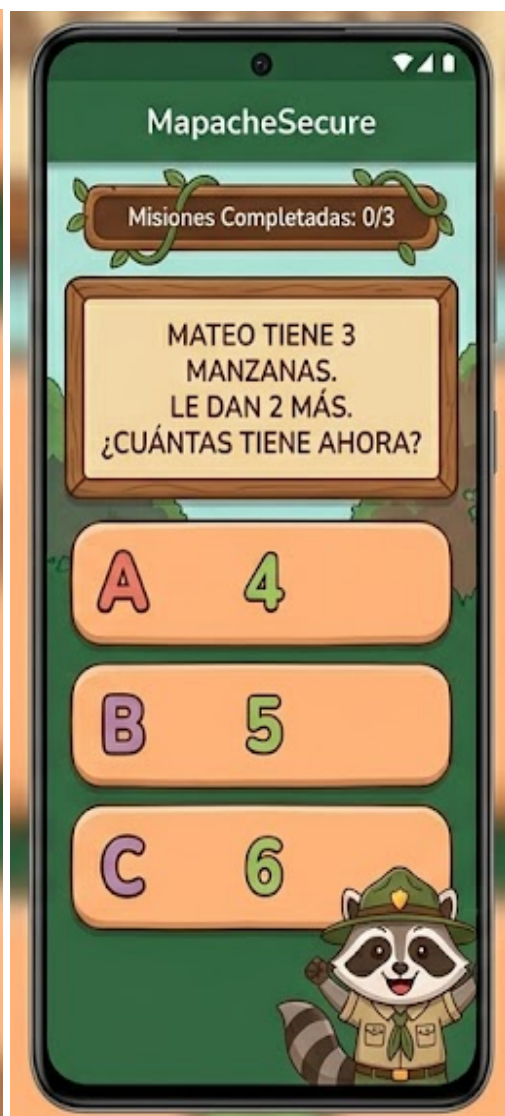
2. Motor de Desafíos Multimodal:

Es el núcleo pedagógico de la app. No se limita a un solo tipo de tarea, sino que ofrece tres categorías para evitar la fatiga y fomentar el desarrollo integral:

- **Cognitivos:** Micro-lecturas con comprensión, trivias o rompecabezas lógicos.
- **Físicos:** Retos detectados mediante sensores (acelerómetro) como realizar 20 saltos.
- **De Orden/Hogar:** Tareas como hacer la cama, enviando una foto como evidencia.



Motor de desafíos



Desafío cognitivo



Evidencia del desafío completado

3. Sistema de Economía de Fichas (Puntos y Recompensas):

Basado en psicología conductual, cada desafío superado otorga "MapachePoints". El sistema incluye un catálogo de recompensas configurables donde el niño canjea sus puntos por beneficios (ej. "30 minutos extra de juego" o "elegir la película").



Tienda de recompensas

4. Panel de Control Parental (Gestor de Reglas):

Interfaz exclusiva para el adulto, protegida por autenticación biométrica o PIN. Permite:

- Establecer "Horas de Enfoque" (bloqueo automático).
- Seleccionar qué aplicaciones específicas requieren desafíos para abrirse.
- Personalizar la dificultad de los retos según la edad del menor.



Panel de control parental

5. Sistema de Notificaciones y Reportes:

Mediante **Firestore Cloud Messaging (FCM)**, el sistema mantiene un flujo de comunicación constante:

- **Para el niño:** Alertas de "logro desbloqueado" o recordatorios de desafíos pendientes.
- **Para el padre:** Notificaciones sobre el cumplimiento de metas y un reporte diario resumido que indica el tiempo total de pantalla y el nivel de cumplimiento de los desafíos propuestos.



Panel del niño



Panel del padre

6. Arquitectura de Sincronización Cloud:

Utiliza un backend desarrollado en FastAPI que actúa como puente entre el dispositivo del niño y el del padre. Toda la progresión, puntos acumulados y logs de intentos de apertura se sincronizan en tiempo real con una base de datos PostgreSQL en Supabase. Esto garantiza que, aunque se cambie de dispositivo, el historial y los puntos se mantengan íntegros.

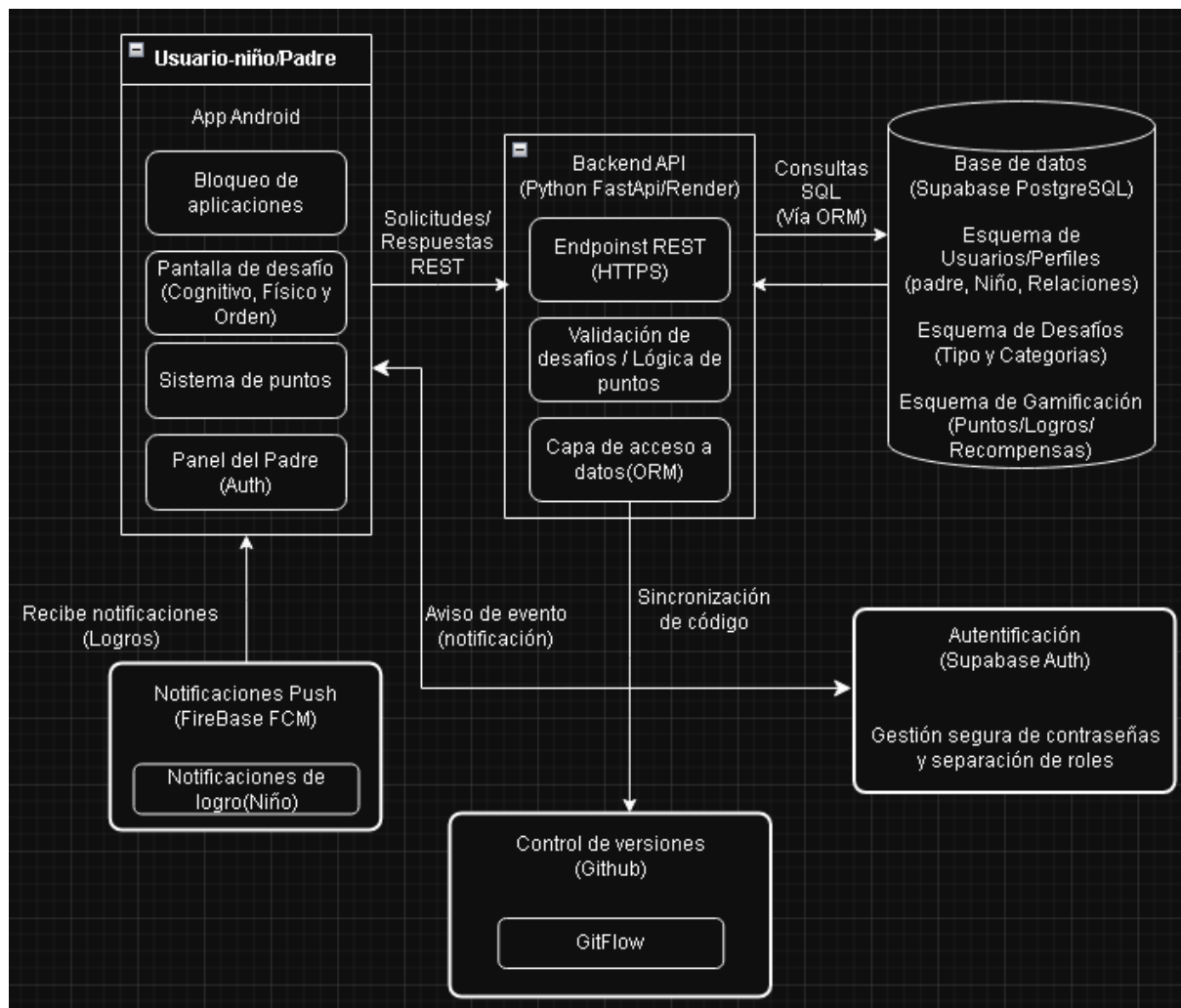


Diagrama de arquitectura proceso de aplicación

10. Alcance, supuestos y restricciones

Alcance del Proyecto

MapacheSecure contempla el desarrollo de una aplicación móvil Android orientada al control parental y la autorregulación digital infantil mediante un enfoque gamificado. El sistema incluirá un mecanismo de bloqueo inteligente de aplicaciones de entretenimiento utilizando la UsageStats API de Android, permitiendo detectar aplicaciones en primer plano e interceptar su uso mediante desafíos interactivos antes de autorizar el acceso.

La solución incorpora un motor de desafíos compuesto por actividades cognitivas, físicas y de orden/hábitos del hogar, diseñadas para reemplazar el tiempo de pantalla por actividades saludables y educativas. Cada desafío completado otorgará puntos acumulables dentro de un sistema de economía de fichas, los cuales podrán ser canjeados por recompensas configuradas previamente por el padre o tutor.

El sistema considera además:

- Panel parental protegido mediante autenticación segura con JWT.
- Gestión de aplicaciones bloqueadas por horarios.
- Sistema de gamificación con avatar evolutivo "Raccu".
- Personalización de foto de perfil del hijo.
- Generación de desafíos con IA.
- Backend REST desarrollado con FastAPI.
- Persistencia de datos mediante Supabase/PostgreSQL.
- Notificaciones push utilizando Firebase Cloud Messaging.
- Emails transaccionales mediante Resend.
- Despliegue cloud en Render.
- Landing page informativa publicada en raccu.cl.

El alcance contempla un prototipo funcional orientado a validación técnica y académica, no una solución comercial final lista para producción masiva.

Exclusiones del Proyecto

El proyecto no contempla el desarrollo de:

- Versión para iOS o multiplataforma fuera de Android.
- Compatibilidad con dispositivos Android inferiores a la versión 8.0.
- Compatibilidad específica para tablets o Smart TVs.
- Módulo de pagos online o monetización.
- Marketplace de recompensas digitales automatizadas.
- Plataforma web independiente para el panel parental.

- Soporte multilenguaje.
- Sincronización offline o funcionamiento sin conexión a internet.
- Integración con servicios externos de control parental de terceros.

La funcionalidad de generación dinámica de desafíos mediante IA queda únicamente como propuesta futura asociada al Sprint 3.

Entregables

Los entregables oficiales del proyecto corresponden a:

- Informe técnico completo del sistema.
- Repositorio GitHub del frontend y backend.
- Aplicación Android instalable (.apk).
- Backend desplegado y funcional en Render.
- Base de datos operativa en Supabase.
- Landing page publicada en raccu.cl.
- Diagramas técnicos y documentación de arquitectura.
- Evidencias funcionales y pruebas del sistema.
- Configuración básica de entorno cloud y variables de entorno.

Supuestos

Para el correcto funcionamiento y desarrollo del proyecto se asume que:

- Los dispositivos de prueba utilizan Android 8.0 o superior.
- El equipo dispone de acceso estable a internet durante el desarrollo.
- Los servicios gratuitos de Render y Supabase mantendrán disponibilidad durante el semestre.
- El usuario padre posee conocimientos básicos para configurar la aplicación.
- El dispositivo del hijo mantiene habilitados los permisos necesarios del sistema (Usage Access, superposición, notificaciones).
- Los niños no tienen acceso a las credenciales administrativas del padre.
- Firebase FCM mantendrá compatibilidad con la configuración Android utilizada.
- La cantidad de usuarios durante la etapa académica será reducida y compatible con los límites gratuitos de infraestructura cloud.

Restricciones

El proyecto presenta las siguientes restricciones técnicas y operativas:

- Duración máxima de 11 semanas organizadas en tres sprints Scrum.
- Equipo reducido de 3 integrantes sin dedicación exclusiva.

- Ausencia de presupuesto para infraestructura o APIs de pago.
- Dependencia directa de servicios cloud externos gratuitos.
- Limitaciones impuestas por Android respecto al monitoreo de aplicaciones en segundo plano.
- Restricciones de consumo y suspensión automática del plan gratuito de Render tras períodos de inactividad.
- Restricciones de permisos del sistema Android que pueden variar según fabricante del dispositivo.
- Alcance académico limitado a un MVP funcional y no a un producto comercial completo.
- Capacidad limitada de pruebas masivas en múltiples dispositivos reales.

11. Metodología de desarrollo y gestión del proyecto

Metodología Scrum

Para el desarrollo de MapacheSecure se adoptó la metodología ágil Scrum, complementada con el uso de Trello como herramienta de gestión visual del flujo de trabajo.

Scrum resulta adecuada para este proyecto por varias razones. En primer lugar, el equipo es pequeño (3 integrantes), lo que se alinea perfectamente con la estructura ligera que propone esta metodología. En segundo lugar, el proyecto tiene un plazo definido de 11 semanas dividido en sprints, lo que permite entregar avances concretos y funcionales en cada iteración en lugar de esperar al final para tener resultados visibles.

Cada sprint tiene un objetivo claro: el primero se enfocó en la planificación y diseño del sistema, el segundo en el desarrollo del núcleo técnico (backend, base de datos y módulo de bloqueo), y el tercero en la integración, pruebas y preparación del entregable final. Esta división permite al equipo detectar problemas técnicos de forma temprana y ajustar el desarrollo sin afectar el plazo total.

El tablero Trello organiza el trabajo mediante historias de usuario (HU-01 a HU-07) distribuidas en columnas de estado: Backlog, To Do, Doing, Review y Done, lo que da visibilidad total del avance a todos los integrantes en tiempo real.

A diferencia de metodologías tradicionales como el modelo en cascada, Scrum permite incorporar cambios durante el desarrollo, algo especialmente relevante en un proyecto universitario donde los requerimientos pueden ajustarse según el feedback del docente o las limitaciones técnicas que surjan en el camino.

Planificación, Seguimiento y Control Semanal

Nuestro equipo se organizará en ciclos semanales enmarcados dentro de tres sprints principales que abarcan las 11 semanas del semestre. Cada sprint tiene un objetivo concreto y medible que guía las decisiones del equipo durante ese período.

Planificación

Al inicio de cada sprint el equipo realiza una reunión de planificación donde se revisan las historias de usuario pendientes en el backlog de Trello, se estima el esfuerzo requerido para cada tarea y se asignan responsabilidades según los roles definidos: Benjamín en arquitectura de datos y lógica de negocio, Javier en backend y despliegue, y Fernanda en frontend móvil y control de calidad. Las tareas seleccionadas se mueven a la columna "To Do" del tablero, estableciendo así el compromiso de trabajo para la semana.

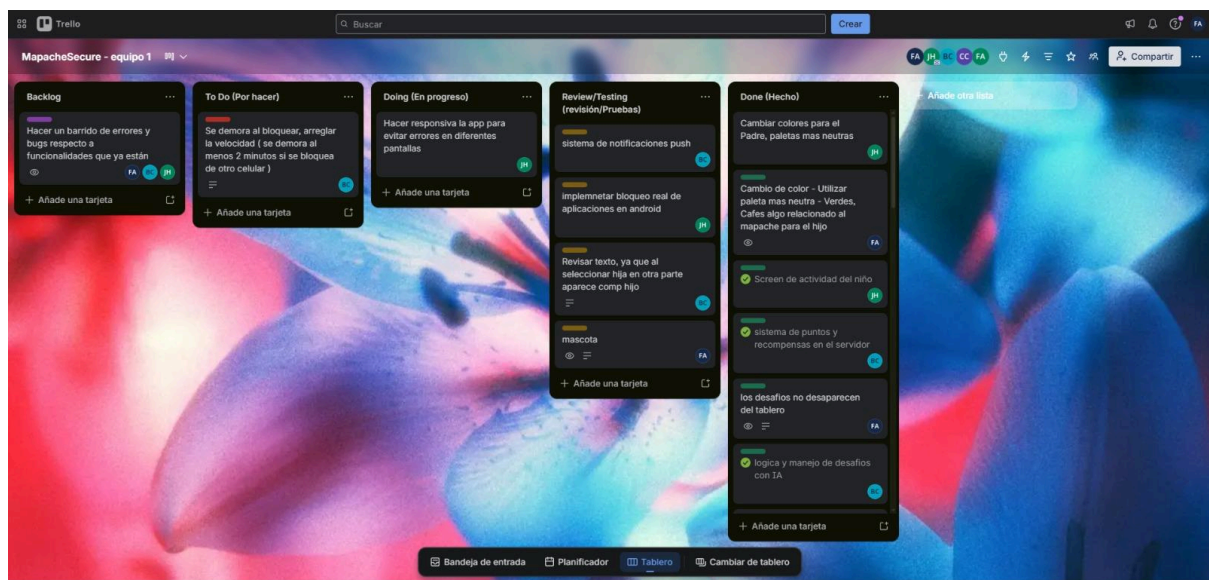
Seguimiento

Durante la semana cada integrante actualiza el estado de sus tareas en el tablero Trello, moviendo las tarjetas entre las columnas "Doing", "Review" y "Done" según el avance real.

El repositorio de GitHub actúa como registro técnico del progreso, ya que cada commit representa un avance concreto y verificable. El equipo se comunica de forma asíncrona durante la semana mediante WhatsApp para resolver bloqueos técnicos o coordinar decisiones que afecten al trabajo de otro integrante.

Control

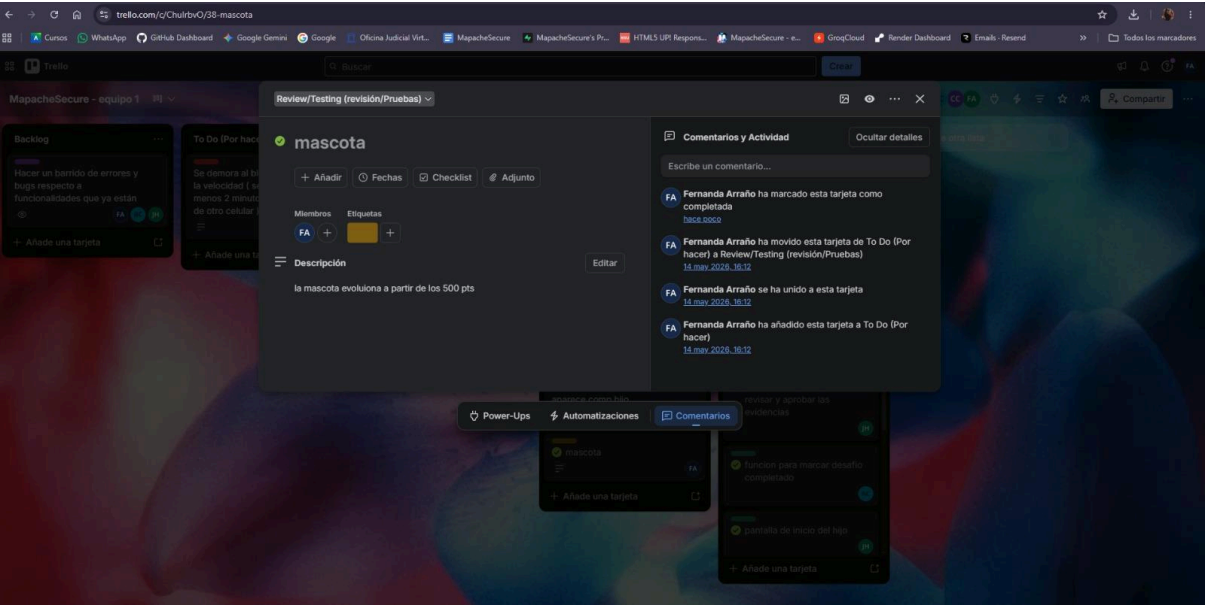
Al final de cada semana el equipo realiza una revisión informal donde se evalúa qué tareas se completaron, qué quedó pendiente y por qué. Las tareas no completadas se reincorporan al backlog con una nueva estimación. Adicionalmente, al cierre de cada sprint se realiza una retrospectiva breve donde el equipo identifica qué funcionó bien, qué debe mejorar y qué ajustes se aplicarán al siguiente sprint. Este ciclo de inspección y adaptación continua es el núcleo de la metodología Scrum y permite al equipo mantener el control del proyecto sin depender de una planificación rígida que no contemple los imprevistos propios del desarrollo de software.



Tablero Trello 1

Tablero Trello 2

Tablero Trello 3



Tablero Trello 4

Integrante	Rol Scrum	Responsabilidades Principales	Tecnologías
Benjamín Castillo	Líder /BDA	Diseño del modelo de datos, lógica de negocio, estructura de tablas en Supabase	Supabase, Python
Javier Vieytes	Backend/cloud	Desarrollo del backend, despliegue y mantención del servidor en producción	FastAPI, Python, Render
Fernanda Arraño	Frontend/QA	Desarrollo del frontend móvil Android, control de calidad y pruebas	Flutter, Dart, Android

Roles del equipo (quién hace qué) y responsabilidades.

Definición de “Hecho” (Definition of Done)

Para MapacheSecure, un elemento del backlog se considera "hecho" cuando cumple con todos los criterios siguientes:

El código ha sido desarrollado, probado manualmente por el desarrollador responsable y revisado por al menos un integrante del equipo. La funcionalidad implementada responde correctamente a los casos de uso esperados, incluyendo tanto los flujos exitosos como los casos de error más comunes. El endpoint o módulo desarrollado ha sido probado en el entorno local mediante la documentación interactiva de la API (Swagger UI en /docs) o directamente desde la aplicación Android según corresponda. Los cambios han sido commiteados al repositorio de GitHub con un mensaje descriptivo y el código no genera errores al iniciar el servidor. Finalmente, la funcionalidad ha sido desplegada exitosamente en el entorno de producción (Render) y responde correctamente desde la URL pública.

Funcionalidad	Revisado	Probado Swagger/app	Commiteado en Github	Desplegado en render
Autenticación	✓	✓	✓	✓
Gestion de usuarios	✓	✓	✓	✓
Módulo de desafíos	✓	✓	✓	✓
Sistema de recompensas	✓	✓	✓	✓
Apps bloqueadas	✓	✓	✓	✓
Avatar y evolucion de mascota	✓	✓	✓	✓
Foto de perfil del hijo	✓	✓	✓	✓
Notificaciones FCM	✓	-	✓	En progreso

Criterios de calidad mínimos

Los criterios de calidad mínimos que debe cumplir cada entregable son los siguientes. Todos los endpoints de la API deben retornar códigos HTTP correctos (200 para éxito, 404 para recurso no encontrado, 500 para errores del servidor). Todas las respuestas de la API deben estar en formato JSON válido y seguir una estructura consistente: las respuestas exitosas deben incluir los datos solicitados junto a un mensaje descriptivo, mientras que las respuestas de error deben incluir siempre el campo **detail** con una descripción clara del problema ocurrido, permitiendo que la app Android pueda interpretarlas y mostrar mensajes apropiados al usuario.

La base de datos no debe contener registros huérfanos ni inconsistencias entre tablas relacionadas. El módulo de bloqueo en Android debe interceptar correctamente las aplicaciones marcadas como bloqueadas sin afectar el rendimiento general del dispositivo. El sistema de puntos debe reflejar con exactitud los desafíos completados y validados, sin duplicar ni perder registros. La interfaz del panel parental debe estar protegida por autenticación y no debe ser accesible sin credenciales válidas. Por último, la aplicación debe funcionar correctamente en dispositivos Android con versión 8.0 o superior, que es el mínimo definido en el alcance del proyecto.

Sprint	Periodo	Objetivo principal
1	16 mar - 5 abril	Investigación, planificación y documentación del proyecto: levantamiento de requerimientos, diseño de diagramas wireframes y arquitectura.
2	7 abril - 24 may	Desarrollo e implementación de la aplicación: backend (FastAPI), base de datos (Supabase), módulo de bloqueo(UsageStats), sistema de desafíos, recompensas y mascota.
3	26 may - 30 jun	Integración final, pruebas de sistema, correcciones y preparación del entregable definitivo: notificaciones push (FCM), módulo IA y pulido de la app

Lista de hitos o sprints con fechas

12.Documentos y diagramas de diseño de la solución

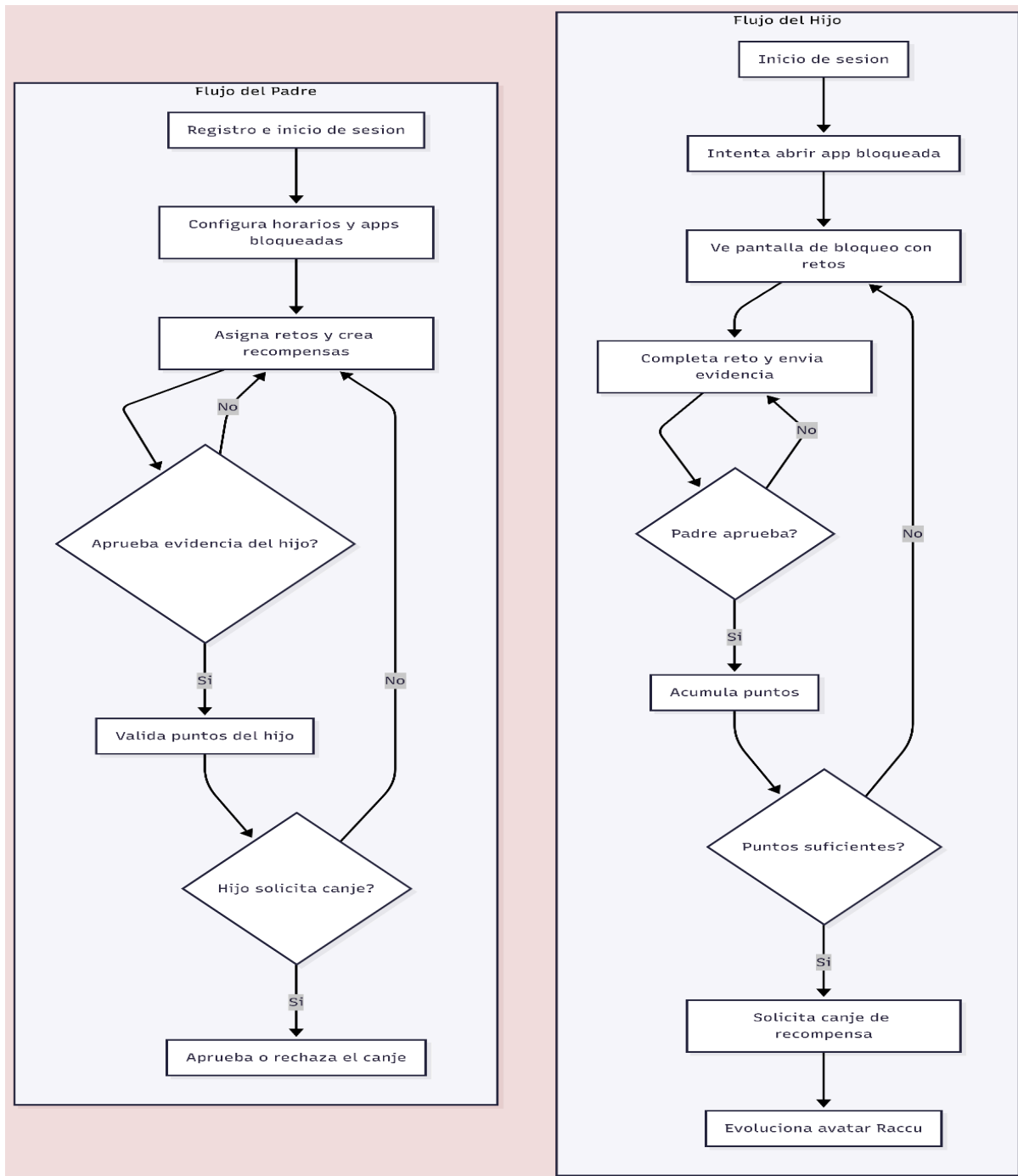


Diagrama de flujo del usuario (User Flow) o BPMN del proceso.

Propósito:

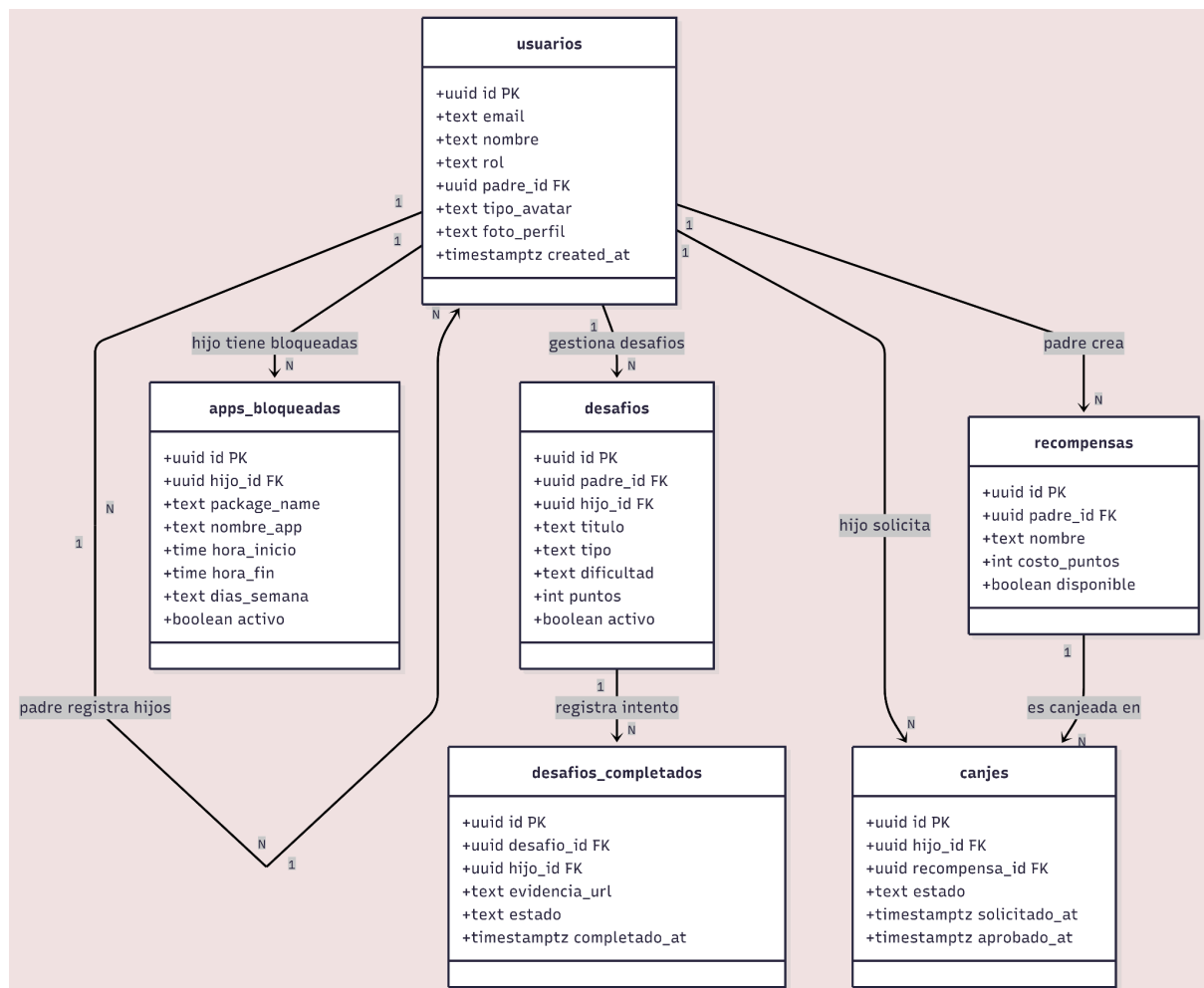
Mostrar cómo interactúan el hijo y el padre dentro del sistema.

Qué representa:

Los flujos de interacción de los dos actores principales del sistema

Flujo del padre: registra al hijo, configura bloques de horario para bloquear aplicaciones (mínimo de 2 horas), asigna retos y define mínimo 3 recompensas disponibles, aprueba o rechaza evidencias enviadas por el hijo, y aprueba las recompensas solicitadas para el próximo ciclo.

Flujo del hijo: visualiza sus retos pendientes, los completa y envía evidencia fotográfica, acumula puntos para canjear recompensas, personaliza su foto de perfil con imágenes predeterminadas de Raccu, y sube de nivel a su avatar según los puntos acumulados.



Modelo de datos (MER/DER) o esquema de BD (tablas, relaciones)

Propósito:

Definir la estructura de la base de datos del sistema.

Qué representa: La estructura completa de la base de datos del sistema, compuesta por 6 tablas.

- usuarios — almacena perfiles de padres e hijos, incluyendo configuración de bloqueo, avatar y foto de perfil
- desafíos — retos asignados por el padre al hijo, con tipo, dificultad y puntos asociados

- desafios_completados — registro de cada reto completado por el hijo con su evidencia y estado de validación
- apps_bloqueadas — aplicaciones restringidas por horario para cada hijo
- recompensas — beneficios creados por el padre canjeables con puntos
- canjes — historial de recompensas solicitadas por el hijo y aprobadas por el padre

Relación con el problema:

Permite almacenar y gestionar el progreso del hijo, el control parental y el sistema de puntos.

Relación con los objetivos:

Es la base para implementar la lógica del sistema y asegurar que la información se maneje correctamente.

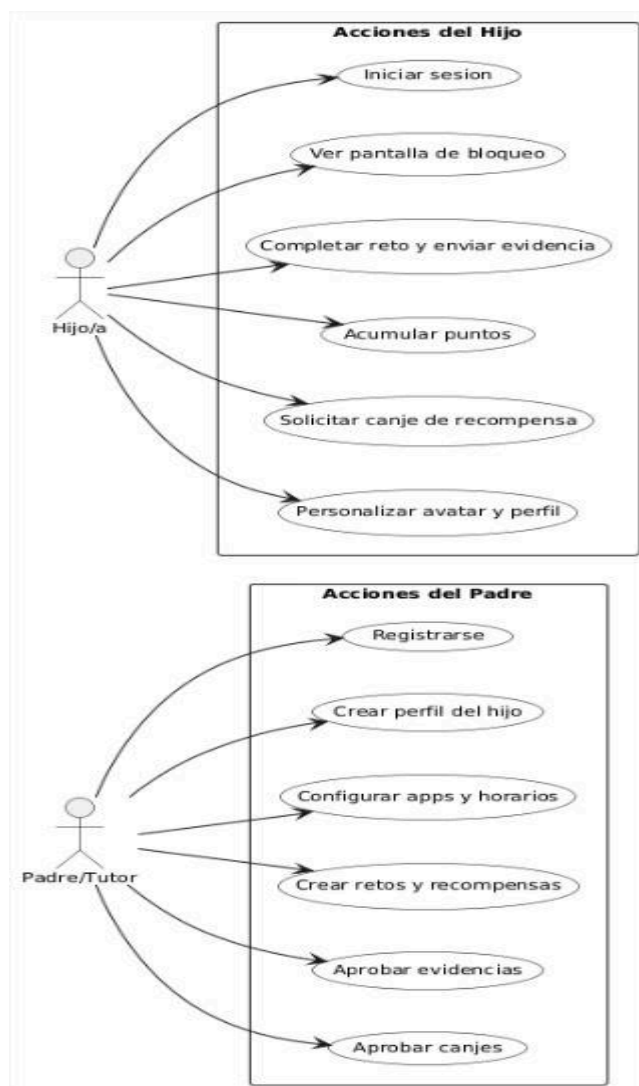


Diagrama de casos de uso o historias de usuario (con roles/actores).

Propósito:

Representar las funcionalidades del sistema desde la perspectiva de los usuarios (padre e hijo).

Qué representa:

Las funcionalidades del sistema desde la perspectiva de los dos actores principales y sus interacciones

Relación con el problema:

Permite identificar cómo cada actor interactúa con el sistema para controlar el uso de aplicaciones, gestionar desafíos y motivar al hijo mediante recompensas y progresión del avatar

Relación con los objetivos:

Asegura que todas las funcionalidades necesarias estén cubiertas: control parental, desafíos y evidencias, recompensas canjeables y gamificación mediante el avatar Raccu.

Historias de usuario.

HU-01: Como padre, quiero registrarme con mi correo y contraseña, para tener una cuenta personal y poder acceder al panel de control parental.

HU-02: Como padre, quiero crear un perfil para mi hijo dentro de la app, para vincularlo a mi cuenta y poder manejar sus horarios

HU-03: Como hijo, quiero iniciar sesión con las credenciales que ingresó mi padre, para acceder a la app desde mi dispositivo y ver los desafíos asignados con los puntos ganados.

HU-04: Como padre, quiero seleccionar qué aplicaciones deseas bloquear en el dispositivo de mi hijo, para poder limitar el acceso en horarios educativos o para que descanse de este.

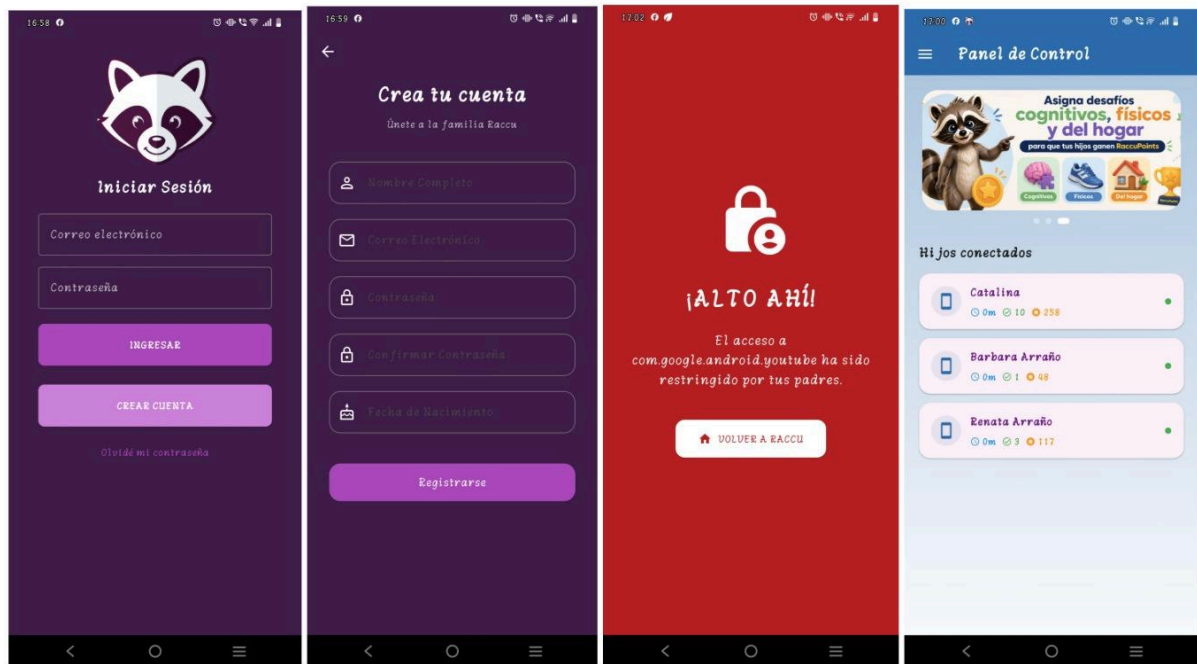
HU-05: Como hijo, quiero poder ver una pantalla de bloqueo cuando intento abrir una app restringida, para entender que es mi momento de parar de ocuparme y realizar mis tareas/desafíos.

HU-06: Como hijo, quiero algún tipo de recompensa al completar el desafío, así me da una motivación a seguir completando las

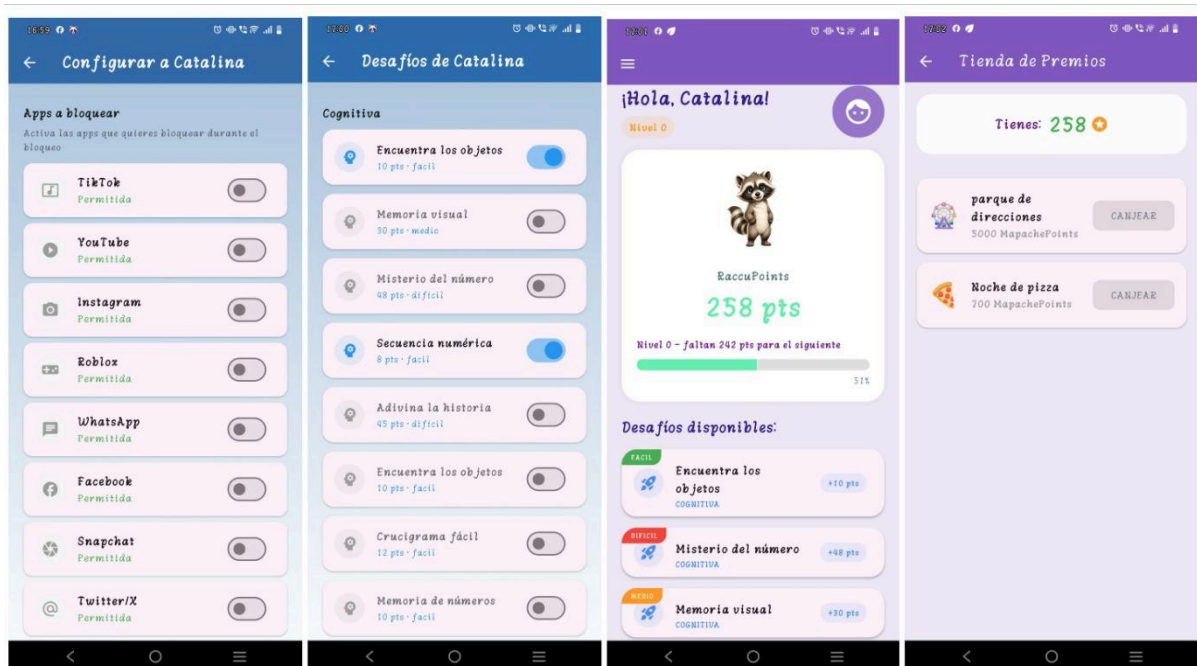
HU-07: Como hijo, me gustaría poder ver los desafíos que tengo pendientes, para saber que debo hacer y completarlos para ganar los puntos.

HU-08: Como padre, me gustaría poder crear o generar desafíos para que mi hijo no se quede sin desafíos a realizar.

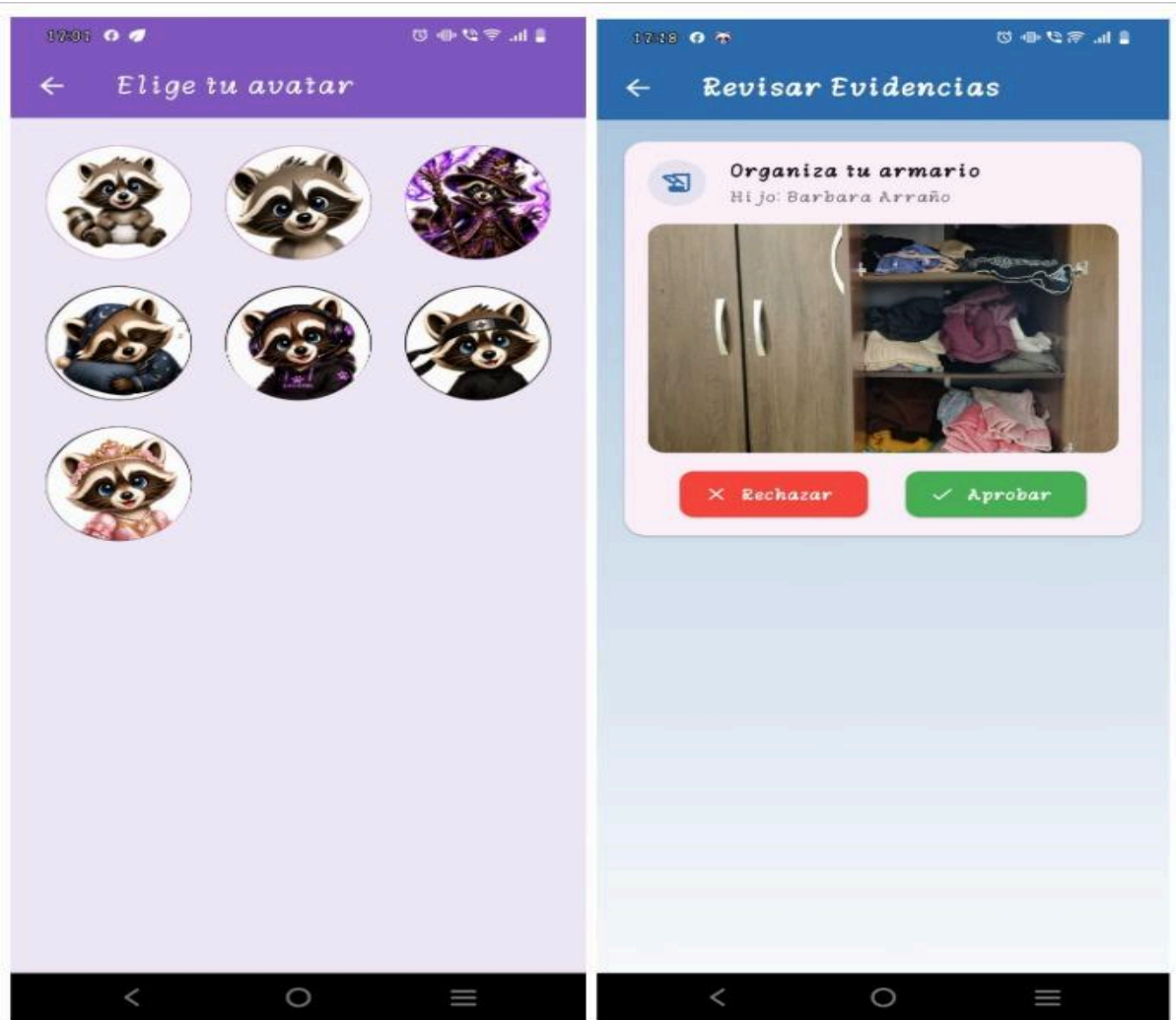
HU-09: Como padre, quiero crear recompensas con un nombre y un costo en puntos, para motivar a mi hijo a completar desafíos con metas concretas.



Mockups de la app: Inicio de sesión, Crear cuenta, Pantalla bloqueada, Panel de control del padre



Mockups de la app: configuración de bloqueo del hijo, desafíos, panel del hijo, tienda de premios



Mockups de la app: seleccione foto de perfil, revisión de evidencias enviadas

Propósito:

Mostrar la estructura visual de las pantallas del sistema antes de su implementación.

Qué representa:

- Pantalla de Inicio de sesión
- Pantalla de registro de usuario
- Pantalla de bloqueo de aplicaciones
- Pantalla de desafíos
- Subida de evidencia (foto)
- Panel parental
- Gestión de recompensas y canjes
- Selección de avatar
- Evolución del avatar
- Foto de perfil del hijo

Relación con el problema:

Permite entender cómo interactúan el padre y el hijo dentro del sistema de control parental y el flujo de uso de la aplicación.

Relación con los objetivos:

Define la estructura de la interfaz necesaria para implementar el control parental, la gamificación y la comunicación entre usuarios.

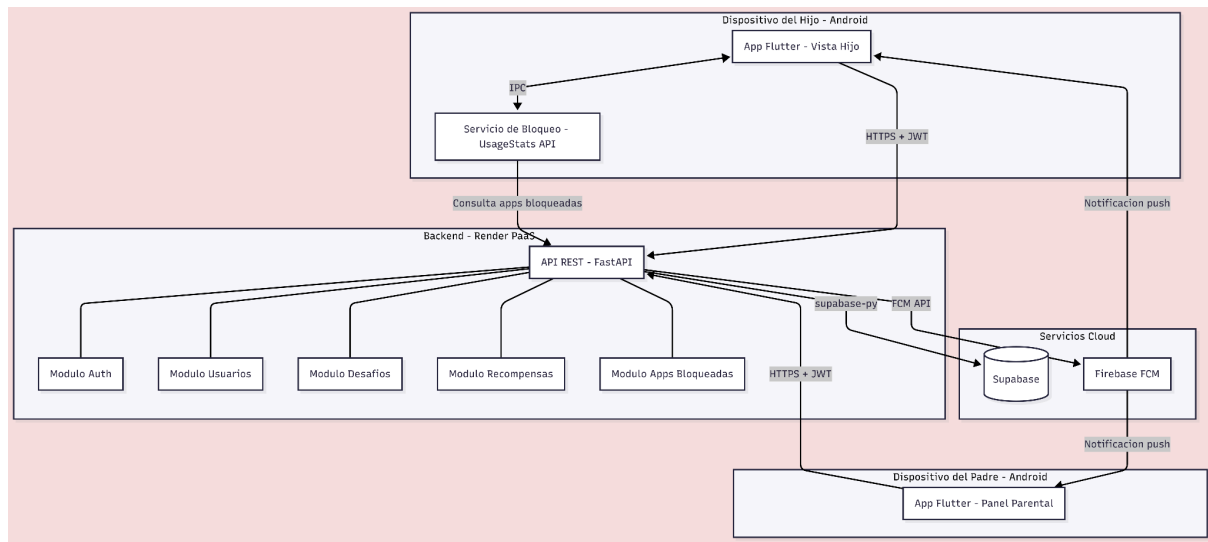


Diagrama de arquitectura (alto nivel).

Propósito:

Mostrar cómo se estructuran los componentes principales del sistema y cómo interactúan entre sí.

Qué representa:

- Aplicaciones móviles (padre e hijo)
- Backend/API
- Base de datos en Supabase
- Firebase FCM para notificaciones push y UsageStatsManager nativo de Android para monitoreo de uso de aplicaciones en el dispositivo del hijo

Relación con el problema:

Permite entender cómo el sistema controla el uso de aplicaciones, gestiona desafíos y comunica eventos entre padre e hijo.

Relación con los objetivos:

Define la base tecnológica necesaria para implementar el control parental, la gamificación y la comunicación en tiempo real.

13.Arquitectura de software y patrones de diseño

La arquitectura de MapacheSecure se ha diseñado bajo un modelo Cliente-Servidor con un enfoque de Arquitectura Limpia (Clean Architecture). Esta estructura garantiza que la lógica de negocio (desafíos, puntos y reglas de bloqueo) esté desacoplada de los servicios externos, permitiendo que el sistema sea testeable y escalable.

13.1. Arquitectura de N-Capas (Monolito Modular)

El sistema se organiza en capas con responsabilidades estrictamente separadas, tanto en el cliente como en el servidor:

- **Capa de Presentación (Flutter Client):** Gestiona la interfaz de usuario reactiva. Utiliza widgets que responden a cambios de estado y se comunica con las APIs nativas de Android (vía *Platform Channels*) para ejecutar el servicio de bloqueo de aplicaciones.
- **Capa de Dominio y Aplicación (FastAPI Backend):** Es el núcleo de la solución. Organizada en Routers (entrada de datos), Services (lógica de negocio y validación de retos) y Repositories (gestión de persistencia).
- **Capa de Infraestructura (Servicios Cloud):** Delegada a Supabase para la persistencia de datos y Firebase para las notificaciones push

13.2. Patrones de Diseño Implementados

Se han seleccionado patrones que optimizan el flujo de datos y la mantenibilidad del código:

Patrón Provider: Utilizado en Flutter para gestionar el estado de la aplicación. Permite separar la UI de la lógica de temas y sesiones, actualizando las vistas reactivamente ante cambios de estado.

Patrón Repository: Implementado en ambos extremos.

- En **Flutter**, centraliza las llamadas a la API de FastAPI.
- En el **Backend**, encapsula las consultas a Supabase, permitiendo que la lógica de negocio no dependa de la base de datos específica.

Patrón Observer: Utilizado mediante Streams en Flutter y Firebase Cloud Messaging en el sistema general. El dispositivo del padre "observa" los cambios en el estado del hijo, recibiendo notificaciones push inmediatas cuando se completa un desafío.

Patrón DTO (Data Transfer Object): Implementado mediante Pydantic en FastAPI. Define contratos estrictos para el intercambio de información (JSON), asegurando que los datos enviados desde Flutter sean validados antes de procesar cualquier lógica de puntos.

13.3. Justificación de la Arquitectura

Esta arquitectura es la más adecuada para el equipo por tres factores: Modularidad, Reactividad y Seguridad

- **Modularidad:** La división en routers/services/repositories nos permite poder trabajar en módulos independientes sin generar conflictos de código.
- **Reactividad:** Flutter y FastAPI (asíncrono) permiten que la experiencia de bloqueo y desbloqueo ocurra en menos de 500 ms, cumpliendo con los requisitos no funcionales de rendimiento.
- **Seguridad:** La separación entre capas garantiza que los secretos (SUPABASE_KEY, FCM_SERVER_KEY) nunca estén expuestos en el código. La autenticación JWT valida cada petición antes de acceder a los datos, protegiendo la información de los menores.

13.4. Representación Visual de la Arquitectura

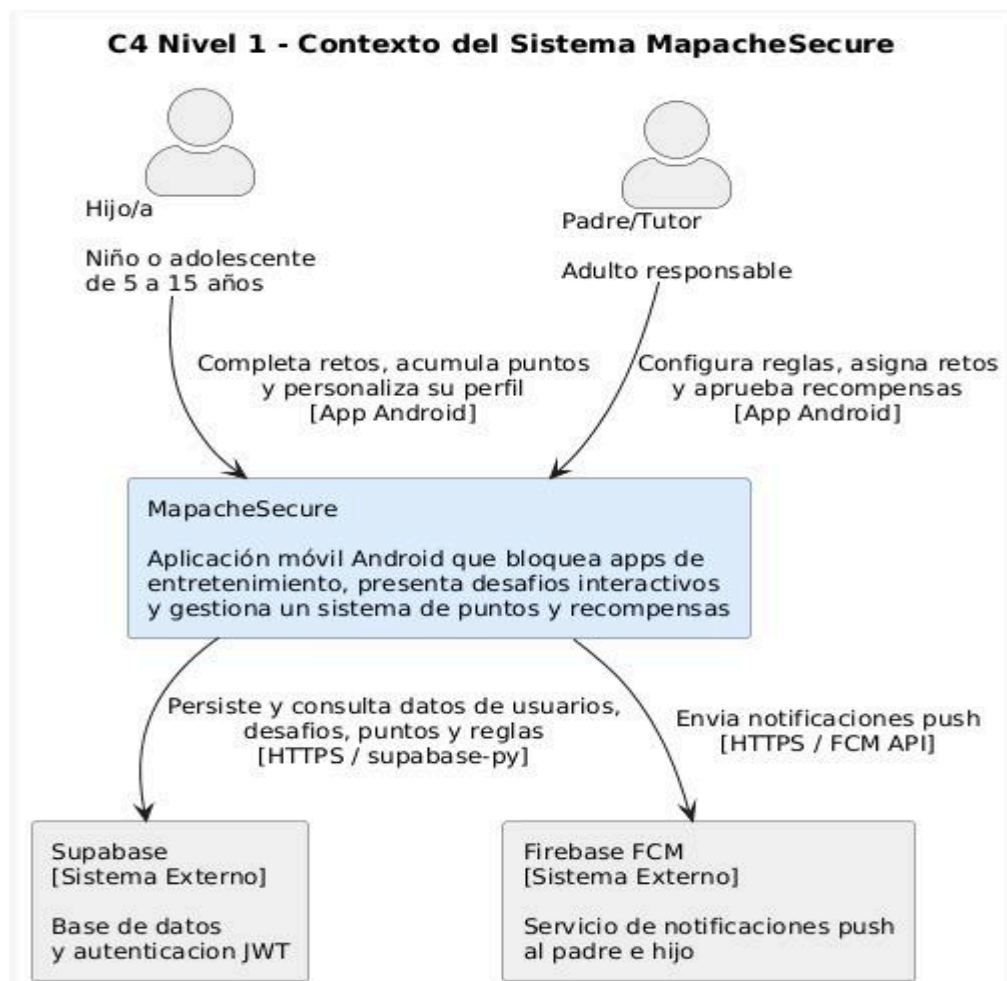


Diagrama C4 nivel 1: Contexto del sistema Mapacheseure

C4 Nivel 1 - Contexto del Sistema

Este diagrama representa el ecosistema de MapacheSecure a alto nivel. Muestra la interacción de los actores principales (Hijo/a y Padre/Tutor) con el sistema central. Se destaca la dependencia de servicios externos críticos: Supabase para la persistencia de datos y gestión de identidad, y Firebase FCM para el despacho de notificaciones push, estableciendo un flujo de comunicación bidireccional donde el sistema central actúa como

mediador entre las reglas de control parental, la gamificación mediante desafíos y recompensas, y el comportamiento del usuario menor.

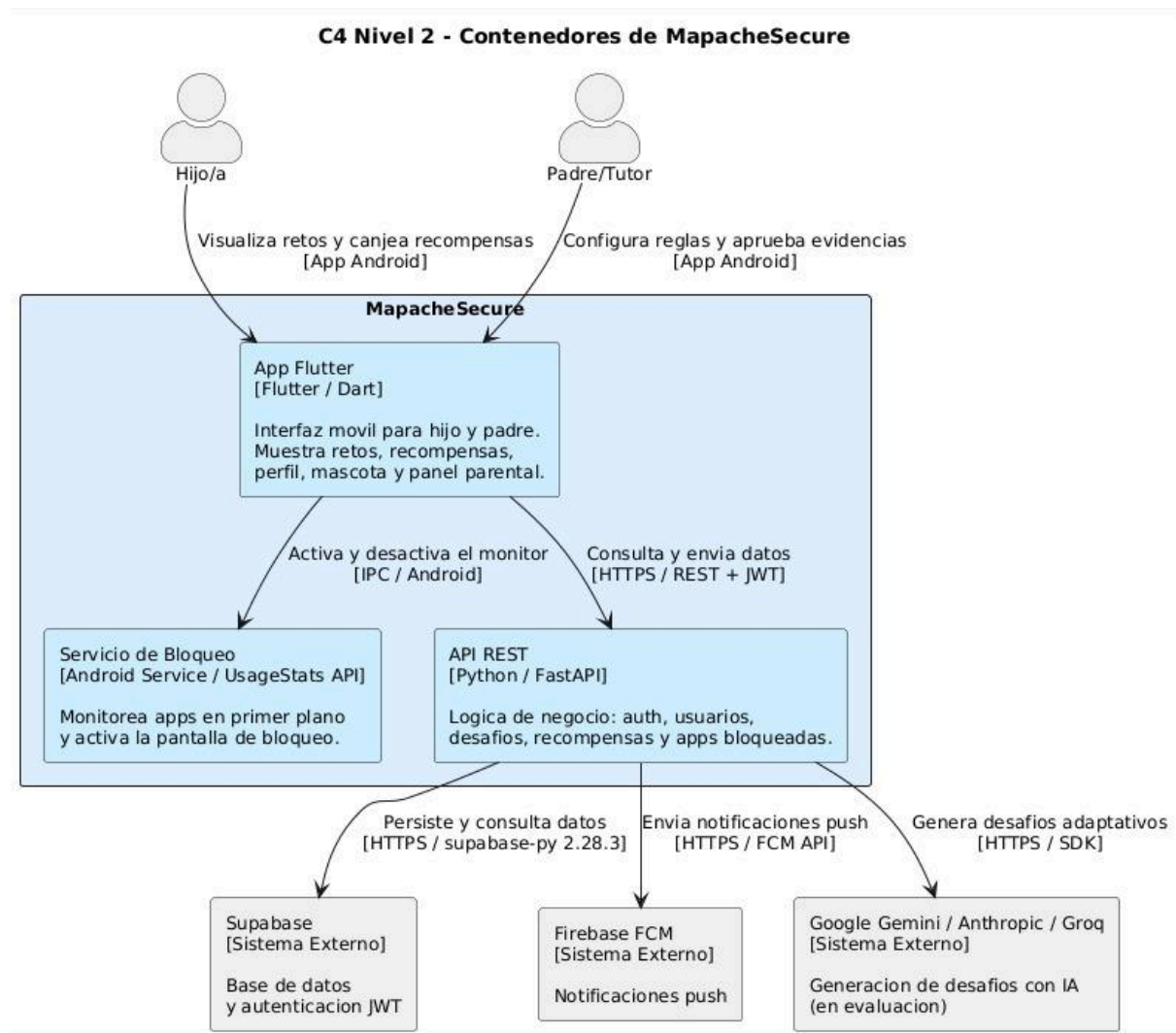


Diagrama C4 nivel 2: Contenedores de Mapacheseccure

C4 Nivel 2 - Contenedores

Este diagrama detalla la arquitectura interna del sistema, desglosada en componentes modulares. Se observa la App Flutter como interfaz principal, la cual se comunica mediante IPC con el BloqueoService (módulo nativo Android) para la intercepción de aplicaciones. El núcleo del negocio reside en la API REST (FastAPI), que coordina la lógica a través de diversos módulos especializados (Auth, Usuarios, Desafíos, Recompensas, Apps e IA) y gestiona la integración segura con las APIs externas de Supabase y Firebase mediante protocolos HTTPS y autenticación JWT.

14.Tecnologías, lenguaje y dependencias (stack definitivo)

Capa	Tecnología	Versión
App móvil	Flutter	3.41.7(Dart 3.11.5)
Backend / API	Python + FastAPI	Python 3.11
Base de datos	Supabase	—
Autenticación	Supabase Auth (JWT)	—
Notificaciones push	Firestore Cloud Messaging (FCM)	—
Hosting backend	Render (PaaS)	—
Cliente Supabase	supabase-py	2.28.3
Validación de datos	Pydantic	2.12.5
Servidor ASGI	Uvicorn	0.44.0
Gestión de proyecto	Trello (SCRUM)	—

Stack Tecnológico Definitivo

Librería	Versión
fastapi	0.135.3
uvicorn	0.44.0
supabase	2.28.3
python-dotenv	1.2.2
pydantic	2.12.5
google-generativeai	1.72.0
anthropic	0.94.0
firebase-admin	6.5.0
groq	0.28.0
resend	2.10.0
python-multipart	0.0.20

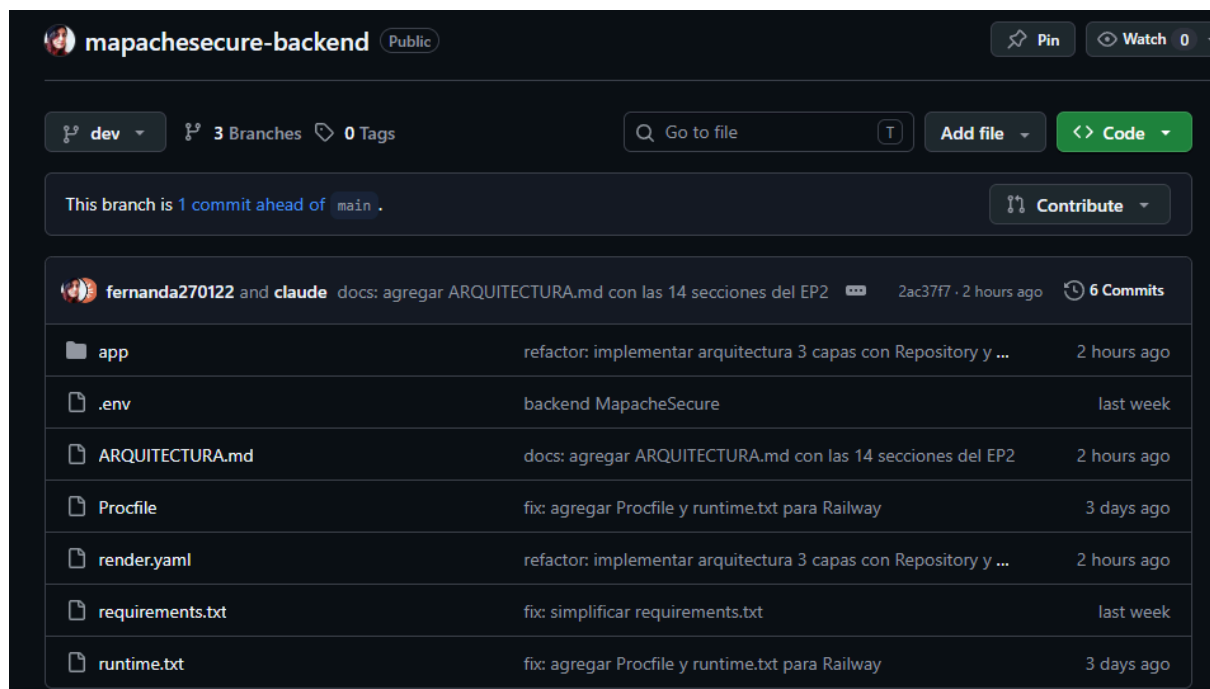
Dependencias del Backend (requirements.txt)

Estándares de Código

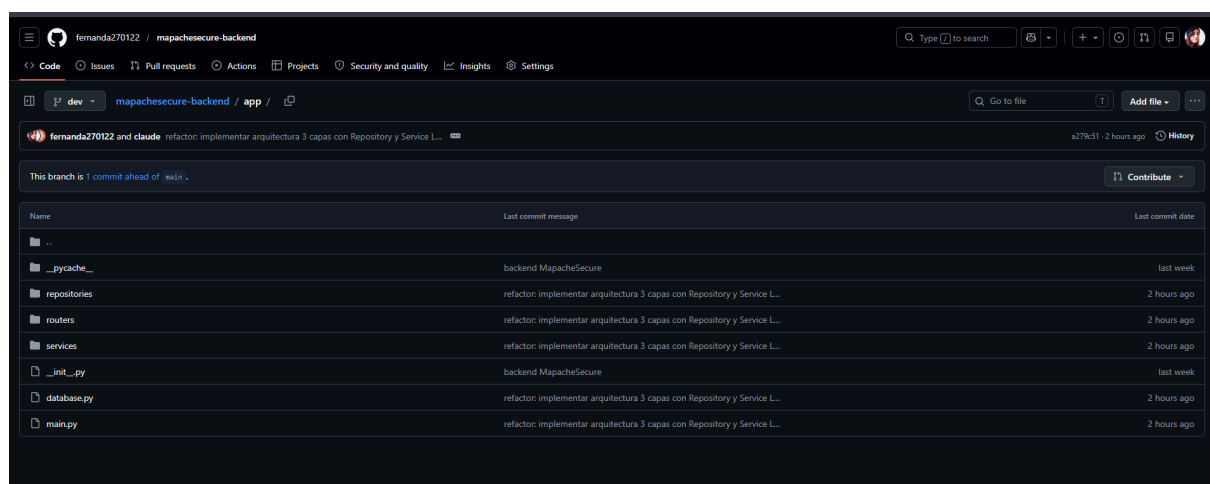
- Nomenclatura: snake_case para variables y funciones en Python; camelCase en Flutter.
- Estructura backend: routers / services / repositories (arquitectura en 3 capas).

- Commits: mensajes descriptivos con prefijos feat:, fix:, docs:, refactor:.
- Secretos: nunca en el código. Siempre en .env (local) o variables de entorno en Render (producción).
- Repositorio: github.com/fernanda270122/mapachese-secure-backend
- Ramas: main (producción) / dev (integración) / feature/* / fix/*
 - Link al repositorio.

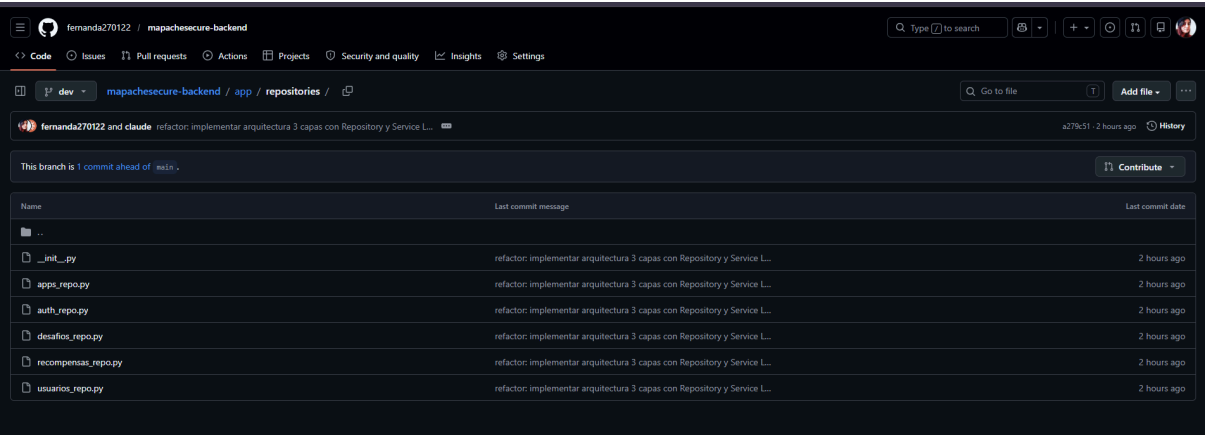
<https://github.com/fernanda270122/mapachese-secure-backend.git>



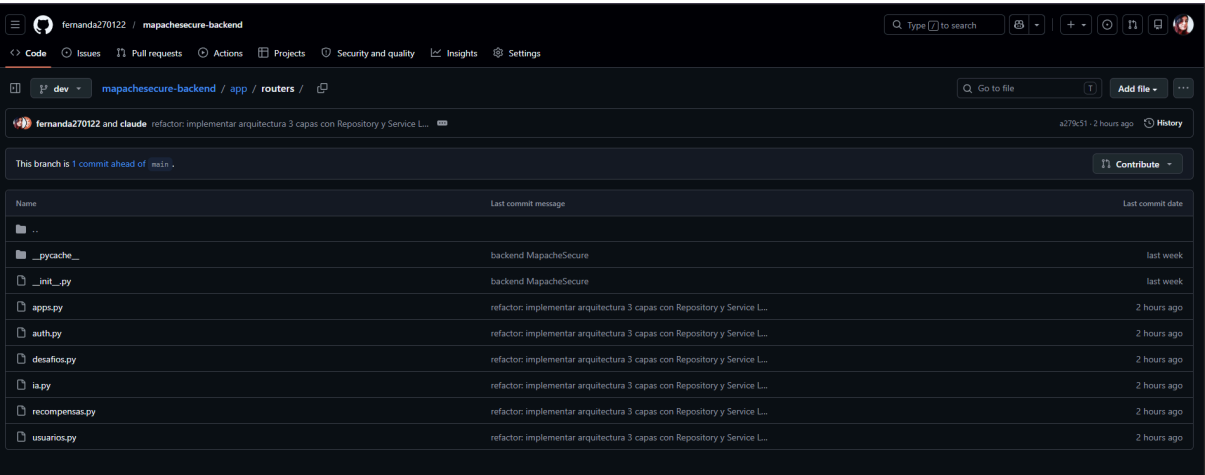
Github Mapachese-secure-backend



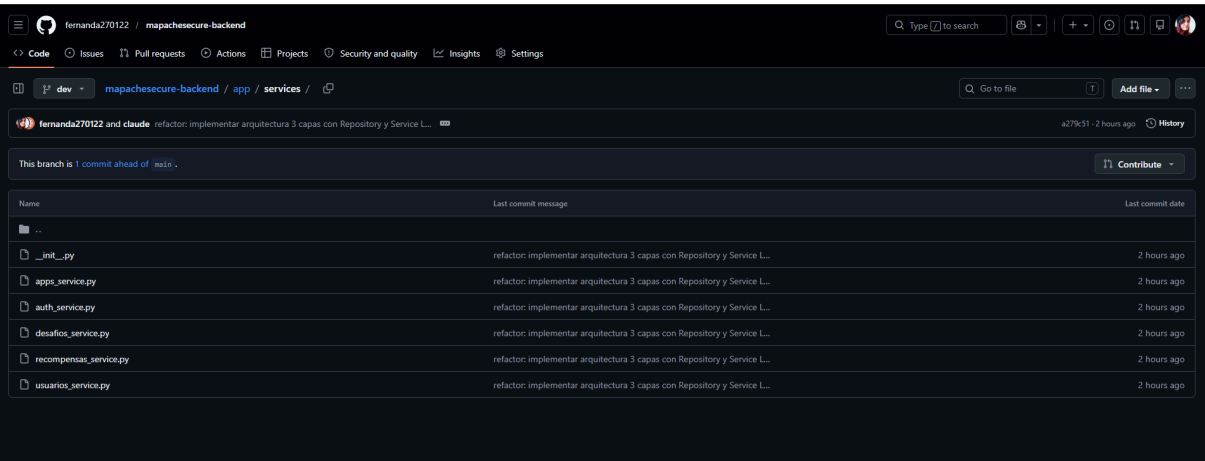
Github app



Github carpeta repositorio



Github carpeta Routes



Github carpeta Services

15. Configuración de servidor de producción (paso a paso)

Plataforma Cloud

El backend se despliega en Render (PaaS) con CI/CD automático desde GitHub. La base de datos y la autenticación se gestionan con Supabase (DBaaS). Ambas plataformas ofrecen plan gratuito suficiente para el desarrollo del semestre.

Requisitos del Servidor

- Runtime: Python 3.11.0
- Puerto: variable \$PORT asignada dinámicamente por Render
- Variables de entorno:
 - SUPABASE_URL,
 - SUPABASE_KEY
 - ANTHROPIC_API_KEY
 - GOOGLE_API_KEY
 - GROQ_API_KEY
 - RESEND_API_KEY
 - FCM_SERVER_KEY
- Comando de inicio: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- Archivo de configuración: `render.yaml` en la raíz del repositorio

Paso a Paso — Backend en Render

1. Crear cuenta en render.com y conectar la cuenta de GitHub.
2. New Web Service → conectar repositorio `mapachese-secure-backend` → rama `main`.
3. Configurar: Runtime = Python, Build Command = `pip install -r requirements.txt`, Start Command = `uvicorn app.main:app --host 0.0.0.0 --port $PORT`.
4. En Environment Variables, agregar todas las variables de entorno necesarias con los valores del proyecto Supabase.
5. Clic en Deploy. Render instala dependencias y levanta el servidor automáticamente.
6. Verificar que la API responde accediendo a `/docs` (Swagger UI) en la URL pública de Render.
7. Cada push a `main` dispara un deploy automático.

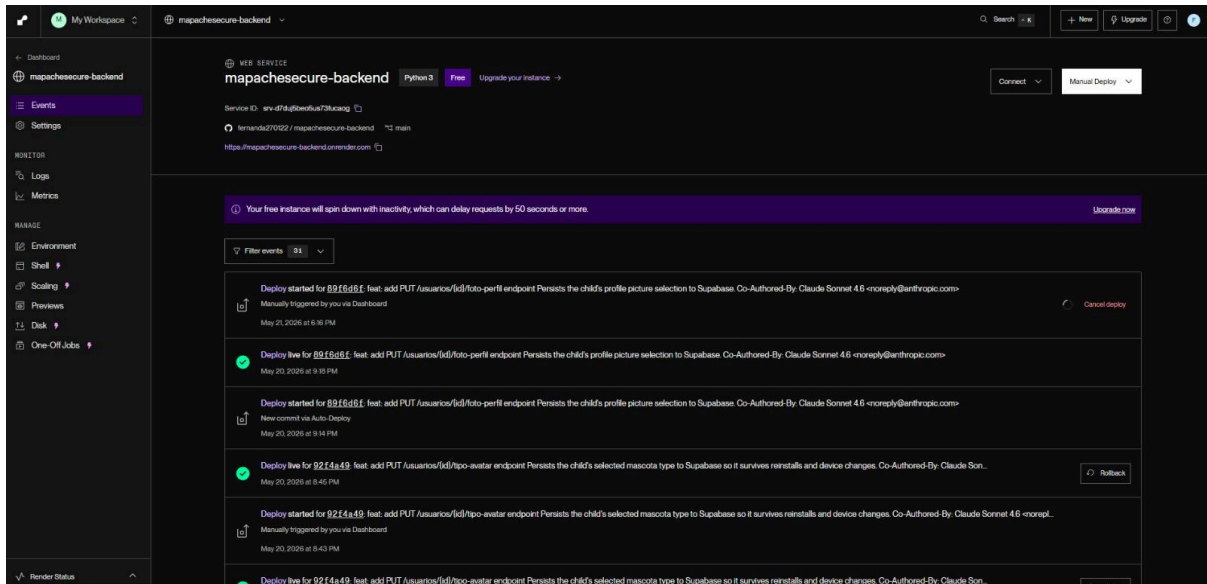
<https://dashboard.render.com/web/srv-d7duj5beo5us73fucaog>

<https://mapachese-secure-backend.onrender.com>

Paso a Paso — Base de Datos en Supabase

1. Crear proyecto en supabase.com.
2. Crear tablas: `usuarios`, `desafios`, `desafios_completados`, `apps_bloqueadas`, `recompensas`, `canjes`.

3. Activar Supabase Auth para registro y login con email/password.
4. Ir a Settings → API y copiar Project URL (SUPABASE_URL) y anon key (SUPABASE_KEY).
5. Pegar esas claves como variables de entorno en Render.



Despliegue en Render

<https://supabase.com/dashboard/project/xmsiydjwqvuzyykaidgj>

Nota sobre disponibilidad gratuita

El plan gratuito de Render duerme el servicio tras 15 minutos de inactividad. Se mitiga configurando un ping automático con cron-job.org cada 10 minutos para mantener el servidor activo durante la demo.

16.Estado de avance del desarrollo (evidencia de producto funcional)

Funcionalidades Implementadas

1. Autenticación

Registro y login con Supabase Auth, generación de JWT, vinculación padre-hijo, logout.

Evidencia: POST /auth/registro y POST /auth/login respondiendo con token y perfil. Probado en Swagger UI (/docs).

2. Gestión de Usuarios

Creación de perfiles, obtención de usuario por ID, listado de hijos por padre_id.

Evidencia: GET /usuarios/, POST /usuarios/, GET /usuarios/{padre_id}/hijos.

3. Módulo de Desafíos

Listado de desafíos por tipo, completar desafío con o sin foto, validación por el padre, consulta de puntos.

Evidencia: POST /desafíos/completar, PUT /desafíos/validar/{id}, GET /desafíos/puntos/{hijo_id}.

4. Sistema de Recompensas

Creación de recompensas por el padre, canje de puntos, historial de canjes.

Evidencia: POST /recompensas/, POST /recompensas/canjear, GET /recompensas/historial/{hijo_id}.

5. Apps Bloqueadas

Agregar y eliminar apps por hijo. Endpoint de verificación consultado por la app Android.

Evidencia: GET /apps/{hijo_id}, POST /apps/, GET /apps/verificar/{hijo_id}/{package_name}.

6. Avatar y evolución de mascota — Implementada

Sistema de gamificación donde el hijo selecciona un avatar entre 6 opciones (mago, dormilón, gamer, ninja, samurái, princesa) y lo hace subir de nivel acumulando puntos mediante desafíos completados. Niveles desbloqueables en 500, 1100, 1900, 2900, 4100 y 5500 puntos. Evidencia: PUT /usuarios/{id}/tipo-avatar, GET /desafíos/puntos/{hijo_id}.

7. Foto de perfil del hijo — Implementada

El hijo puede personalizar su foto de perfil eligiendo entre imágenes predeterminadas de Raccu. Evidencia: PUT /usuarios/{id}/foto-perfil.

Funcionalidades En Progreso

1. Notificaciones Push (FCM) — En progreso

Integración de Firebase Cloud Messaging para alertas al padre cuando el hijo completa un desafío.

2. App Android — En progreso

Pantalla de bloqueo mediante UsageStatsManager con polling cada 500 ms para detectar la app en primer plano. Integración con la API vía HTTP usando api service.dart.

Calidad y Seguridad

- Todos los endpoints retornan códigos HTTP correctos: 200, 404, 500.
- Las respuestas de error siempre incluyen el campo detail con descripción.
- Secretos nunca en el código fuente. Uso de .env local y variables de entorno en Render.
- Código refactorizado en 3 capas: routers / services / repositories.
- Control de versiones con ramas main, dev, feature/* y fix/*.

La pauta evalúa que el desarrollo avance con tecnologías adecuadas y buenas prácticas, cumpliendo funcionalidades aprobadas.

17.Integraciones necesarias

El ecosistema de **MapacheSecure** se sustenta en la integración de servicios de terceros que permiten delegar responsabilidades críticas como la seguridad, la persistencia y la comunicación en tiempo real. Esto permite que el equipo se enfoque en la lógica de negocio y la gamificación, asegurando un estándar profesional en la gestión de datos.

17.1. Detalle de Integraciones Existentes y Planificadas

1. Supabase — Base de Datos y Autenticación

Es el pilar de persistencia y seguridad del sistema. Supabase actúa como el backend-as-a-service (BaaS) que provee una base de datos y un sistema de gestión de identidades.

- **Qué hace:** Persiste en todas las entidades del sistema (usuarios, desafíos, puntos, configuración de apps bloqueadas) y gestiona el ciclo de vida de las sesiones mediante tokens JWT.
- **Datos de entrada:** Credenciales de usuario (Email/Password) para autenticación; estructuras JSON para operaciones CRUD (Create, Read, Update, Delete).
- **Datos de salida:** Tokens JWT de sesión, perfiles de usuario y registros de tablas solicitados por la API.
- **Librería técnica:** supabase-py versión 2.28.3.

2. Firebase Cloud Messaging (FCM) — Notificaciones Push

Este servicio es el canal de comunicación asíncrona entre el servidor y los dispositivos móviles, eliminando la necesidad de que la app realice consultas constantes (polling) al servidor.

- **Qué hace:** Envía notificaciones push al padre cuando el hijo completa un desafío y alertas de incentivo al dispositivo del hijo.
- **Datos de entrada:** Device Token (identificador único del celular), título, cuerpo del mensaje y metadatos de navegación.
- **Datos de salida:** ID de mensaje de Firebase y estado de confirmación de entrega.
- **Estado:** Pendiente de implementación técnica para el Sprint 3.

3. Google Gemini / Anthropic Claude — Módulo IA/ Groq — Módulo IA

Integración orientada a la generación de contenido dinámico para evitar la fatiga del usuario por retos repetitivos.

- **Qué hace:** Generación adaptativa de desafíos (trivias, lecturas, retos lógicos) basados en la edad del menor y su historial de cumplimiento.
- **Librerías preparadas:** google-genai (1.72.0) y anthropic (0.94.0), groq (0.28.0)
- **Estado:** En fase de evaluación y prototipado lógico.

4. Resend — Envío de Emails

Qué hace: Envía emails transaccionales durante el flujo de autenticación, incluyendo recuperación de contraseña y verificación de identidad.

Datos de entrada: Dirección de email del usuario, asunto y cuerpo del mensaje.

Datos de salida: Confirmación de entrega del email.

Librería técnica: resend 2.10.0

Estado: Integrado y funcionando en producción.

17.2. Consideraciones de Seguridad y Protección de Datos

Dada la naturaleza del proyecto y el manejo de datos de menores, la seguridad en las integraciones es la máxima prioridad:

- **Gestión de Secretos:** Todas las credenciales críticas (SUPABASE_KEY, FCM_SERVER_KEY, API Keys de IA) se gestionan estrictamente como Variables de Entorno en la plataforma de hosting (Render). Se prohíbe el almacenamiento de estas claves en archivos de texto plano o en el repositorio de GitHub.
- **Autenticación por Token:** Todas las llamadas a la API de FastAPI requieren un encabezado de autorización Bearer con un JWT válido emitido por Supabase. El backend valida la firma del token antes de permitir el acceso a los datos.
- **Configuración de Cliente:** La integración con la app móvil requiere el archivo google-services.json, el cual está configurado para permitir la comunicación segura solo desde el paquete de aplicación (com.mapacheseccure) firmado por el equipo.

17.3. Flujo de Datos y Comunicación entre Servicios

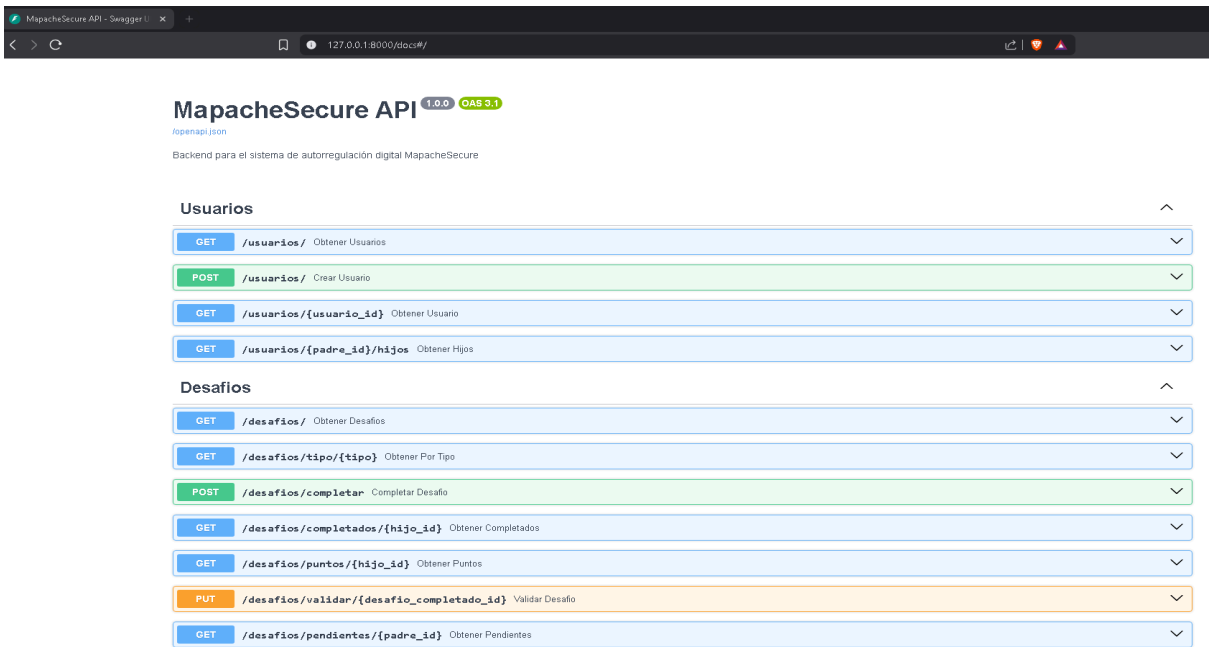
Para que la integración sea efectiva, el sistema sigue un flujo de información bidireccional donde el backend actúa como mediador central:

- **Flujo de Autenticación:** El cliente (Flutter) envía credenciales a la API; esta consulta a Supabase Auth, el cual retorna un JWT. Este token es almacenado localmente en el dispositivo para firmar cada petición posterior.
- **Flujo de Notificaciones:** Cuando el DesafiosService en FastAPI detecta que un desafío ha sido validado, gatilla una petición hacia los servidores de Firebase (FCM) enviando el *payload* del mensaje. Firebase se encarga de despertar el servicio en segundo plano del dispositivo del padre para mostrar la alerta.
- **Generación de Contenido (IA):** El backend envía un *prompt* estructurado con el nivel de dificultad y edad del menor. La IA responde con un objeto JSON que el backend procesa, guarda en la base de datos y finalmente entrega al móvil para ser renderizado como un desafío interactivo.

17.4. Evidencias de Pruebas de Integración

Como respaldo del avance técnico y la correcta comunicación entre los componentes, se presentan las siguientes evidencias:

1. Documentación y Contrato de API (Swagger UI)



2. Validación de Repuesta de Datos

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/usuarios/' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/usuarios/
```

Server response

Code	Details
200	Response body <pre>[{ "id": "93fe88fa-8604-440f-8516-a0f4faef728f", "email": "elena@mapachecure.com", "nombre": "Elena", "rol": "padre", "padre_id": null, "edad": 36, "created_at": "2026-04-12T16:53:20.9964" }, { "id": "19562a00-5eaa-4407-badf-46f74532180c", "email": null, "nombre": "mateo", "rol": "hijo", "padre_id": null, "edad": 10, "created_at": "2026-04-12T16:55:36.205969" }]</pre> Response headers <pre>content-length: 335 content-type: application/json date: Tue, 21 Apr 2026 02:49:33 GMT server: uvicorn</pre>

1. Supabase — Base de Datos y Autenticación

Qué hace: persiste todos los datos del sistema (usuarios, desafíos, puntos, apps bloqueadas) y gestiona el registro y login con tokens JWT.

Librería: supabase-py 2.28.3

Datos de entrada: credenciales de usuario para auth; datos de entidad para CRUD.

Datos de salida: tokens JWT, datos de usuario y registros de tablas.

Seguridad: SUPABASE_URL y SUPABASE_KEY almacenadas como variables de entorno, nunca en el repositorio.

Estado: integrado y funcionando en producción (Render).

2. Firebase Cloud Messaging (FCM) — Notificaciones Push

Qué hace: envía notificaciones push al padre cuando el hijo completa un desafío, y alertas de logro al hijo.

Datos de entrada: token FCM del dispositivo destino, título y cuerpo del mensaje.

Datos de salida: confirmación de entrega de la notificación.

Seguridad: FCM Server Key almacenada como variable de entorno.

Consideraciones: requiere configurar google-services.json en la app Android y habilitar FCM en la consola de Firebase.

Estado: pendiente de implementación (sprint 3).

3. Google Gemini / Anthropic Claude — Módulo IA (planificado)

Qué hace: generación dinámica de desafíos adaptativos según el perfil del hijo (edad, historial de desafíos completados).

Las dependencias google-genai y anthropic están incluidas en requirements.txt como preparación para su implementación.

Estado: en evaluación. No está implementado en el sprint actual.

18. Estrategia y enfoque de calidad

Atributos de calidad evaluados

En Raccu, la calidad se define como la capacidad de la aplicación de proteger efectivamente a los niños dentro de los horarios configurados por sus padres, sin interrupciones ni vulnerabilidades explotables. Los atributos priorizados son:

1. Funcionalidad: Que cada módulo cumpla su propósito: bloqueo de apps, desafíos, mascota, perfil, notificaciones e IA.
2. Fiabilidad: El guardián de bloqueo debe operar en background de forma continua y sin caídas.
3. Seguridad: Las sesiones deben estar correctamente protegidas; la pantalla de bloqueo no debe ser eludible.
4. Rendimiento: El parsing de datos y las búsquedas en memoria deben responder dentro de umbrales aceptables incluso con volúmenes altos.
5. Usabilidad: Los flujos críticos (login, agregar hijo, seleccionar avatar) deben completarse sin ambigüedades visuales.

Niveles y tipos de prueba

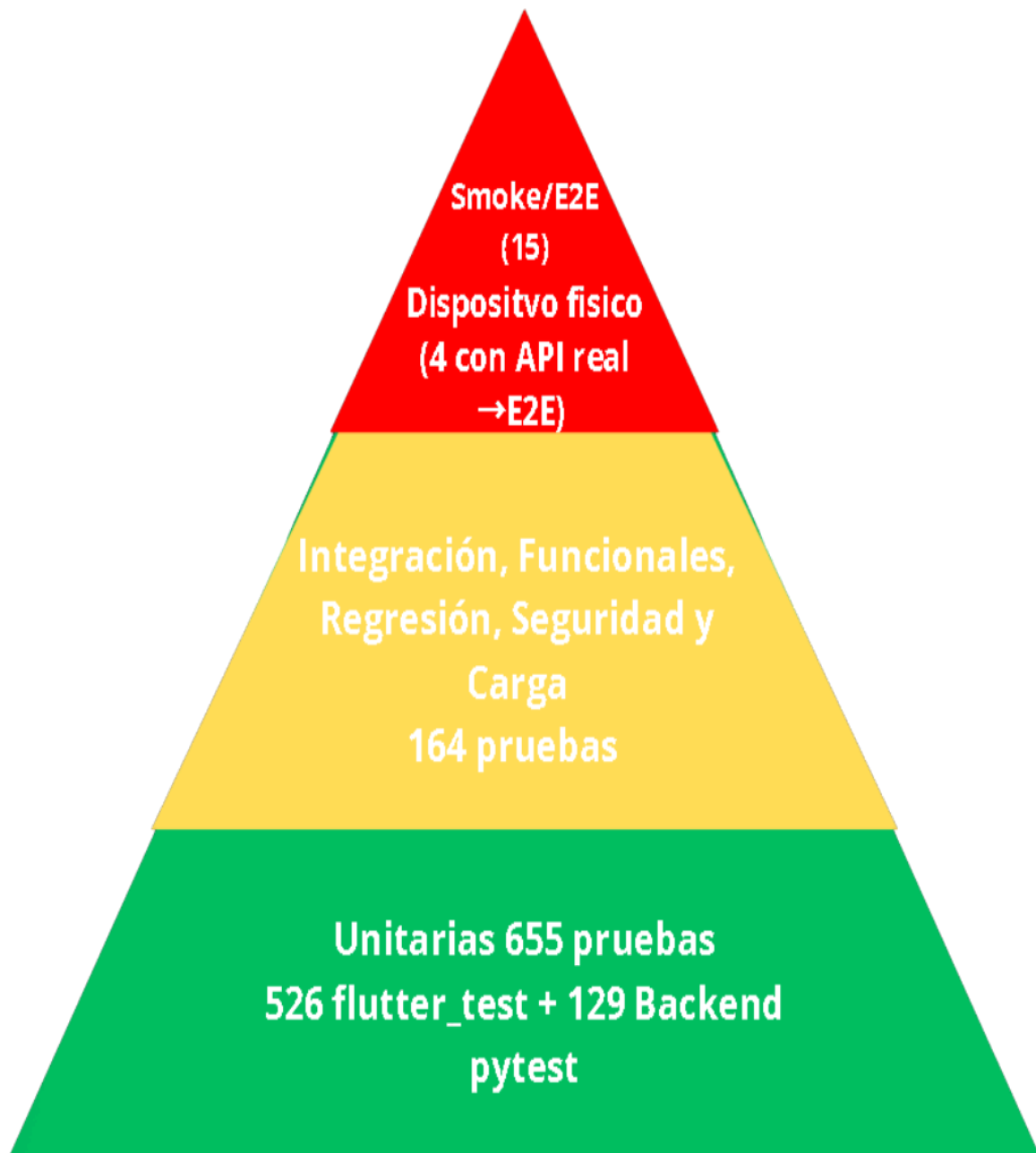
1. Se aplica una estrategia en capas, de lo más granular a lo más completo:
2. Pruebas unitarias: verifican modelos, providers y servicios de forma aislada.
3. Pruebas de integración: validan que los módulos colaboren correctamente (AuthService + SharedPreferences, providers entre si).
4. Pruebas funcionales: recorren flujos completos desde la perspectiva del usuario (login, bloqueo, avatar, recuperación de contraseña).
5. Pruebas de humo (smoke): comprueban en dispositivo real que las pantallas críticas arrancan tras cada compilación.
6. Pruebas de regresión: aseguran que cambios recientes no rompan comportamientos previos.
7. Pruebas de rendimiento/carga: miden tiempo de procesamiento con volúmenes altos de datos.
8. Pruebas de seguridad: verifican autenticación, control de sesión e impermeabilidad de la pantalla de bloqueo.

Enfoque de prueba

La totalidad de las pruebas está automatizada con el framework `flutter_test` (pruebas unitarias, integración, funcionales, regresión, carga y seguridad) y con `integration_test` para las pruebas de humo en dispositivo. Los datos de prueba son sintéticos (objetos mock y SharedPreferences simulados), no se utilizan credenciales ni endpoints reales durante la ejecución. El ambiente de ejecución es el entorno local de desarrollo con Flutter SDK 3.x (Dart ^3.11.5).

Definition of Done

- El 100% de las pruebas unitarias, de integración, funcionales, de regresión, de carga y de seguridad pasan en verde.
- Las 9 pruebas de humo en un dispositivo físico pasan sin errores visuales.
- No existen defectos abiertos de severidad alta o bloqueante.
- El Código está versionado en la rama correspondiente y el pipeline de pruebas se ejecuta sin fallas.



19. Plan de pruebas

Alcance

El plan cubre el frontend Android de Raccu (proyecto mapacheseccure_app, Flutter/Dart) y el backend (proyecto mapacheseccure-backend, FastAPI/Python). El frontend se prueba con flutter_test e integration_test. El backend se prueba con pytest, usando mocks de Supabase para no depender de credenciales reales ni de red durante la ejecución.

Funcionalidades a probar

- Autenticación de usuarios (login, logout, recuperación de contraseña)
- Registro y gestión de hijos
- Bloqueo de apps por horario (guardian en background)
- Gestión de reglas (panel del padre)
- Sistema de desafíos
- Sistema de mascota y avatares
- Foto de perfil del hijo
- Asistente IA para padres
- Notificaciones push (FCM)
- Dashboard del hijo (actividad y tiempo de pantalla)

Criterios de aceptación

- 100% de las pruebas unitarias, funcionales, de integración, regresión, carga y seguridad aprobadas.
- 4/4 pruebas de humo en dispositivo físico aprobadas.
- 0 defectos críticos (bloqueo de funcionalidad principal) abiertos al cierre.

Criterios de suspensión y reanudación

Las pruebas se suspenden si un defecto impide la compilación o el arranque de la aplicación. Se reanuda una vez corregido el defecto y validado por al menos un integrante del equipo.

Integrante	Rol en pruebas	Responsabilidad principal
Fernanda Arrano	QA Lead / Desarrolladora	Pruebas unitarias, funcionales y humo. Redacción del informe.
Javier Vieytes	Desarrollador / Tester	Pruebas de integración y regresión. Revisión de resultados.
Benjamin Castillo	Desarrollador / Tester	Pruebas de seguridad y carga. Configuración de ambiente.

Recursos y roles

Riesgo	Mitigación
Dispositivo no disponible para pruebas de humo	Usar emulador como respaldo; documentar diferencias.
Cambios de última hora en pantallas	Mantener smoke tests diagnósticos al contenido visual específico.
Dependencias de plugins Android (UsageStats, BackgroundService)	Mockear en pruebas unitarias; validar solo en dispositivo físico.

Riesgos y mitigación

20. Tipos de pruebas aplicadas

20.1a pruebas unitarias (lógica) · Flutter

Herramienta: flutter_test (SDK de Flutter)

Casos: 112 casos: 45 modelos + 16 providers + 51 servicios

Verifican de forma aislada las piezas más pequeñas del código sin interfaz gráfica. Se utilizan SharedPreferences simulados y MockClient para no depender de red ni almacenamiento real.

- Modelos (45): conversión de JSON a objetos del dominio (Usuario, Desafio, Recompensa, AppBloqueada, PetModel, AvatarType), incluyendo cálculo de nivel de mascota según puntos y selección de avatar.
- Providers (16): gestión de estado de temas visuales (TemaProvider, TemaPadreProvider) y actividad, verificando que guardan y recuperan preferencias correctamente.
- Servicios (51): lógica de AuthService, ApiService y GuardianService — manejo de sesión, token, roles y construcción de peticiones HTTP.

20.1b pruebas unitarias (UI / widgets) · Flutter

Herramienta: flutter_test + testWidgets

Casos: 413 casos

Validan que cada pantalla se construye y se comporta correctamente de forma aislada: que muestra los textos y botones esperados, que reacciona a las interacciones del usuario y que navega a la pantalla correcta. Es el bloque más extenso porque la aplicación está centrada en la experiencia de usuario.

- Autenticación: login, registro, recuperar y restablecer contraseña, verificación de identidad.
- Módulo hijo: home, avatares, desafíos, actividad, tienda de recompensas, pantalla de bloqueo y evolución de la mascota.
- Módulo padre: home, configurar hijo, desafíos, recompensas, revisión de evidencias, canjes pendientes y actividad del hijo.

20.2 Pruebas de humo (smoke)

Herramientas: integration_test (Flutter) + dispositivo Android físico · pytest (backend)

Casos: 9 frontend + 6 backend = 15 casos totales

Frontend (9): ejecutados en dispositivo Vivo V2127 (Android 14). Comprueban que cada historia de usuario crítica arranca y responde en hardware real: registro, login, agregar hijo,

configurar hijo, desafíos del hijo, recompensas, mis desafíos, selección de avatar y pantalla de bloqueo. Se realizan dos logins reales al inicio para obtener tokens válidos.

Backend (6): verifican que la API FastAPI levanta correctamente, que el endpoint raíz retorna HTTP 200 con los campos version y mensaje, y que el sistema de autenticación rechaza tokens inválidos o ausentes con HTTP 401.

20.3 Pruebas de integración

Herramienta: flutter_test (Flutter) · pytest + pytest-mock (backend)

Casos: 4 frontend + 141 backend = 145 casos totales

Frontend (4): validan la colaboración entre módulos internos: AuthService leyendo estado de sesión desde SharedPreferences, persistencia y recuperación de temas (TemaProvider y TemaPadreProvider) sin interferencia entre claves, y que la selección de avatar actualiza la clave correcta en el almacenamiento local.

Backend (164): verifican que las capas de rutas, servicios y repositorios se comunican correctamente. Cubren los módulos de autenticación, aplicaciones, desafíos, recompensas, usuarios, notificaciones, bloqueos, canjes y actividad. Se comprueba que los datos fluyen correctamente desde el endpoint HTTP hasta la capa de acceso a Supabase y viceversa, validando códigos de respuesta (200, 400, 403, 404, 500) en cada caso.

20.4 Pruebas funcionales (E2E)

Herramienta: flutter_test + testWidgets

Casos: 4 casos

Recorren flujos completos como lo haría un usuario: login fallido con credenciales vacías, selección visual de avatar con confirmación de borde deepPurple, visualización de pantalla de bloqueo con horario real y navegación al flujo de recuperación de contraseña.

20.5 Pruebas de regresión

Herramienta: flutter_test (Flutter) · pytest (backend)

Casos: 4 frontend + 2 backend = 6 casos totales

Aseguran que bugs ya corregidos no vuelvan a aparecer tras nuevos cambios.

Frontend (4):

- **BUG-001:** cerrar sesión borraba las preferencias de onboarding y el padre volvía a verlo desde cero. Ahora `logout()` preserva esas claves.
- **BUG-002:** desafíos con estado en mayúsculas ("ACTIVO") no se reconocían y la lista quedaba vacía. Ahora la comparación ignora mayúsculas.
- **BUG-003:** un hijo con 0 puntos veía la imagen de nivel 1 en vez de la mascota base. Ahora muestra la imagen base correcta.
- **BUG-004:** un tipo de avatar desconocido proveniente de la base de datos hacía crashear la app. Ahora retorna un avatar por defecto sin lanzar excepción.

Backend (2): confirman que avatares con valores inválidos siguen siendo rechazados con HTTP 400 y que la configuración de tiempo límite para hijos se actualiza correctamente, independientemente de los cambios recientes en el servicio de usuarios.

20.6 Pruebas de rendimiento / carga

Herramientas: `flutter_test` + Stopwatch de Dart (Flutter) · k6 v0.57.0 (backend)

Casos: 4 frontend + 1 escenario k6 backend

Frontend (4): miden el tiempo de procesamiento local bajo volúmenes altos con umbrales estrictos:

20.7 Pruebas de seguridad

Herramienta: `flutter_test` (Flutter) · `pip-audit` + `pytest` (backend)

Casos: 7 frontend + 2 backend = 9 casos totales

Frontend (7) — alineados con OWASP:

- Sin token → `isLoggedIn` retorna false (A07 — Fallo de autenticación)
- Eliminar el token invalida la sesión de inmediato (A07)
- Usuario con rol hijo no puede ser reconocido como padre (A01 — Control de acceso)
- `PantallaBloqueoScreen` tiene `canPop` = false, impidiendo salir con el botón de retroceso (A01)
- Nombre con HTML/script del servidor se trata como texto plano, no se ejecuta (A05 — Configuración incorrecta)

- Package name con intento de inyección SQL se almacena como texto inofensivo (A03 — Inyección)
- Puntos negativos enviados por el servidor no crashean la mascota (A05)

Backend (2): auditoría de dependencias con pip-audit — resultado final 0 CVEs conocidos tras actualizar python-multipart a 0.0.32 e idna a 3.18. Validación de entradas confirma que valores fuera de rango son rechazados antes de llegar a Supabase.

20.8 Pruebas unitarias de backend

Herramienta: pytest + pytest-mock (Python)

Casos: 129 casos de capa de servicios / 276 casos totales del backend

Cubren la capa de servicios del backend FastAPI. Cada servicio se prueba de forma aislada mockeando el repositorio correspondiente, por lo que ningún test toca la base de datos Supabase ni la red. Se validan flujos exitosos, errores esperados (404, 400, 403, 500) y casos borde como correos duplicados, canjes sin puntos suficientes y avatares inválidos.

Módulos cubiertos: autenticación, apps, canjes, desafíos, IA, recompensas y usuarios. Resultado: 276/276 en verde, 94% de cobertura global, superando el mínimo exigido del 90%.

21. Herramientas y software utilizados

Herramienta	Tipo de prueba / uso	Versión	Justificación
flutter_test	Unitarias, integración, funcionales, regresión, carga, seguridad	SDK Flutter 3.x	Framework oficial de Flutter. Incluye testWidgets, pump, pumpAndSettle y matchers. No requiere dependencia externa adicional.
integration_test	Pruebas de humo en dispositivo	SDK Flutter 3.x	Paquete oficial para pruebas en dispositivo físico o emulador. Permite simular interacciones reales sobre la UI compilada.
MockClient (http/testing)	Mock de API REST	http ^1.2.1	Intercepta llamadas HTTP y retorna respuestas simuladas sin contactar el backend real. Usado en servicios y smoke tests.
SharedPreferences fake	Persistencia simulada	shared_preferences ^2.3.2	setMockInitialValues() precarga valores en memoria para que los tests sean deterministas sin tocar disco.

flutter_screenutil	Soporte responsivo en smoke tests	^5.9.3	Necesario en integration_test para inicializar ScreenUtilnit antes de renderizar pantallas con tamaños responsivos.
Stopwatch (Dart core)	Medición de tiempo en pruebas de carga	Dart ^3.11.5	Clase nativa de Dart usada para medir tiempos de ejecución y comparar contra umbrales definidos en cada test.
pytest + pytest-mock	Pruebas unitarias de backend	pytest 9.0.3	Framework de pruebas de Python. pytest-mock proporciona la fixture mocker para reemplazar repositorios con objetos simulados sin tocar Supabase.
GitHub	Control de versiones	N/A	Repositorio centralizado. Las ramas main (producción) y dev (desarrollo) permiten integrar cambios de forma controlada.
VS Code + Dart extension	Entorno de desarrollo y ejecución	N/A	Permite correr flutter test directamente desde el editor con visualización de resultados en tiempo real.

22. Ejecución de pruebas y resultados (Coverage)

Coverage total del Frontend

Las 549 pruebas se ejecutaron con el comando `flutter test --coverage` desde el directorio raíz del proyecto `mapachese-secure-app`, finalizando con resultado 549/549 exitosas, 0 fallidas. El análisis de cobertura procesó un total de 4.432 sentencias distribuidas en 43 archivos del proyecto, logrando cubrir el 93% del código (4.125 sentencias ejecutadas, 307 no alcanzadas).

Durante la ejecución se registraron advertencias de tipo `warnIfMissed` en algunos tests de navegación del Drawer de la pantalla del padre (`home_padre_screen_test.dart`). Estas advertencias indican que ciertos widgets quedaban fuera del área visible del render tree durante la simulación (800×600 px), pero no constituyen fallos: Flutter ejecutó y completó cada test marcándolo como exitoso. Son un artefacto del entorno de prueba, no un error de la aplicación.

En cuanto a la distribución de cobertura por módulo, los modelos, providers y servicios alcanzaron un 100% de cobertura en prácticamente todos sus archivos. Las pantallas presentaron valores entre 76% y 100%, siendo `screens/auth/verificar_identidad_screen.dart` (76%) y `screens/hijo/detalle_desafio_screen.dart` (77%) los archivos con mayor cantidad de sentencias no cubiertas, lo cual se explica por flujos condicionales de validación y manejo de errores que requieren escenarios de prueba más específicos.

Archivo	Stmts	Miss	Cover
models/app_bloqueada.dart	33	0	100%
models/avataar_type.dart	7	0	100%
models/desafio.dart	47	0	100%
models/pet_model.dart	7	0	100%
models/recompensa.dart	36	0	100%
models/usuario.dart	30	0	100%
providers/actividad_provider.dart	69	0	100%
providers/tema_padre_provider.dart	14	0	100%
providers/tema_provider.dart	11	0	100%
screens/auth/login_screen.dart	159	13	92%
screens/auth/recuperar_password.dart	60	0	100%
screens/auth/registro_screen.dart	119	6	95%
screens/auth/reset_password_screen.dart	83	0	100%
screens/auth/verificar_identidad_screen.dart	142	34	76%
screens/hijo/app_bloqueada_screen.dart	17	0	100%
screens/hijo/avataar_screen.dart	37	0	100%
screens/hijo/colores_screen.dart	52	0	100%
screens/hijo/detalle_desafio_screen.dart	125	29	77%
screens/hijo/guia_hijo_screen.dart	43	0	100%
screens/hijo/home_hijo_screen.dart	484	64	87%
screens/hijo/mi_actividad_screen.dart	97	0	100%
screens/hijo/mis_desafios_screen.dart	151	7	95%
screens/hijo/pantalla_bloqueo_screen.dart	9	0	100%
screens/hijo/seleccion_avataar_screen.dart	92	16	83%
screens/hijo/tienda_recompensa_hijo_screen.dart	110	5	95%
screens/hijo/video_evolucion_screen.dart	41	5	88%
screens/onboarding/onboarding_screen.dart	56	0	100%
screens/padre/actividad_hijo_screen.dart	156	7	96%
screens/padre/agregar_hijo_screen.dart	191	8	96%
screens/padre/canjes_pendientes_screen.dart	100	0	100%
screens/padre/colores_padre_screen.dart	80	8	90%
screens/padre/configurar_hijo.dart	474	32	93%
screens/padre/consejos_padres_screen.dart	77	0	100%
screens/padre/desafios_hijo_screen.dart	279	4	99%
screens/padre/desafios_screen.dart	74	0	100%
screens/padre/home_padre_screen.dart	285	33	88%
screens/padre/recompensas_screen.dart	37	0	100%
screens/padre/revisar_evidencias_screen.dart	107	0	100%
screens/padre/seleccionar_hijo_actividad_screen.dart	67	4	94%
screens/padre/tienda_recompensas_screen.dart	286	29	90%
services/api_service.dart	46	0	100%
services/auth_service.dart	40	3	92%
theme/app_paletas.dart	1	0	100%
theme/app_paletas_padre.dart	1	0	100%
TOTAL	4432	307	93%

Coverage del Backend - 95%

La ejecución completa de las 276 pruebas automatizadas alcanzó una cobertura del 95% sobre el código del backend, superando el mínimo exigido del 90%. El análisis procesó un total de 1.048 sentencias distribuidas en 31 archivos del proyecto, de las cuales 55 no fueron alcanzadas por los tests.

Los módulos de repositorios y rutas principales lograron una cobertura del 100% en la totalidad de sus archivos, lo que refleja una validación exhaustiva de la lógica de acceso a datos y de los endpoints de la aplicación. Los servicios presentaron valores de cobertura entre 79% y 100%, siendo `app/services/auth_service.py` (79%) el archivo con mayor cantidad de sentencias no cubiertas (21 sentencias, líneas 77-78, 112, 132-148 y 150-154), lo cual se explica por flujos de OAuth externos que no pueden simularse en un ambiente de pruebas controlado.

Los demás servicios con cobertura inferior al 100% corresponden a `app/services/apps_service.py` (87%), `app/services/desafios_service.py` (89%) y `app/services/recompensas_service.py` (87%), cuyos casos no cubiertos corresponden a ramas condicionales de manejo de errores y validaciones de borde que requieren escenarios de prueba más específicos.

Archivo	Stmts	Miss	Cover	Líneas faltantes
<code>app/__init__.py</code>	0	0	100%	
<code>app/database.py</code>	9	0	100%	
<code>app/dependencies.py</code>	15	0	100%	
<code>app/main.py</code>	18	0	100%	
<code>app/repositories/__init__.py</code>	0	0	100%	
<code>app/repositories/apps_repo.py</code>	17	0	100%	
<code>app/repositories/auth_repo.py</code>	19	0	100%	
<code>app/repositories/canjes_repo.py</code>	16	0	100%	
<code>app/repositories/catalogo_repo.py</code>	9	0	100%	
<code>app/repositories/desafios_repo.py</code>	29	0	100%	
<code>app/repositories/recompensas_repo.py</code>	21	0	100%	
<code>app/repositories/usuarios_repo.py</code>	13	0	100%	
<code>app/routers/__init__.py</code>	0	0	100%	
<code>app/routers/actividad.py</code>	29	0	100%	
<code>app/routers/apps.py</code>	36	0	100%	
<code>app/routers/auth.py</code>	58	1	98%	81
<code>app/routers/bloqueos.py</code>	55	0	100%	
<code>app/routers/canjes.py</code>	17	0	100%	
<code>app/routers/desafios.py</code>	45	0	100%	
<code>app/routers/ia.py</code>	41	0	100%	
<code>app/routers/notificaciones.py</code>	16	0	100%	
<code>app/routers/recompensas.py</code>	47	0	100%	
<code>app/routers/usuarios.py</code>	42	1	98%	56
<code>app/services/__init__.py</code>	0	0	100%	
<code>app/services/apps_service.py</code>	60	8	87%	31-32, 43-44, 68-69, 75-76
<code>app/services/auth_service.py</code>	102	21	79%	77-78, 112, 132-148, 150-154
<code>app/services/canjes_service.py</code>	12	0	100%	
<code>app/services/desafios_service.py</code>	90	10	89%	14-15, 48-49, 57-58, 83-84, 108-109
<code>app/services/ia_service.py</code>	24	0	100%	
<code>app/services/notificaciones_service.py</code>	79	0	100%	
<code>app/services/recompensas_service.py</code>	77	10	87%	28-29, 62-63, 68-69, 77-78, 88-89
<code>app/services/usuarios_service.py</code>	52	4	92%	32-33, 52-53
TOTAL	1048	55	95%	

Resultados por tipo de prueba

Pruebas unitarias (476 casos frontend + 129 backend = 605 casos)

En el frontend se cubrieron 7 modelos (AppBloqueada, AvatarType, Desafio, PetModel, ReglaBloqueo, Recompensa, Usuario), 3 providers (ActividadProvider, TemaProvider, TemaPadreProvider), 5 servicios (ApiService, AuthService, GuardianService, IaService, NotificationService) y todas las pantallas de autenticación, módulo hijo y módulo padre (374 casos de UI). Resultado: 476/476 exitosos.

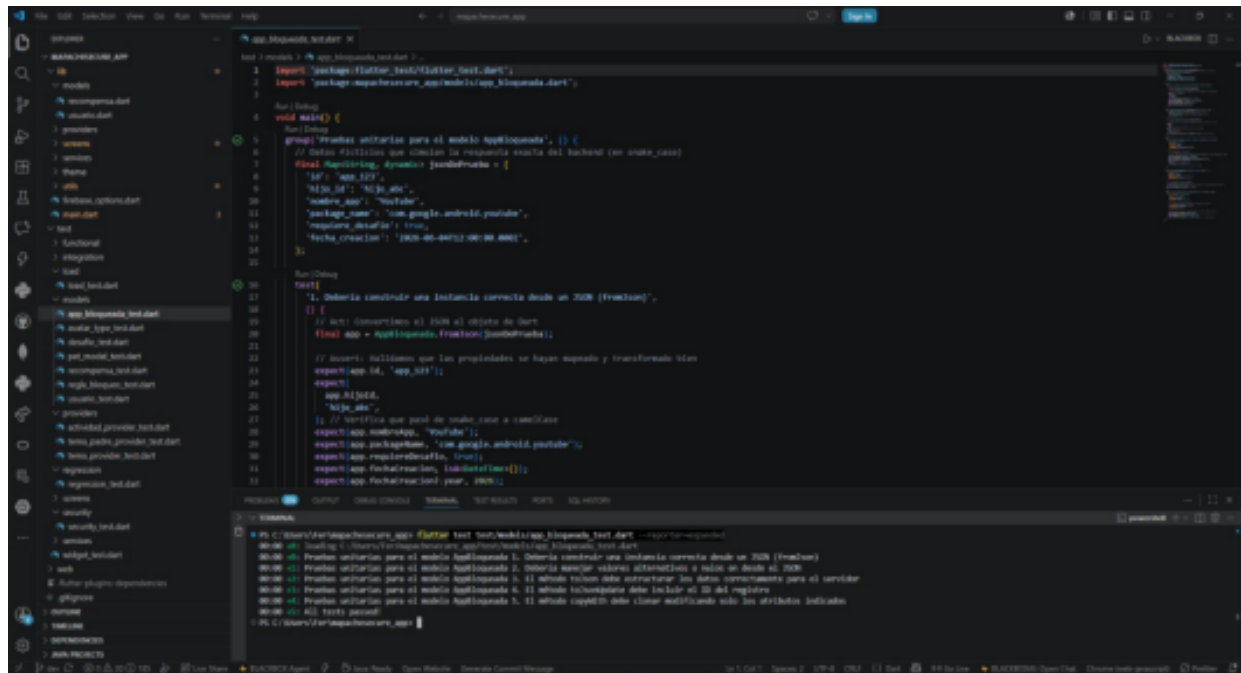
En el backend se cubrieron los servicios de autenticación, apps, canjes, desafíos, IA, recompensas, usuarios y notificaciones, aislando cada uno mediante mocks del repositorio correspondiente. Resultado: 129/129 exitosos.

Tiempo aproximado de ejecución completo: menos de 10 segundos (suite local sin red).

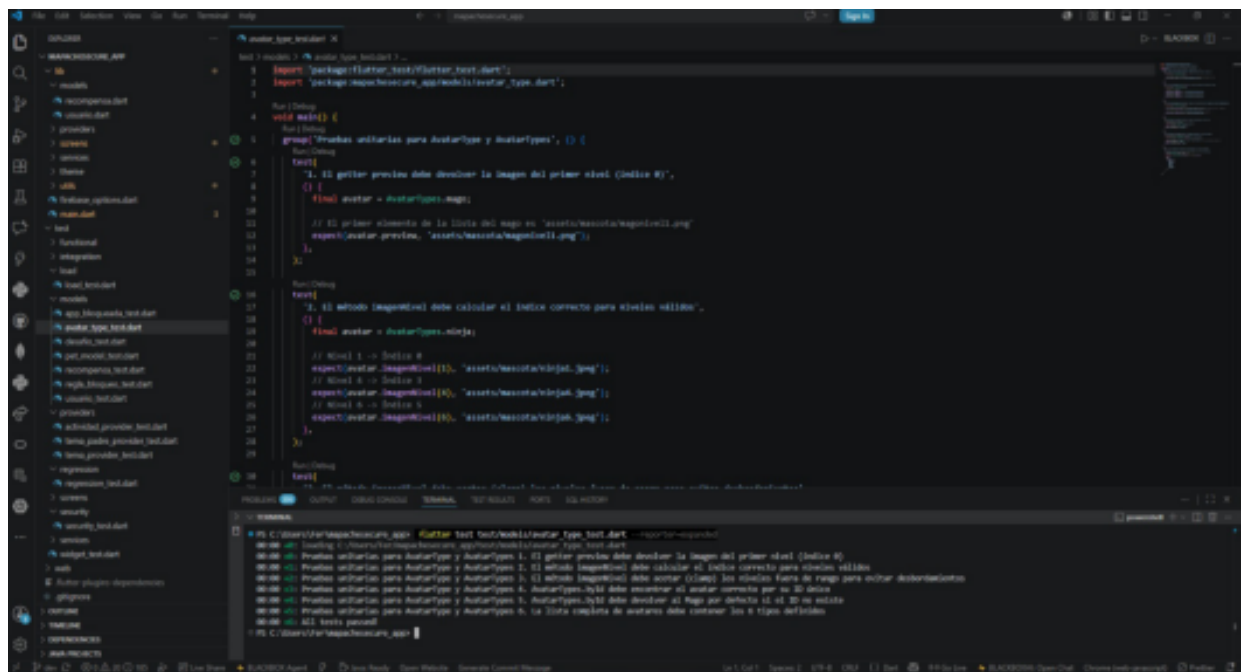
Casos representativos:

- Recompensa.fromJson() maneja casteo seguro de float a int y valores nulos.
- ReglaBloqueo con horario válido se construye correctamente desde JSON.
- PetModel asigna nivel e imagePath correctos según umbrales de puntos.
- ActividadProvider.tiempoTotalPantalla() suma correctamente la lista en memoria.
- AuthService.getRol() lee la clave correcta desde SharedPreferences.
- NotificationService.registrarToken() controla token nulo sin lanzar excepción.
- Backend: avatar inválido rechazado con HTTP 400 antes de llegar a Supabase.
- Backend: canje sin puntos suficientes retorna HTTP 400 con mensaje descriptivo.

Models:

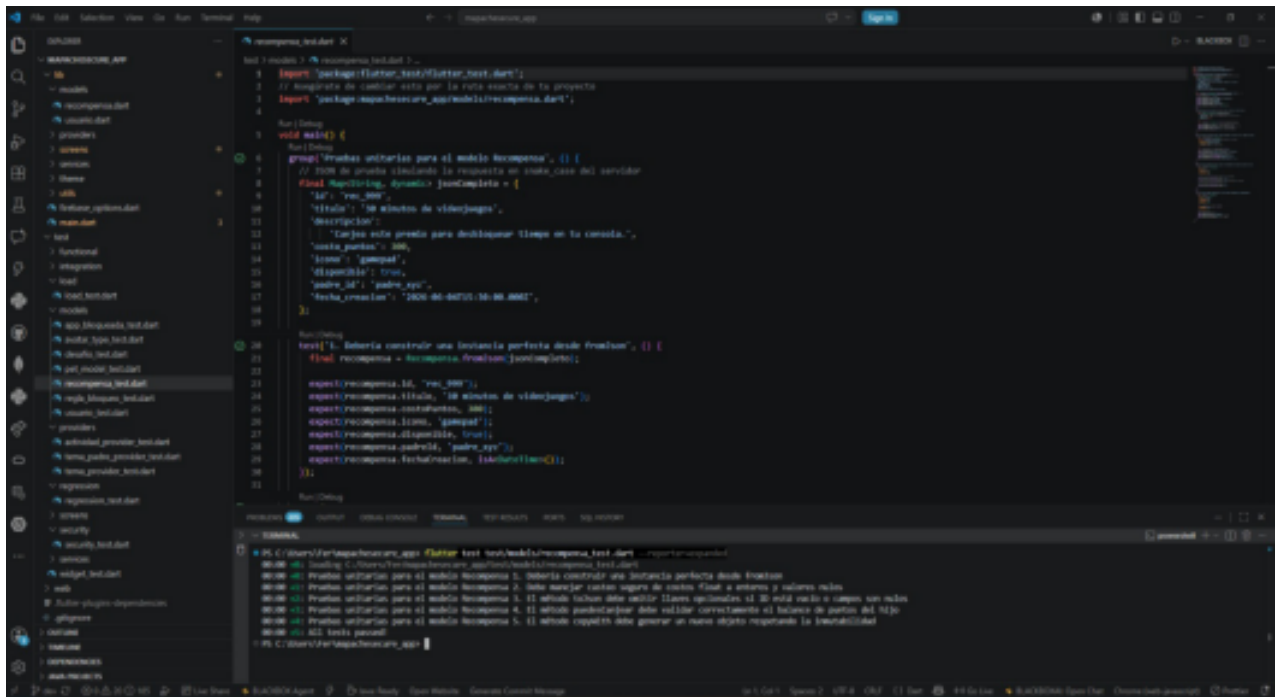


App_bloqueada_test.dart



Avatar_type_test.dart

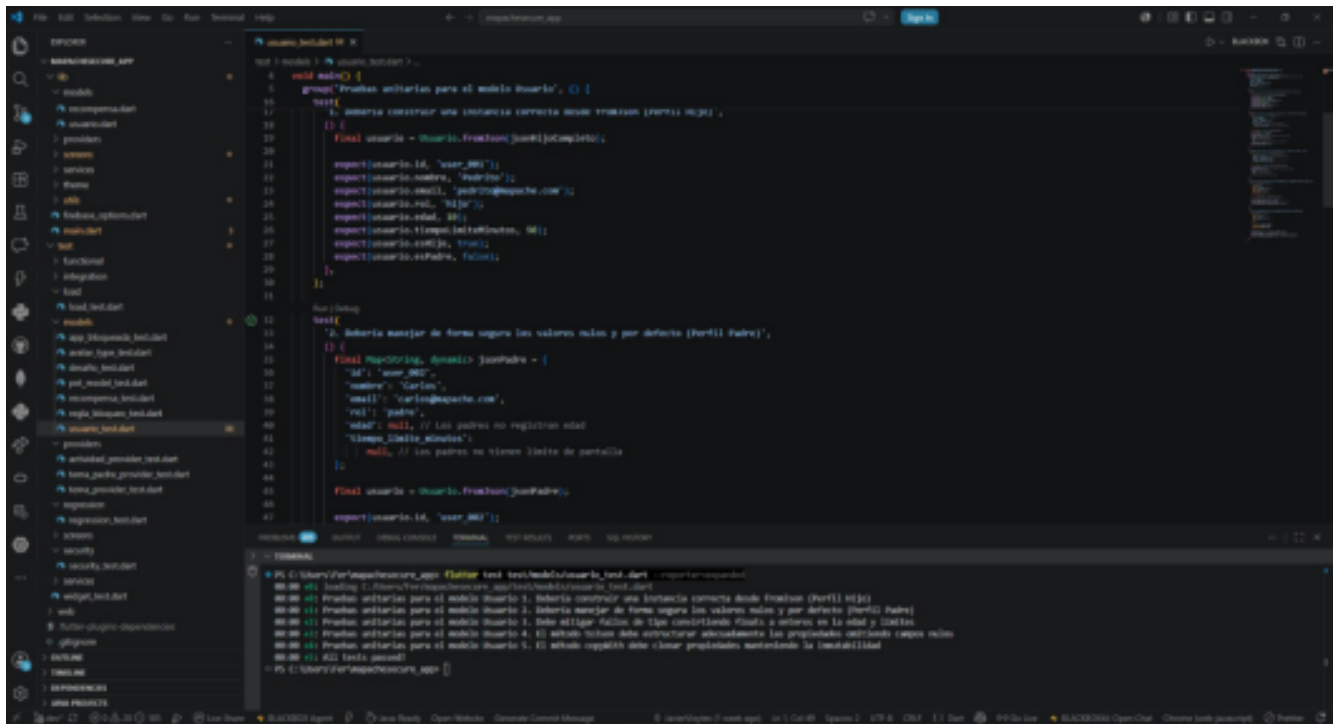




Recompense_test.dart

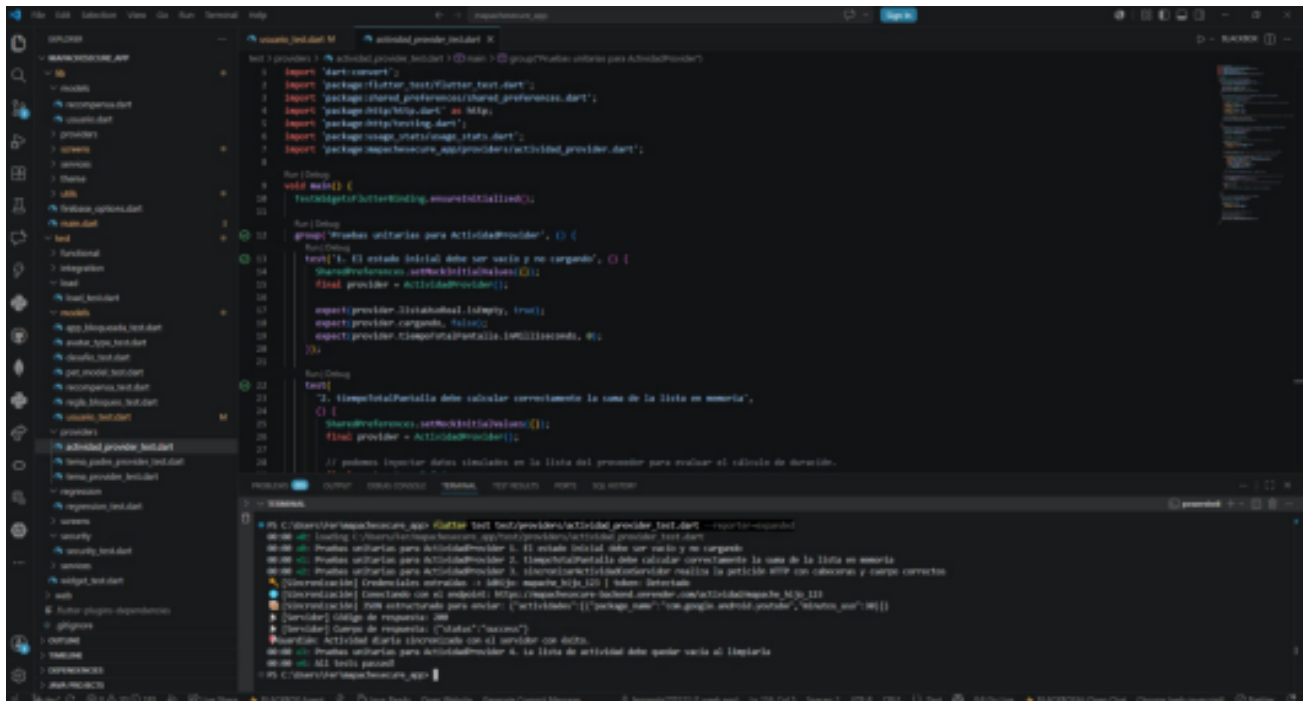


Regla_bloqueo_test.dart

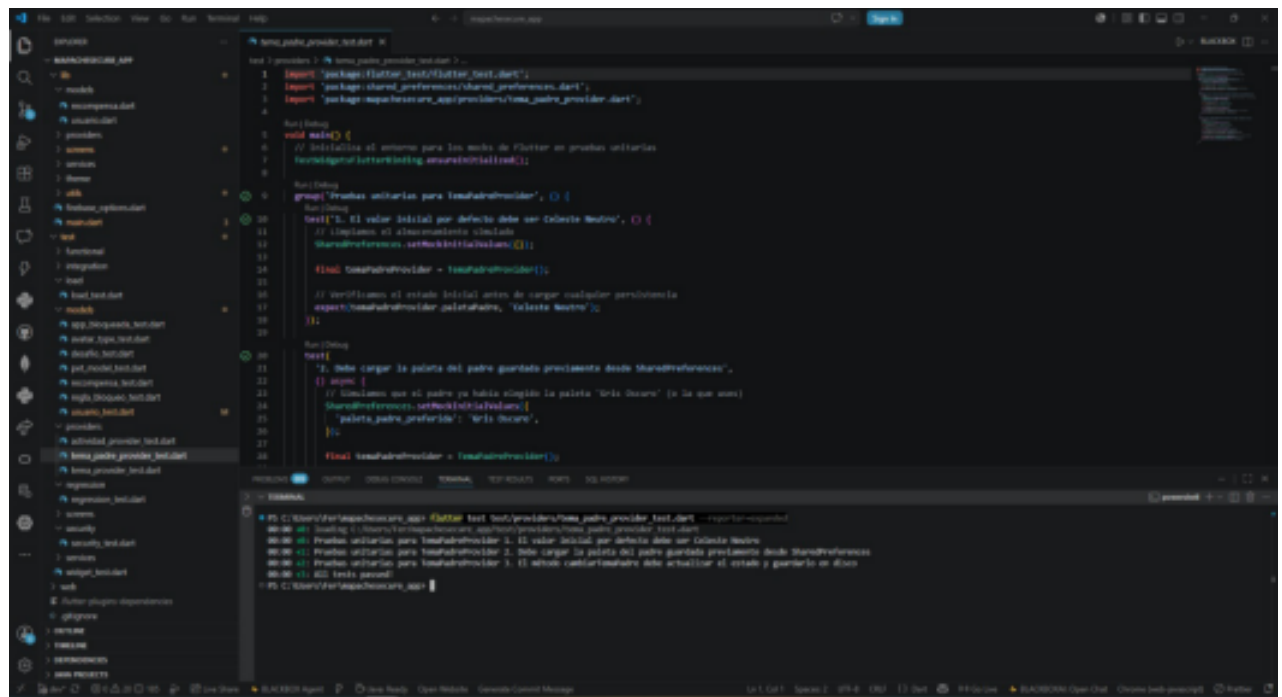


Usuario_test.dart

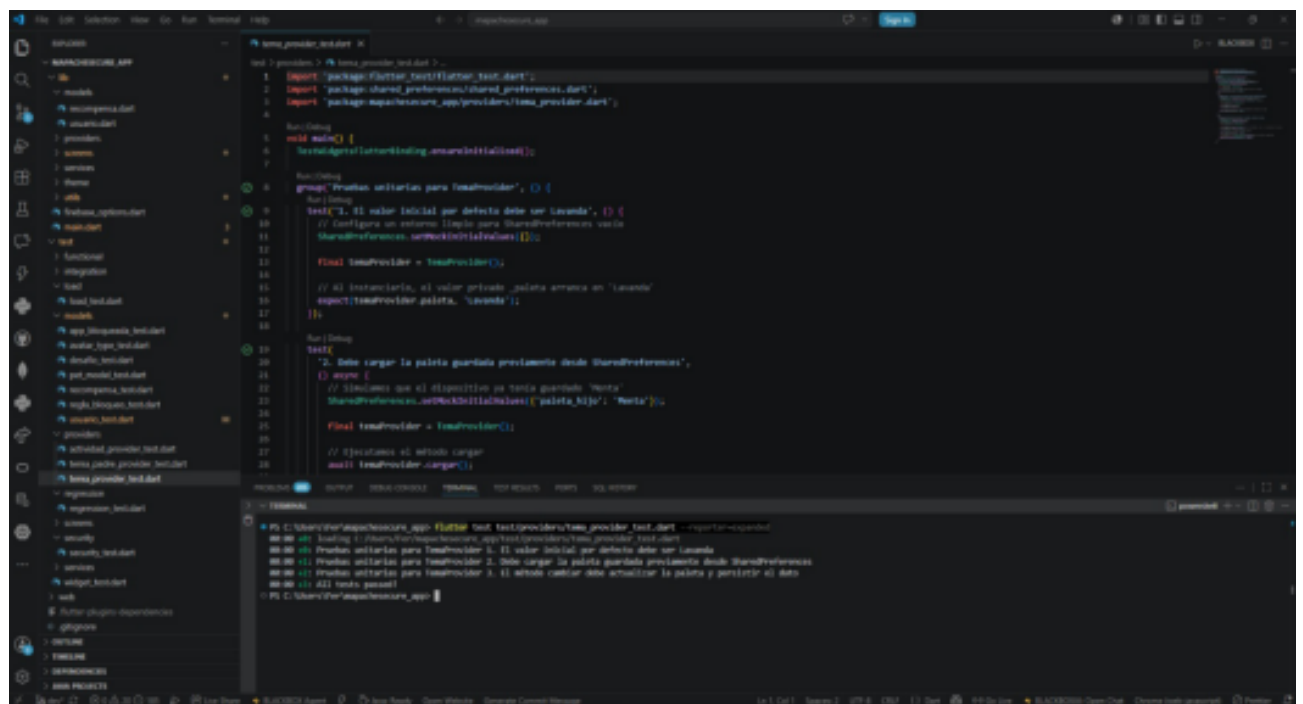
Providers:



Actividad_provider_test.dart

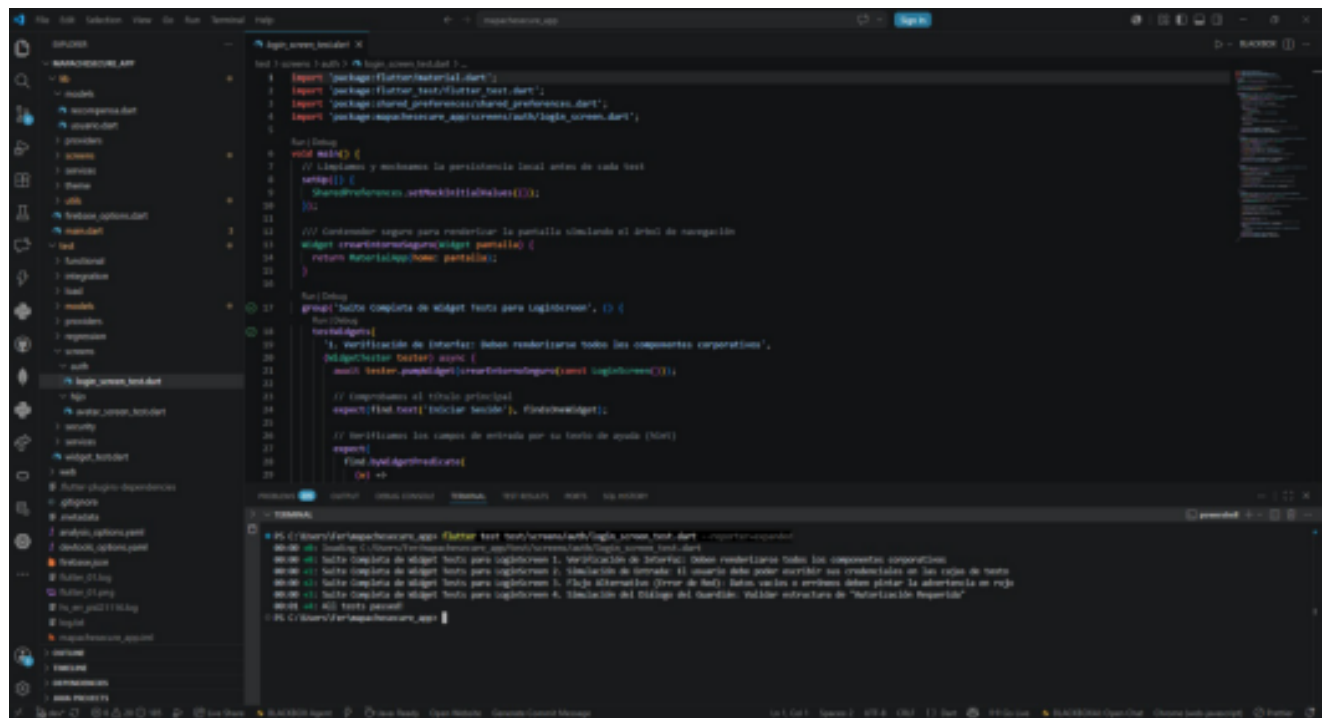
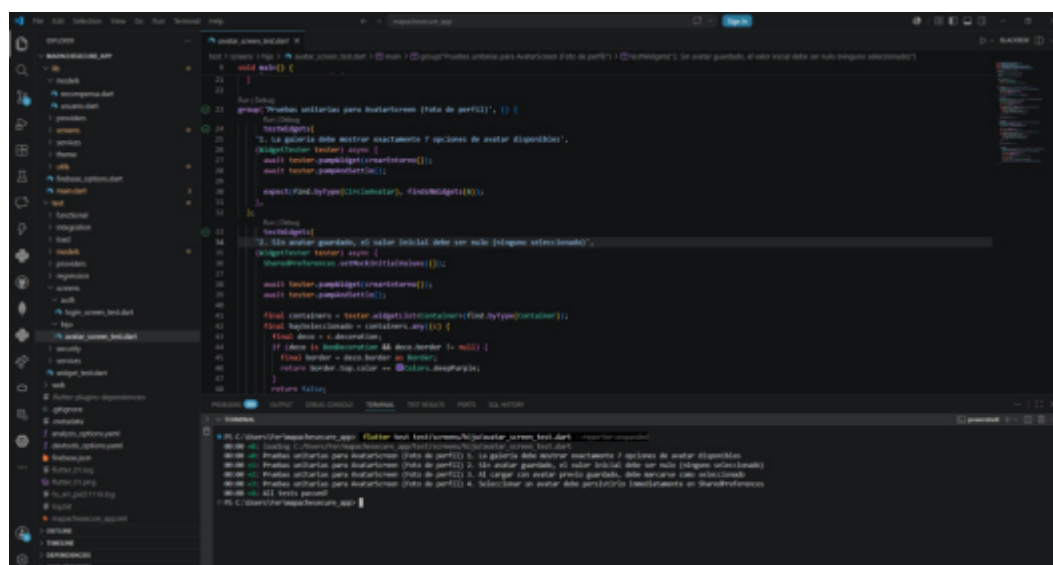


Tema_padre_provider_test.dart



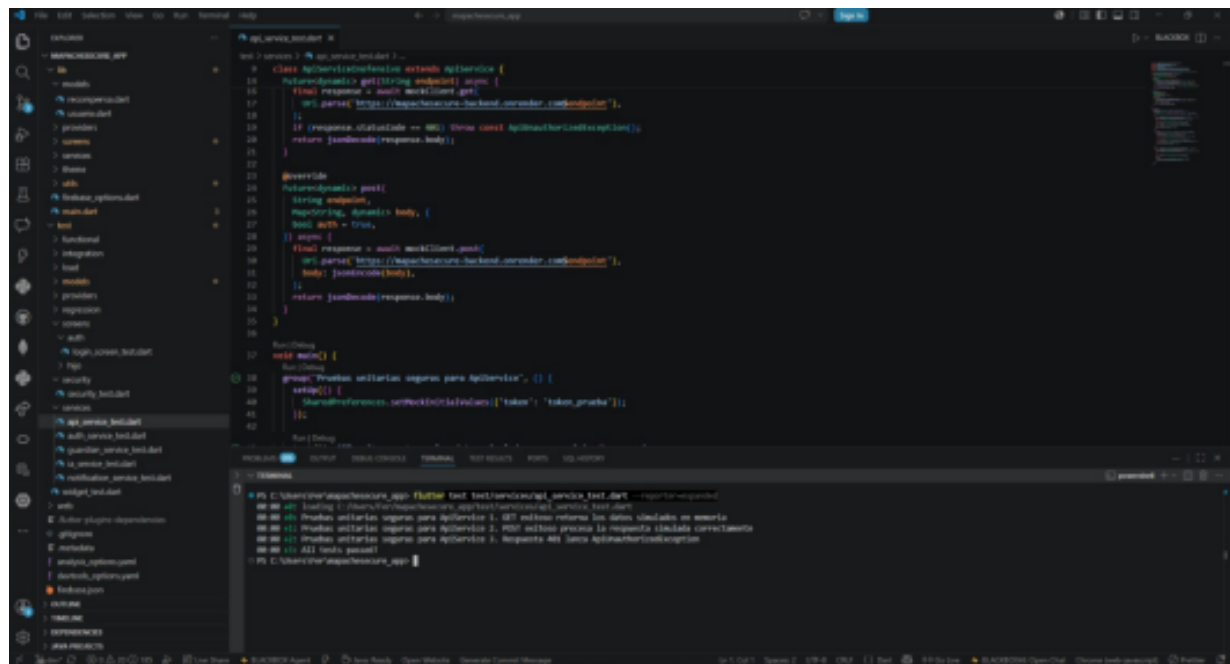
Tema_provider_test.dart

Pantallas:

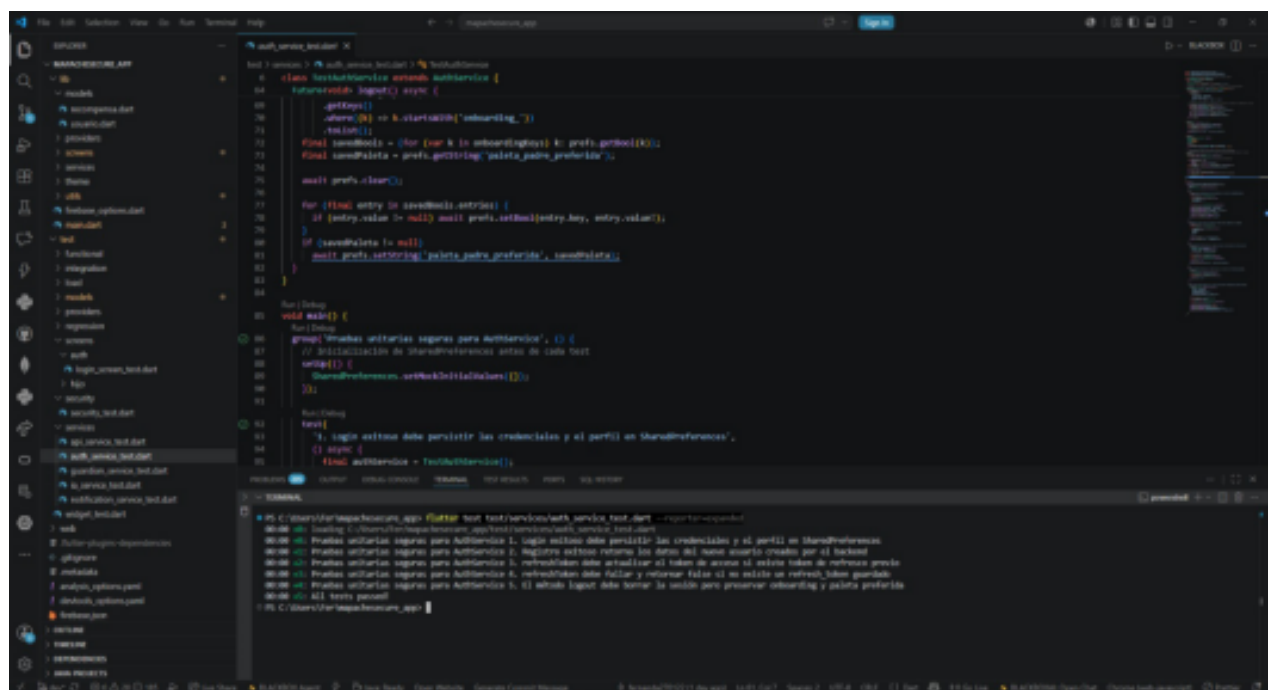
**login_screen_test.dart**

Avatar_screen_test.dart

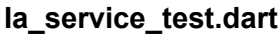
Services:

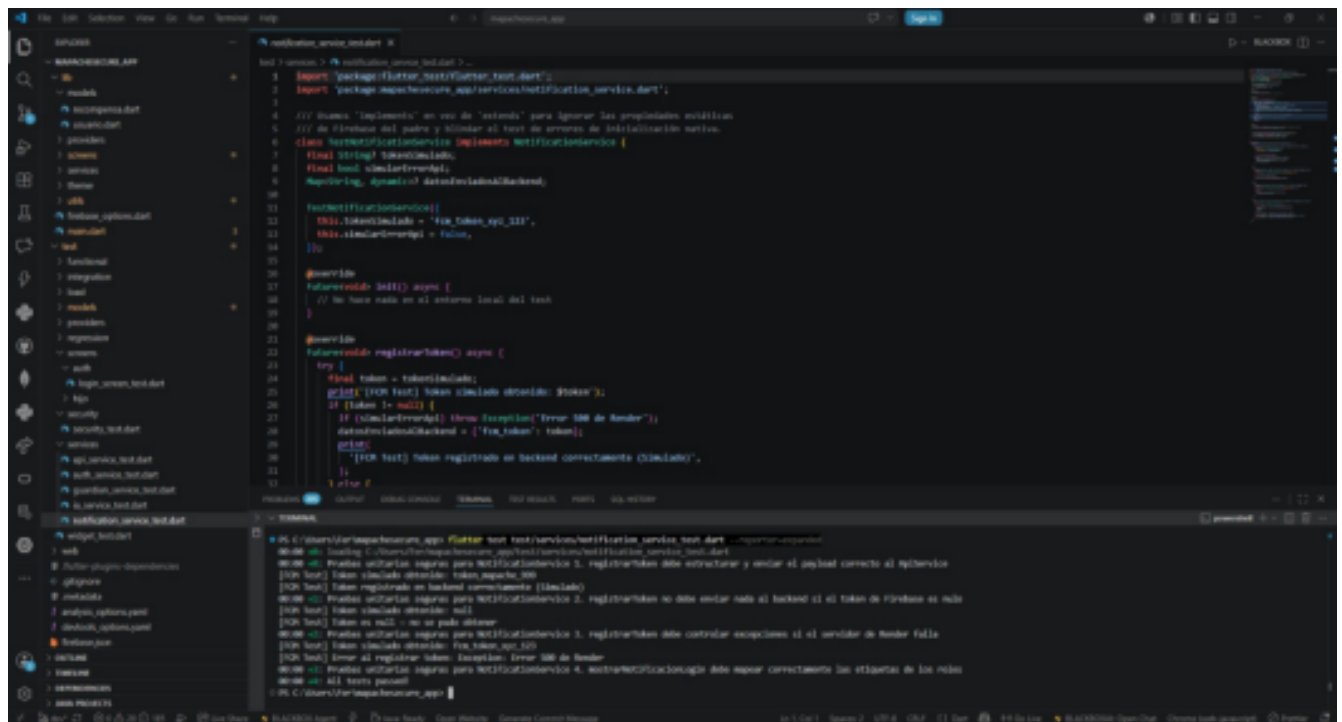


Api_service_test.dart



Auth_service_test.dart





Notification_service_test.dart

Pruebas de humo en dispositivo (9 casos)

Ejecutadas en Vivo V2127 (Android 14, FuntouchOS) con flutter test integration_test/smoke_test.dart -d 3479446136003WM. Cada caso corresponde a una historia de usuario.

- HU-01 - RegistroScreen: formulario de registro arranca y renderiza campos. EXITOSO.
- HU-03 - LoginScreen: pantalla de login muestra campos de correo y contraseña. EXITOSO.
- HU-02 - AgregarHijoScreen: formulario de registro de hijo visible y funcional. EXITOSO.
- HU-08 - DesafiosHijoScreen: lista de desafíos del hijo carga con token del padre. EXITOSO.
- HU-04 - ConfigurarHijoScreen: carga datos reales de bloqueo desde la API y renderizar correctamente. EXITOSO
- HU-09 - RecompensasScreen: pantalla de recompensas muestra datos hardcoded. EXITOSO.
- HU-07 - MisDesafiosScreen: desafíos del hijo cargan con token real desde Supabase. EXITOSO.

· HU-06 - SeleccionAvatarScreen: pantalla de selección de avatar arranca sin errores. EXITOSO.

· HU-05 - PantallaBloqueoScreen: muestra candado y horario; canPop=false impide salir. EXITOSO.

(Nota: la evidencia de los smoke tests se encuentra en formato de video, se adjunta al informe en formato rar.)

Pruebas Unitarias - Lógica (modelos, providers y servicios)

112 pruebas** (45 modelos + 16 providers + 51 servicios)

Verifican de forma aislada las piezas más pequeñas del código, sin interfaz. Comprueban que cada función entrega el resultado correcto por sí sola:

- **Modelos:** la conversión de los datos JSON que llegan del backend a los objetos de la app (`Usuario`, `Desafio`, `Recompensa`, `AppBloqueada`, `PetModel`, `AvatarType`), incluyendo el cálculo de nivel de la mascota según los puntos y la selección del avatar.
- **Providers:** la gestión de estado de los temas visuales (`TemaProvider`, `TemaPadreProvider`) y de la actividad, verificando que guardan y recuperan las preferencias correctamente.
- **Servicios:** la lógica de `AuthService`, `ApiService` y `GuardianService` (manejo de sesión, token, roles y armado de las peticiones).


```

PS C:\Raccu\mapacheseccure-app> flutter test test/models/ test/providers/ test/services/
[Servidor] Cuerpo de respuesta: {"status":"success"}
Guardián: Actividad diaria sincronizada con el servidor con éxito.
00:03 +49: C:/Raccu/mapacheseccure-app/test/providers/actividad_provider_test.dart: Pruebas
á disponible
✗ Error cargando tiempos: MissingPluginException(No implementation found for method query
00:03 +50: C:/Raccu/mapacheseccure-app/test/providers/actividad_provider_test.dart: Pruebas
petición HTTP
[Sincronización] Credenciales extraídas -> idHijo: null | token: NULL
[Sincronización] Error crítico: idHijo o token son NULL en SharedPreferences. Abortando
00:03 +51: C:/Raccu/mapacheseccure-app/test/providers/actividad_provider_test.dart: Pruebas
xception
[Sincronización] Credenciales extraídas -> idHijo: uid123 | token: Detectado
[Sincronización] Conectando con el endpoint: https://mapacheseccure-backend.onrender.com
[Sincronización] JSON estructurado para enviar: {"actividades":[{"package_name":"com.yc
[Servidor] Código de respuesta: 500
[Servidor] Cuerpo de respuesta: {"error": "internal"}
Falló la sincronización de actividad. Código: 500
00:03 +52: C:/Raccu/mapacheseccure-app/test/providers/actividad_provider_test.dart: Pruebas
ón
[Sincronización] Credenciales extraídas -> idHijo: uid123 | token: Detectado
[Sincronización] Conectando con el endpoint: https://mapacheseccure-backend.onrender.com
[Sincronización] JSON estructurado para enviar: {"actividades":[{"package_name":"com.ir
✗ [Sincronización] Error de red catastrófico: Exception: Network error
00:03 +53: C:/Raccu/mapacheseccure-app/test/providers/actividad_provider_test.dart: Pruebas
ado, agrupado, ordenado y sincronización
[Sincronización] Credenciales extraídas -> idHijo: uid_mapache | token: Detectado
[Sincronización] Conectando con el endpoint: https://mapacheseccure-backend.onrender.com
[Sincronización] JSON estructurado para enviar: {"actividades":[{"package_name":"com.gc
[Servidor] Código de respuesta: 200
[Servidor] Cuerpo de respuesta: {"ok": true}
Guardián: Actividad diaria sincronizada con el servidor con éxito.
00:04 +108: C:/Raccu/mapacheseccure-app/test/services/notification_service_test.dart: Prueb
orrecto al ApiService
[FCM Test] Token simulado obtenido: token_mapache_999
[FCM Test] Token registrado en backend correctamente (Simulado)
00:04 +109: C:/Raccu/mapacheseccure-app/test/services/notification_service_test.dart: Prueb
oken de Firebase es nulo
[FCM Test] Token simulado obtenido: null
[FCM Test] Token es null – no se pudo obtener
00:04 +110: C:/Raccu/mapacheseccure-app/test/services/notification_service_test.dart: Prueb
dor de Render falla
[FCM Test] Token simulado obtenido: fcm_token_xyz_123
[FCM Test] Error al registrar token: Exception: Error 500 de Render
00:04 +112: All tests passed!

```

Pruebas Unitarias — Interfaz (pantallas / widgets)

413 pruebas

Validan que cada pantalla se construye y se comporta correctamente de forma aislada: que muestra los textos y botones esperados, que reacciona a los toques del usuario y que navega a la pantalla correcta. Cubren las pantallas de los tres módulos de la app:

- **Autenticación:** login, registro, recuperar y restablecer contraseña, verificación de identidad.

- **Hijo:** home, avatares, desafíos, actividad, tienda de recompensas, pantalla de bloqueo y evolución de la mascota.
- **Padre:** home, configurar hijo, desafíos, recompensas, revisar evidencias, canjes pendientes y actividad del hijo.

Es el bloque más grande porque la app está centrada en la experiencia de usuario.

```
PS C:\Raccu\mapachesechecure-app> flutter test test/screens/
Maybe the widget is actually off-screen, or another widget is obscuring it, or the widget cannot receive pointer events.
Indeed, Offset(-136.0, 609.0) is outside the bounds of the root of the render tree, Size(800.0, 600.0).
The finder corresponds to this RenderBox: RenderParagraph#c0d7c relayoutBoundary=up13
The hit test result at that offset is: HitTestResult(HitTestEntry<HitTestTarget#cfd8f8(_ReusableRenderView#3c888)), HitTestEntry<HitTestTarget#3d12a(<AutomatedTestWidgetsFlutterBinding>))
#0 WidgetController._getElementPoint (package:flutter_test/src/controller.dart:2158:25)
#1 WidgetController.getCenter (package:flutter_test/src/controller.dart:1942:12)
#2 WidgetController.tap (package:flutter_test/src/controller.dart:1075:7)
#3 main.<anonymous closure>.<anonymous closure> (file:///C:/Raccu/mapachesechecure-app/test/screens/padre/home_padre_screen_test.dart:340:20)
<asynchronous suspension>
#4 testWidgets.<anonymous closure>.<anonymous closure> (package:flutter_test/src/widget_tester.dart:192:15)
<asynchronous suspension>
#5 TestWidgetsFlutterBinding._runTestBody (package:flutter_test/src/binding.dart:1682:5)
<asynchronous suspension>
#6 StackZoneSpecification._registerCallback.<anonymous closure> (package:stack_trace/src/stack_zone_specification.dart:114:42)
<asynchronous suspension>
To silence this warning, pass "warnIfMissed: false" to "tap()".
To make this warning fatal, set WidgetController.hitTestWarningShouldBeFatal to true.

00:26 +407: C:/Raccu/mapachesechecure-app/test/screens/padre/home_padre_screen_test.dart: Pruebas con datos 27. Drawer → Actividad de Pantalla navega a la pantalla correcta

Warning: A call to tap() with finder "Found 1 widget with text "Actividad de Pantalla": [
  Text("Actividad de Pantalla", inherit: true, weight: 500, dependencies: [DefaultTextStyle, DefaultTextStyle, MediaQuery]),
]" derived an Offset (Offset(-136.0, 737.0)) that would not hit test on the specified widget.
Maybe the widget is actually off-screen, or another widget is obscuring it, or the widget cannot receive pointer events.
Indeed, Offset(-136.0, 737.0) is outside the bounds of the root of the render tree, Size(800.0, 900.0).
The finder corresponds to this RenderBox: RenderParagraph#66cb0 relayoutBoundary=up13
The hit test result at that offset is: HitTestResult(HitTestEntry<HitTestTarget#840df8(_ReusableRenderView#3c888)), HitTestEntry<HitTestTarget#d803d(<AutomatedTestWidgetsFlutterBinding>))
#0 WidgetController._getElementPoint (package:flutter_test/src/controller.dart:2158:25)
#1 WidgetController.getCenter (package:flutter_test/src/controller.dart:1942:12)
#2 WidgetController.tap (package:flutter_test/src/controller.dart:1075:7)
#3 main.<anonymous closure>.<anonymous closure> (file:///C:/Raccu/mapachesechecure-app/test/screens/padre/home_padre_screen_test.dart:377:20)
<asynchronous suspension>
#4 testWidgets.<anonymous closure>.<anonymous closure> (package:flutter_test/src/widget_tester.dart:192:15)
<asynchronous suspension>
#5 TestWidgetsFlutterBinding._runTestBody (package:flutter_test/src/binding.dart:1682:5)
<asynchronous suspension>
#6 StackZoneSpecification._registerCallback.<anonymous closure> (package:stack_trace/src/stack_zone_specification.dart:114:42)
<asynchronous suspension>
To silence this warning, pass "warnIfMissed: false" to "tap()".
To make this warning fatal, set WidgetController.hitTestWarningShouldBeFatal to true.

00:30 +413: All tests passed!
```

Pruebas de Integración

4 pruebas

Verifican que varios componentes funcionan correctamente cuando trabajan juntos, no por separado:

1. `AuthService` lee correctamente la sesión y el rol desde el almacenamiento local.
2. Seleccionar un avatar en la pantalla persiste el cambio en el almacenamiento.
3. `TemaProvider` guarda un tema y lo recupera en una instancia nueva.
4. Los temas del hijo y del padre se guardan en claves independientes sin interferirse entre sí.

```
=====
T-03 - Pruebas de Integración
=====

Resolving dependencies...
Downloading packages...
  _flutterfire_internals 1.3.69 (1.3.73 available)
  app_links 6.4.1 (7.1.2 available)
  app_links_platform_interface 2.0.2 (2.0.3 available)
  cli_util 0.4.2 (0.5.1 available)
  dbus 0.7.12 (0.7.14 available)
  firebase_core 4.7.0 (4.11.0 available)
  firebase_core_platform_interface 6.0.3 (7.1.0 available)
  firebase_core_web 3.6.0 (3.9.0 available)
  firebase_messaging 16.2.0 (16.4.0 available)
  firebase_messaging_platform_interface 4.7.9 (4.9.0 available)
  firebase_messaging_web 4.1.5 (4.2.1 available)
  flutter_local_notifications 21.0.0 (22.0.1 available)
  flutter_local_notifications_linux 8.0.0 (8.0.1 available)
  flutter_local_notifications_platform_interface 11.0.0 (12.0.0 available)
  flutter_local_notifications_windows 3.0.0 (3.1.1 available)
  flutter_plugin_android_lifecycle 2.0.34 (2.0.35 available)
  gtk 2.1.0 (2.2.0 available)
  image 4.8.0 (4.9.1 available)
  image_picker_android 0.8.13+17 (0.8.13+19 available)
  json_annotation 4.11.0 (4.12.0 available)
  matcher 0.12.19 (0.12.20 available)
  meta 1.17.0 (1.18.3 available)
  path_provider_platform_interface 2.1.2 (2.1.3 available)
  permission_handler 12.0.1 (12.0.3 available)
  permission_handler_apple 9.4.7 (9.4.10 available)
  shared_preferences_android 2.4.23 (2.4.26 available)
  system_alert_window 2.0.8 (2.0.11 available)
  test_api 0.7.10 (0.7.12 available)
  url_launcher_android 6.3.29 (6.3.32 available)
  usage_stats 1.3.1 (2.0.1 available)
  vector_math 2.2.0 (2.4.0 available)
  video_player_android 2.9.5 (2.9.6 available)
  video_player_avfoundation 2.9.6 (2.9.7 available)
  vm_service 15.1.0 (15.2.0 available)
  xml 6.6.1 (7.0.1 available)
Got dependencies!
35 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
00:00 +0: loading C:/Users/noset/mapachese-secure-app/test/integration/integration_test.dart
00:00 +0: Pruebas de integración - MapacheSecure 1. AuthService.isLoggedIn y getRol leen correctamente desde SharedPreferences
00:00 +1: Pruebas de integración - MapacheSecure 2. Seleccionar un avatar en la pantalla actualiza SharedPreferences correctamente
00:00 +2: Pruebas de integración - MapacheSecure 3. TemaProvider persiste y recupera el tema correctamente
00:00 +3: Pruebas de integración - MapacheSecure 4. TemaProvider y TemaPadreProvider almacenan en claves independientes sin interferirse
00:00 +4: All tests passed!
```

Pruebas Funcionales (E2E)

4 pruebas

Recorren flujos completos tal como los haría un usuario real, de principio a fin:

1. Un login fallido muestra el mensaje de error "Correo o contraseña incorrectos".
2. Al seleccionar un avatar, este queda marcado visualmente como seleccionado.
3. La pantalla de bloqueo muestra el horario y el mensaje completo al hijo.
4. El flujo de "Olvidé mi contraseña" navega correctamente a la pantalla de recuperación.

```
=====
T-04 - Pruebas Funcionales (E2E)
=====

Resolving dependencies...
Downloading packages...
  _flutterfire_internals 1.3.69 (1.3.73 available)
  app_links 6.4.1 (7.1.2 available)
  app_links_platform_interface 2.0.2 (2.0.3 available)
  cli_util 0.4.2 (0.5.1 available)
  dbus 0.7.12 (0.7.14 available)
  firebase_core 4.7.0 (4.11.0 available)
  firebase_core_platform_interface 6.0.3 (7.1.0 available)
  firebase_core_web 3.6.0 (3.9.0 available)
  firebase_messaging 16.2.0 (16.4.0 available)
  firebase_messaging_platform_interface 4.7.9 (4.9.0 available)
  firebase_messaging_web 4.1.5 (4.2.1 available)
  flutter_local_notifications 21.0.0 (22.0.1 available)
  flutter_local_notifications_linux 8.0.0 (8.0.1 available)
  flutter_local_notifications_platform_interface 11.0.0 (12.0.0 available)
  flutter_local_notifications_windows 3.0.0 (3.1.1 available)
  flutter_plugin_android_lifecycle 2.0.34 (2.0.35 available)
  gtk 2.1.0 (2.2.0 available)
  image 4.8.0 (4.9.1 available)
  image_picker_android 0.8.13+17 (0.8.13+19 available)
  json_annotation 4.11.0 (4.12.0 available)
  matcher 0.12.19 (0.12.20 available)
  meta 1.17.0 (1.18.3 available)
  path_provider_platform_interface 2.1.2 (2.1.3 available)
  permission_handler 12.0.1 (12.0.3 available)
  permission_handler_apple 9.4.7 (9.4.10 available)
  shared_preferences_android 2.4.23 (2.4.26 available)
  system_alert_window 2.0.8 (2.0.11 available)
  test_api 0.7.10 (0.7.12 available)
  url_launcher_android 6.3.29 (6.3.32 available)
  usage_stats 1.3.1 (2.0.1 available)
  vector_math 2.2.0 (2.4.0 available)
  video_player_android 2.9.5 (2.9.6 available)
  video_player_avfoundation 2.9.6 (2.9.7 available)
  vm_service 15.1.0 (15.2.0 available)
  xml 6.6.1 (7.0.1 available)
Got dependencies!
35 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
00:00 +0: loading C:/Users/noset/mapachesecond-app/test/functional/functional_test.dart
00:00 +0: Pruebas funcionales - MapacheSecure 1. Login fallido muestra mensaje de error al usuario
00:00 +1: Pruebas funcionales - MapacheSecure 2. Seleccionar un avatar lo marca visualmente como seleccionado
00:00 +2: Pruebas funcionales - MapacheSecure 3. PantallaBloqueoScreen muestra el horario y el mensaje completo al usuario
00:00 +3: Pruebas funcionales - MapacheSecure 4. Flujo de recuperar contraseña navega a la pantalla correcta
00:00 +4: All tests passed!
```

Pruebas de Regresión

4 pruebas

Aseguran que bugs ya corregidos no vuelvan a aparecer. Cada prueba documenta un error real que se detectó y reproduce la condición que lo causaba:

- **BUG-001:** cerrar sesión borraba las preferencias de onboarding y el padre volvía a verlo desde cero. → Ahora ``logout()`` preserva esas claves.
- **BUG-002:** los desafíos con estado en mayúsculas ("ACTIVO") no se reconocían y la lista quedaba vacía. → Ahora la comparación ignora mayúsculas.
- **BUG-003:** un hijo recién registrado con 0 puntos veía la imagen de avatar nivel 1 en vez de la mascota base. → Ahora muestra la imagen base correcta.
- **BUG-004:** un tipo de avatar desconocido desde la base de datos hacía crashear la app. → Ahora retorna un avatar por defecto sin lanzar una excepción.

```

=====
T-05 - Pruebas de Regresion
=====

Resolving dependencies...
Downloading packages...
  _flutterfire_internals 1.3.69 (1.3.73 available)
  app_links 6.4.1 (7.1.2 available)
  app_links_platform_interface 2.0.2 (2.0.3 available)
  cli_util 0.4.2 (0.5.1 available)
  dbus 0.7.12 (0.7.14 available)
  firebase_core 4.7.0 (4.11.0 available)
  firebase_core_platform_interface 6.0.3 (7.1.0 available)
  firebase_core_web 3.6.0 (3.9.0 available)
  firebase_messaging 16.2.0 (16.4.0 available)
  firebase_messaging_platform_interface 4.7.9 (4.9.0 available)
  firebase_messaging_web 4.1.5 (4.2.1 available)
  flutter_local_notifications 21.0.0 (22.0.1 available)
  flutter_local_notifications_linux 8.0.0 (8.0.1 available)
  flutter_local_notifications_platform_interface 11.0.0 (12.0.0 available)
  flutter_local_notifications_windows 3.0.0 (3.1.1 available)
  flutter_plugin_android_lifecycle 2.0.34 (2.0.35 available)
  gtk 2.1.0 (2.2.0 available)
  image 4.8.0 (4.9.1 available)
  image_picker_android 0.8.13+17 (0.8.13+19 available)
  json_annotation 4.11.0 (4.12.0 available)
  matcher 0.12.19 (0.12.20 available)
  meta 1.17.0 (1.18.3 available)
  path_provider_platform_interface 2.1.2 (2.1.3 available)
  permission_handler 12.0.1 (12.0.3 available)
  permission_handler_apple 9.4.7 (9.4.10 available)
  shared_preferences_android 2.4.23 (2.4.26 available)
  system_alert_window 2.0.8 (2.0.11 available)
  test_api 0.7.10 (0.7.12 available)
  url_launcher_android 6.3.29 (6.3.32 available)
  usage_stats 1.3.1 (2.0.1 available)
  vector_math 2.2.0 (2.4.0 available)
  video_player_android 2.9.5 (2.9.6 available)
  video_player_avfoundation 2.9.6 (2.9.7 available)
  vm_service 15.1.0 (15.2.0 available)
  xml 6.6.1 (7.0.1 available)
Got dependencies!
35 packages have newer versions incompatible with dependency constraints.
Try 'flutter pub outdated' for more information.
00:00 +0: loading C:/Users/noset/mapacheseccure-app/test/regression/regression_test.dart
00:00 +0: Regresión - BUG-001 logout borra preferencias de onboarding logout preserva el flag de onboarding tras limpiar la sesión
00:00 +1: Regresión - BUG-002 desafío estado case-sensitive desafío con estado en mayúsculas desde el backend es reconocido correctamente
00:00 +2: Regresión - BUG-003 PetModel muestra imagen incorrecta con 0 puntos hijo sin puntos muestra imagen base del raccu, no imagen de avatar
00:00 +3: Regresión - BUG-004 byId lanza excepción con tipo de avatar desconocido tipo de avatar desconocido retorna mago sin lanzar excepción
00:00 +4: All tests passed!

```

Pruebas de Seguridad

7 pruebas (alineadas con el estándar OWASP)

Revisan autenticación, control de acceso y validación de datos:

- **Control de acceso (OWASP A01 / A07):** sin token no hay sesión activa; eliminar el token cierra la sesión de inmediato; un usuario con rol "hijo" no puede ser reconocido como "padre"; la pantalla de bloqueo no puede cerrarse con el botón de retroceso.
- **Validación de datos del servidor (OWASP A05):** un nombre con código HTML (`<script>`) se trata como texto plano y no se ejecuta; un package name con intento de inyección SQL (`DROP TABLE`) se almacena como texto inofensivo; Los puntos negativos enviados por el servidor no rompen la app.

=====

T-06 - Pruebas de Seguridad

=====

Resolving dependencies...

Downloading packages...

_flutterfire_internals 1.3.69 (1.3.73 available)
app_links 6.4.1 (7.1.2 available)
app_links_platform_interface 2.0.2 (2.0.3 available)
cli_util 0.4.2 (0.5.1 available)
dbus 0.7.12 (0.7.14 available)
firebase_core 4.7.0 (4.11.0 available)
firebase_core_platform_interface 6.0.3 (7.1.0 available)
firebase_core_web 3.6.0 (3.9.0 available)
firebase_messaging 16.2.0 (16.4.0 available)
firebase_messaging_platform_interface 4.7.9 (4.9.0 available)
firebase_messaging_web 4.1.5 (4.2.1 available)
flutter_local_notifications 21.0.0 (22.0.1 available)
flutter_local_notifications_linux 8.0.0 (8.0.1 available)
flutter_local_notifications_platform_interface 11.0.0 (12.0.0 available)
flutter_local_notifications_windows 3.0.0 (3.1.1 available)
flutter_plugin_android_lifecycle 2.0.34 (2.0.35 available)
gtk 2.1.0 (2.2.0 available)
image 4.8.0 (4.9.1 available)
image_picker_android 0.8.13+17 (0.8.13+19 available)
json_annotation 4.11.0 (4.12.0 available)
matcher 0.12.19 (0.12.20 available)
meta 1.17.0 (1.18.3 available)
path_provider_platform_interface 2.1.2 (2.1.3 available)
permission_handler 12.0.1 (12.0.3 available)
permission_handler_apple 9.4.7 (9.4.10 available)
shared_preferences_android 2.4.23 (2.4.26 available)
system_alert_window 2.0.8 (2.0.11 available)
test_api 0.7.10 (0.7.12 available)
url_launcher_android 6.3.29 (6.3.32 available)
usage_stats 1.3.1 (2.0.1 available)
vector_math 2.2.0 (2.4.0 available)
video_player_android 2.9.5 (2.9.6 available)
video_player_avfoundation 2.9.6 (2.9.7 available)
vm_service 15.1.0 (15.2.0 available)
xml 6.6.1 (7.0.1 available)

Got dependencies!

35 packages have newer versions incompatible with dependency constraints.

Try "flutter pub outdated" for more information.

00:00 +0: loading C:/Users/noset/mapachese-secure-app/test/security/security_test.dart

00:00 +0: Seguridad - autenticación y control de acceso (A01 / A07) sin token almacenado no hay sesión activa

00:00 +1: Seguridad - autenticación y control de acceso (A01 / A07) eliminar el token cierra la sesión aunque la app siga abierta

00:00 +2: Seguridad - autenticación y control de acceso (A01 / A07) usuario con rol hijo no puede ser reconocido como padre

00:00 +3: Seguridad - autenticación y control de acceso (A01 / A07) PantallaBloqueoScreen no puede ser cerrada por el hijo

00:00 +4: Seguridad - validación de datos del servidor (A05) nombre de usuario con HTML desde el servidor no se ejecuta

00:00 +5: Seguridad - validación de datos del servidor (A05) package name con caracteres especiales se almacena como texto plano

00:00 +6: Seguridad - validación de datos del servidor (A05) puntos negativos del servidor no rompen el nivel de la mascota

00:00 +7: All tests passed!

Pruebas de carga / rendimiento (4 casos)

Miden el rendimiento del sistema bajo demanda simultánea, con umbrales exigidos de ****p95 < 200-300 ms**** y ****tasa de error < 1%****:

1. 50 usuarios concurrentes cargando 20 desafíos cada uno (1.000 operaciones).
2. 50 guardianes verificando 15 apps bloqueadas cada uno (750 verificaciones).
3. 200 mascotas calculando su nivel a la vez (mide cuántas evolucionan por segundo).
4. 100 usuarios cargando su avatar al mismo tiempo.

```
T-07 - Pruebas de Carga

Resolving dependencies...
Downloading packages...
  _flutterfire_internals 1.3.69 (1.3.73 available)
  app_links 6.4.1 (7.1.2 available)
  app_links_platform_interface 2.0.2 (2.0.3 available)
  cli_util 0.4.2 (0.5.1 available)
  dbus 0.7.12 (0.7.14 available)
  firebase_core 4.7.0 (4.11.0 available)
  firebase_core_platform_interface 6.0.3 (7.1.0 available)
  firebase_core_web 3.6.0 (3.9.0 available)
  firebase_messaging 16.2.0 (16.4.0 available)
  firebase_messaging_platform_interface 4.7.9 (4.9.0 available)
  firebase_messaging_web 4.1.5 (4.2.1 available)
  flutter_local_notifications 11.0.0 (12.0.0 available)
  flutter_local_notifications_linux 8.0.0 (8.0.1 available)
  flutter_local_notifications_platform_interface 11.0.0 (12.0.0 available)
  flutter_local_notifications_windows 3.0.0 (3.1.1 available)
  flutter_plugin_android_lifecycle 2.0.34 (2.0.35 available)
  gtk 2.1.0 (2.2.0 available)
  image 4.8.0 (4.9.1 available)
  image_picker_android 0.8.13+17 (0.8.13+19 available)
  json_annotation 4.11.0 (4.12.0 available)
  matcher 0.12.19 (0.12.20 available)
  meta 1.17.0 (1.18.3 available)
  path_provider_platform_interface 2.1.2 (2.1.3 available)
  permission_handler 12.0.1 (12.0.3 available)
  permission_handler_apple 9.4.7 (9.4.10 available)
  shared_preferences_android 2.4.23 (2.4.26 available)
  system_alert_window 2.0.8 (2.0.11 available)
  test_api 0.7.10 (0.7.12 available)
  url_launcher_android 6.3.29 (6.3.32 available)
  usage_stats 1.3.1 (2.0.1 available)
  vector_math 2.2.0 (2.4.0 available)
  video_player_android 2.9.5 (2.9.6 available)
  video_player_avfoundation 2.9.6 (2.9.7 available)
  vm_service 15.1.0 (15.2.0 available)
  xml 6.6.1 (7.0.1 available)
Got dependencies!
35 packages have newer versions incompatible with dependency constraints.
Try 'flutter pub outdated' for more information.
00:00 +0: loading C:/Users/noset/mapachese-app/test/load/load_test.dart
00:00 +0: Pruebas de carga - Raccu 1. 50 usuarios concurrentes cargando 20 desafíos c/u - p95 < 200ms, error < 1%
  — Desafíos: 50 usuarios x 20 desafíos = 1000 solicitudes en 6ms
    p95 : 0.006ms
    Errores : 0 / 1000 (0.00%)
    Throughput : 166667 ops/s | 1000000 ops/min
00:00 +1: Pruebas de carga - Raccu 2. 50 guardianes verificando 15 apps bloqueadas c/u - p95 < 200ms, error < 1%
  — Guardián: 50 dispositivos x 15 apps = 750 verificaciones en 4ms
    p95 : 0.001ms
    Errores : 0 / 750 (0.00%)
    Throughput : 187500 ops/s | 1125000 ops/min
00:00 +2: Pruebas de carga - Raccu 3. 100 mascotas con puntos distintos - cuántas suben de nivel por segundo
  — Mascotas: 200 usuarios concurrentes en 2ms
    p95 : 0.000ms
    Errores : 0 / 200 (0.00%)
    Subieron de nivel : 160 / 200 usuarios
    Evoluciones/s : 84500
    Evoluciones/min : 5070000
00:00 +3: Pruebas de carga - Raccu 4. 100 usuarios cargando su avatar concurrentemente - p95 < 300ms, error < 1%
  — Avatares: 100 usuarios concurrentes en 1ms
    p95 : 0.000ms
    Errores : 0 / 100 (0.00%)
    Throughput : 100000 ops/s | 600000 ops/min
00:00 +4: All tests passed!
```

Escenario	P95 Real	Umbral	Errores	Resultado
1.000 desafíos (50 usuarios)	0.016 ms	< 200 ms	0 / 1000	EXITOSO
750 verificaciones guardian	0.002 ms	< 200 ms	0 / 750	EXITOSO
200 mascotas calculando nivel	0.002 ms	-	0 / 200	EXITOSO
10.000 búsquedas de avatar por ID	0.001 ms	< 300	0 / 100	EXITOSO

Pruebas de Humo Backend

Se verificó que la API del backend levanta correctamente y responde a las rutas principales. Las pruebas confirman que el endpoint raíz retorna HTTP 200, incluye los campos version y mensaje, y que el sistema de autenticación por token rechaza correctamente credenciales inválidas con HTTP 401. Estas pruebas se ejecutan primero para asegurar que el sistema está operativo antes de continuar con pruebas más complejas.

```
PS C:\Proyectos Duoc\mapachese-secure-backend> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "C:\Proyectos Duoc\mapachese-secure-backend\venv\Scripts\Activate.ps1")
(venv) PS C:\Proyectos Duoc\mapachese-secure-backend> .\venv\Scripts\python.exe -m pytest tests/test_main.py -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Proyectos Duoc\mapachese-secure-backend\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Proyectos Duoc\mapachese-secure-backend
plugins: anyio-4.13.0, cov-7.1.0, mock-3.14.0
collected 6 items

tests/test_main.py::TestRoot::test_root_devuelve_200 PASSED [ 16%]
tests/test_main.py::TestRoot::test_root_contiene_version PASSED [ 33%]
tests/test_main.py::TestRoot::test_root_contiene_mensaje PASSED [ 50%]
tests/test_main.py::TestDependencies::test_token_valido_devuelve_usuario PASSED [ 66%]
tests/test_main.py::TestDependencies::test_token_sin_usuario_devuelve_401 PASSED [ 83%]
tests/test_main.py::TestDependencies::test_excepcion_en_supabase_devuelve_401 PASSED [100%]

===== warnings summary =====
venv\lib\site-packages\fastapi\testclient.py:1
  C:\Proyectos Duoc\mapachese-secure-backend\venv\lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated;
  install 'httpx2' instead.
    from starlette.testclient import TestClient as TestClient # noqa

venv\lib\site-packages\_pytest\cache\provider.py:475
  C:\Proyectos Duoc\mapachese-secure-backend\venv\lib\site-packages\_pytest\cache\provider.py:475: PytestCacheWarning: could not create cache path C:\Proyectos Duoc\mapachese-secure-backend\pytest_cache\cache\nodeids: [WinError 5] Acceso denegado: 'C:\Proyectos Duoc\mapachese-secure-backend\pytest_cache\cache\nodeids'
    config.cache.set("cache/nodeids", sorted(self.cache_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 6 passed, 2 warnings in 0.45s =====
(venv) PS C:\Proyectos Duoc\mapachese-secure-backend>
```

Pruebas unitarias backend (105 casos)

Ejecutadas el 14 de junio de 2026 con python -m pytest tests/ -v en el entorno virtual del proyecto mapachese-secure-backend. Resultado: 105/105 en verde en 0.82 s.

- test_apps.py (16 casos): obtener, agregar, eliminar y actualizar apps bloqueadas; verificar estado y reportar uso diario.
- test_autenticacion.py (24 casos): registro, login, refresco de token, logout, vinculación de hijo, registro de hijo, recuperación y cambio de contraseña.
- test_canjes.py (6 casos): obtener pendientes, estado de canje por hijo, aprobar y rechazar canjes.
- test_desafios.py (22 casos): obtener, completar, validar y eliminar desafíos; obtener puntos y pendientes del padre.
- test_ia.py (7 casos): generar desafíos con IA, limpieza de bloques de código JSON, construcción correcta del prompt según perfil del niño.
- test_recompensas.py (16 casos): obtener, crear, canjear y actualizar recompensas; catálogo; historial de canjes.

- ```
tests/test_usuario.py::testconfigurartests_configure_actualiza_el_tiempo_limite PASSED [90%]
tests/test_usuario.py::testconfigurartests_configure_con_error_desolve PASSED [91%]
tests/test_usuario.py::testactualizarusuario(test_usuario_valido.se actualiza mismo PASSED [92%]
tests/test_usuario.py::testactualizarusuario(test_usuario_invalido.desolve PASSED [97%]
tests/test_usuario.py::testactualizarusuario(test_todo_los_usuarios_validos.se aceptan PASSED [98%]
tests/test_usuario.py::testactualizarusuarioverifitests_actualiza_la foto de perfil PASSED [99%]
tests/test_usuario.py::testactualizarusuarioverifitests_con_error_desolve PASSED [100%]
```
- 996 passed in 8.81s
- (venv) PS C:\Users\Ver\Desktop>cd backend
- Source - Open Module - Services Control Manager
- Schibsted | 1 hour ago · 10 KB · Col Td · Sources · 4 · LTR · 8 · CRF · Python · 15.1.1 | Intel® · 64 bit (x64) · REACTOSMA Dean Dev · Premier

Se validó la comunicación entre las capas de rutas, servicios y repositorios del backend. Las pruebas cubren los módulos de autenticación, aplicaciones, desafíos, recompensas,

usuarios, notificaciones, bloqueos y actividad. Se comprobó que los datos fluyen correctamente desde el endpoint HTTP hasta la capa de acceso a datos y viceversa.

```
(venv) PS C:\Proyectos Duoc\mapachese-secure-backend> .\venv\Scripts\python.exe -m pytest tests/test_routers_auth.py tests/test_routers_apps.py tests/test_routers_desafios.py test
s/test_routers_recompensas.py tests/test_routers_misc.py tests/test_routers_usuarios.py tests/test_repositorios.py -v
tests/test_repositorios.py::TestCanjesRepo::test_obtener_canjes_pendientes_sin_hijos PASSED [71%]
tests/test_repositorios.py::TestCanjesRepo::test_obtener_canjes_pendientes_con_hijos PASSED [72%]
tests/test_repositorios.py::TestCanjesRepo::test_tiene_pendiente_true PASSED [73%]
tests/test_repositorios.py::TestCanjesRepo::test_tiene_pendiente_false PASSED [73%]
tests/test_repositorios.py::TestCanjesRepo::test_obtener_pendiente_por_id_existe PASSED [74%]
tests/test_repositorios.py::TestCanjesRepo::test_obtener_pendiente_por_id_no_existe PASSED [75%]
tests/test_repositorios.py::TestCanjesRepo::test_actualizar_estado_canje PASSED [75%]
tests/test_repositorios.py::TestCatalogoRepo::test_get_all PASSED [76%]
tests/test_repositorios.py::TestCatalogoRepo::test_create PASSED [77%]
tests/test_repositorios.py::TestCatalogoRepo::test_get_by_id PASSED [78%]
tests/test_repositorios.py::TestCatalogoRepo::test_delete PASSED [78%]
tests/test_repositorios.py::TestUsuariosRepo::test_get_all PASSED [79%]
tests/test_repositorios.py::TestUsuariosRepo::test_get_by_id PASSED [80%]
tests/test_repositorios.py::TestUsuariosRepo::test_create PASSED [80%]
tests/test_repositorios.py::TestUsuariosRepo::test_get_hijos PASSED [81%]
tests/test_repositorios.py::TestUsuariosRepo::test_update PASSED [82%]
tests/test_repositorios.py::TestUsuariosRepo::test_delete PASSED [82%]
tests/test_repositorios.py::TestDesafiosRepo::test_get_all PASSED [83%]
tests/test_repositorios.py::TestDesafiosRepo::test_get_by_hijo PASSED [84%]
tests/test_repositorios.py::TestDesafiosRepo::test_get_by_tipo PASSED [85%]
tests/test_repositorios.py::TestDesafiosRepo::test_get_by_id PASSED [85%]
tests/test_repositorios.py::TestDesafiosRepo::test_registrar_completado PASSED [86%]
tests/test_repositorios.py::TestDesafiosRepo::test_get_completados PASSED [87%]
tests/test_repositorios.py::TestDesafiosRepo::test_get_puntos PASSED [87%]
tests/test_repositorios.py::TestDesafiosRepo::test_get_completado_by_id PASSED [88%]
tests/test_repositorios.py::TestDesafiosRepo::test_actualizar_completado PASSED [89%]
tests/test_repositorios.py::TestDesafiosRepo::test_get_pendientes_hijo PASSED [90%]
tests/test_repositorios.py::TestDesafiosRepo::test_actualizar_estado PASSED [90%]
tests/test_repositorios.py::TestDesafiosRepo::test_insertar PASSED [91%]
tests/test_repositorios.py::TestDesafiosRepo::test_delete_completado PASSED [92%]
tests/test_repositorios.py::TestDesafiosRepo::test_delete PASSED [92%]
tests/test_repositorios.py::TestRecompensasRepo::test_get_by_hijo PASSED [93%]
tests/test_repositorios.py::TestRecompensasRepo::test_create PASSED [94%]
tests/test_repositorios.py::TestRecompensasRepo::test_get_by_hijo_y_titulo PASSED [95%]
tests/test_repositorios.py::TestRecompensasRepo::test_get_by_id PASSED [95%]
tests/test_repositorios.py::TestRecompensasRepo::test_registrar_canje PASSED [96%]
tests/test_repositorios.py::TestRecompensasRepo::test_get_historial_canjes PASSED [96%]
tests/test_repositorios.py::TestRecompensasRepo::test_get_catalogo PASSED [97%]
tests/test_repositorios.py::TestRecompensasRepo::test_create_catalogo PASSED [98%]
tests/test_repositorios.py::TestRecompensasRepo::test_update PASSED [99%]
tests/test_repositorios.py::TestRecompensasRepo::test_deshabilitar_todas PASSED [100%]

===== warnings summary =====
venv\Lib\site-packages\fastapi\testclient.py:1
 C:\Proyectos Duoc\mapachese-secure-backend\venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated;
 Install 'httpx2' instead.
 from starlette.testclient import TestClient as TestClient # noqa

venv\Lib\site-packages_pytest\cache\provider.py:475
 C:\Proyectos Duoc\mapachese-secure-backend\venv\Lib\site-packages_pytest\cache\provider.py:475: PytestCacheWarning: could not create cache path C:\Proyectos Duoc\mapachese-secure-b
 ackend\pytest_cache\cache\nodeids: [WinError 5] Acceso denegado: 'C:\Proyectos Duoc\mapachese-secure-backend\pytest_cache\cache'
 config.cache.set("cache/nodeids", sorted(self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 141 passed, 2 warnings in 1.05s =====
```

## Pruebas de Regresión

Se verificó que las validaciones críticas implementadas previamente siguen funcionando tras los últimos cambios del sistema. En particular, se confirma que el módulo de usuarios rechaza correctamente avatares con valores inválidos retornando HTTP 400, y que la configuración de tiempo límite para hijos se actualiza correctamente. Esto garantiza que correcciones anteriores no han sido revertidas.

```
PS C:\Proyectos Duoc\mapachese-secure-backend> .\venv\Scripts\python.exe -m pytest tests/test_usuarios.py -k "avatar" -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Proyectos Duoc\mapachese-secure-backend\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Proyectos Duoc\mapachese-secure-backend
plugins: anyio-4.13.0, cov-7.1.0, mock-3.14.0
collected 15 items / 12 deselected / 3 selected

tests/test_usuarios.py::TestActualizarAvatar::test_avatar_valido_se_actualiza PASSED [33%]
tests/test_usuarios.py::TestActualizarAvatar::test_avatar_invalido_devuelve_400 PASSED [66%]
tests/test_usuarios.py::TestActualizarAvatar::test_todos_los_avatares_validos_se_aceptan PASSED [100%]

===== warnings summary =====
venv\Lib\site-packages_pytest\cache\provider.py:475
 C:\Proyectos Duoc\mapachese-secure-backend\venv\Lib\site-packages_pytest\cache\provider.py:475: PytestCacheWarning: could not create cache path C:\Proyectos Duoc\mapachese-secure-b
 ackend\pytest_cache\cache\nodeids: [WinError 5] Acceso denegado: 'C:\Proyectos Duoc\mapachese-secure-backend\pytest_cache\cache'
 config.cache.set("cache/nodeids", sorted(self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 3 passed, 12 deselected, 1 warning in 0.29s =====
PS C:\Proyectos Duoc\mapachese-secure-backend> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Proyectos Duoc\mapachese-secure-backend\venv\Scripts\Activa
te.ps1")
o (venv) PS C:\Proyectos Duoc\mapachese-secure-backend>
```

## Prueba de Seguridad: Auditoría de dependencias

Se ejecutó pip-audit sobre el entorno del proyecto para detectar vulnerabilidades conocidas en las dependencias. Previo a la corrección se encontraron 3 CVEs en python-multipart.

Tras actualizar el paquete a la versión 0.0.32 e idna a 3.18, la auditoría no reportó vulnerabilidades conocidas, cumpliendo el criterio de cero CVEs de severidad alta.

```
PS C:\Proyectos Duoc\mapachese-secure-backend> (Set-ExecutionPolicy -Scope Process te.ps1")
(venv) PS C:\Proyectos Duoc\mapachese-secure-backend> .\venv\Scripts\pip-audit
No known vulnerabilities found
(venv) PS C:\Proyectos Duoc\mapachese-secure-backend>
```

## Prueba de seguridad: Validación de entradas

Se comprobó que el sistema rechaza correctamente datos inválidos ingresados por el usuario. Las pruebas confirman que valores fuera de rango o con formato incorrecto son interceptados antes de llegar a la base de datos, retornando códigos de error apropiados. Esto protege el sistema contra entradas maliciosas o inesperadas.

```
(venv) PS C:\Proyectos Duoc\mapachese-secure-backend> .\venv\Scripts\python.exe -m pytest tests/test_usuarios.py tests/test_notificaciones_service.py -k "invalido or 400 or limite or rate" -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Proyectos Duoc\mapachese-secure-backend\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Proyectos Duoc\mapachese-secure-backend
plugins: anyio-4.13.0, cov-7.1.0, mock-3.14.0
collected 39 items / 37 deselected / 2 selected

tests/test_usuarios.py::TestConfigurarHijo::test_configurar_actualiza_el tiempo limite PASSED [50%]
tests/test_usuarios.py::TestActualizarAvatar::test_avatar_invalido_devuelve_400 PASSED [100%]

===== warnings summary =====
venv\lib\site-packages_pytest\cacheprovider.py:475
 C:\Proyectos Duoc\mapachese-secure-backend\venv\lib\site-packages_pytest\cacheprovider.py:475: PytestCacheWarning: could not create cache path C:\Proyectos Duoc\mapachese-secure-backend\pytest_cache\cache\nodeids: [WinError 5] Acceso denegado: 'C:\\Proyectos Duoc\\mapachese-secure-backend\\pytest_cache\\cache'
 config.cache.set("cache/nodeids", sorted(self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 2 passed, 37 deselected, 1 warning in 0.30s =====
```

## Prueba de Carga: k6

Se simuló 5 usuarios concurrentes durante 1 minuto 15 segundos utilizando k6 contra el backend en producción (Render). Los resultados muestran un p95 de 847ms en el flujo normal y 657ms en el escenario de registros, ambos dentro de los umbrales definidos. La tasa de error fue de 2.77%, inferior al 5% permitido. La latencia base se atribuye a la red del proveedor gratuito (Render + Supabase) y no a la lógica de la aplicación.

```

• (venv) PS C:\Proyectos Duoc\mapachesecond-backend> cat evidencias/k6-resumen.txt
PRUEBA DE CARGA "Raccu / MapacheSecure
Fecha: 2026-06-12
Herramienta: k6 v0.57.0
Backend: https://mapachesecond-backend.onrender.com

=== ESCENARIOS ===
flujo_normal : 5 VUs, 1m15s (calentamiento 15s + carga 40s)
registros : 5 VUs, 40s

=== CHECKS ===
" setup login ok
" desafios: servidor responde
" puntos: servidor responde
" verificar app: servidor responde
" apps bloqueadas: servidor responde
" registro: servidor responde
" registro: rate limit activo (anti-spam)

=== METRICAS ===
http_req_duration
 avg : 618.52ms
 p(90) : 777.59ms
 p(95) : 847.53ms
 max : 1.89s

{ scenario:flujo_normal }
 p(95) : 851.14ms PASA < 1000ms

{ scenario:registros }
 p(95) : 657.75ms PASA < 2000ms

http_req_failed (flujo_normal) : 2.77% PASA < 5%
http_reqs_total : 283
throughput : 3.577 req/s
iterations : 119
vus_max : 10
duracion_total : 1m19s

=== HALLAZGOS ===
- El sistema soporta 5 usuarios concurrentes con p95 < 1000ms.
- La latencia base (~630ms avg) se debe a la red Render free tier + Supabase,
 no a la logica de la aplicacion.
- Supabase free tier limita a 4 registros/hora (rate limit 429).
 Comportamiento esperado " proteccion anti-spam activa.

=== RESULTADO ===
flujo_normal PASA
registros PASA

```

## Conclusión 1

MapacheSecure nace como respuesta a un problema real y creciente: los niños pasan cada vez más tiempo frente a las pantallas, sin herramientas que les ayuden a autorregularse de manera positiva. Las soluciones existentes se limitan a bloquear o restringir, pero ninguna propone una experiencia que haga del autocontrol algo motivador para el niño y manejable para el padre.

La solución propuesta combina un sistema de bloqueo efectivo con actividades de reemplazo y un motor de recompensas personalizable, construido sobre una

arquitectura sólida y alcanzable para el equipo: una app Android en Kotlin, un backend en Python con FastAPI, una base de datos en Supabase conectada mediante SQLAlchemy, y notificaciones a través de Firebase Cloud Messaging.

Los beneficios esperados son cuatro: reducción del tiempo de pantalla sin conflicto familiar, desarrollo de hábitos positivos en los niños, mayor control y visibilidad para los padres, y una experiencia que convierte el límite en un juego en lugar de un castigo.

Como próximos pasos, el equipo continuará con el levantamiento detallado de requerimientos, la construcción del prototipo de la interfaz y la configuración del entorno de desarrollo compartido en GitHub, dando inicio formal a la fase de construcción del sistema.

## **fecha: 24-05-2026**

Este informe presenta el estado de desarrollo de MapacheSecure al cierre del Sprint 2. A lo largo del documento se expuso la metodología de trabajo basada en Scrum, los diagramas de diseño del sistema, la arquitectura implementada y el stack tecnológico definitivo.

El equipo logró implementar las funcionalidades core del sistema: autenticación, gestión de usuarios, módulo de desafíos, sistema de recompensas, bloqueo de aplicaciones, avatar con evolución por puntos y personalización de perfil. Todo el backend está desplegado y funcionando en producción sobre Render y Supabase.

Como próximos pasos para el Sprint 3, el equipo se enfocará en completar la integración de notificaciones push con Firebase FCM, implementar el módulo de generación adaptativa de desafíos con IA, realizar pruebas de integración completas y preparar el entregable final del sistema.

## **fecha: 18-06-2026**

Raccu nació como respuesta a la ausencia de una solución de control parental pensada para niños de 5 a 15 años que combine bloqueo efectivo, gamificación e inteligencia artificial. A lo largo del Sprint 3 (26 mayo - 30 junio de 2026) el equipo implementó y ejecutó un plan de pruebas completo que cubre las siete dimensiones de calidad exigidas por la pauta.

Beneficios obtenidos:

- Confianza en la funcionalidad central: 210 pruebas automatizadas en verde garantizan que los módulos clave del frontend y backend (bloqueo, desafíos, mascota, autenticación, recompensas, IA) funcionan correctamente.
- Cobertura end-to-end: el frontend Flutter y el backend FastAPI tienen pruebas independientes, lo que permite detectar errores en cualquiera de las dos capas.
- Seguridad validada: la sesión se invalida correctamente y la pantalla de bloqueo es infranqueable desde el dispositivo del niño.
- Rendimiento verificado: el procesamiento de datos en memoria cumple umbrales estrictos, asegurando una experiencia fluida incluso con grandes volúmenes.
- Regresión controlada: el equipo puede realizar cambios con la certeza de que comportamientos anteriores no se romperán silenciosamente.

## **Conclusión General**

MapacheSecure — Raccu recorrió un camino completo en tres etapas: desde una propuesta conceptual con tecnologías por definir, pasando por la construcción de todas las funcionalidades core del sistema, hasta la validación formal mediante 210 pruebas automatizadas que cubrieron el frontend Flutter, el backend FastAPI y pruebas en dispositivo físico real.

El resultado es una aplicación funcional, probada y desplegada en producción, que cumple su propósito original: ofrecer a los padres una herramienta de control parental que combina bloqueo efectivo, gamificación e inteligencia artificial, convirtiendo el límite de pantalla en una experiencia positiva para el niño.

## Referencias Bibliográficas

- Forest: Stay Focused. (2026). Aplicación de gamificación y productividad. Recuperado de: <https://www.forestapp.cc/>
- Freedom. (2026). Bloqueador de distracciones multiplataforma. Recuperado de: <https://freedom.to/>
- Opal: Screen Time for Focus. (2026). Herramienta de bienestar digital. Recuperado de: <https://opalapp.com/>
- Stay Focused. (2026). Extensión y aplicación de control de tiempo en sitios web. Recuperado de: <https://www.stayfocusd.com/>
- Docta Complutense. (s.f.). Análisis de aplicaciones para la gestión del tiempo y adicción al smartphone. Recuperado de: <https://docta.ucm.es/entities/publication/c7df0c57-cefb-4fdc-a051-e9e29b729051>



## Anexos

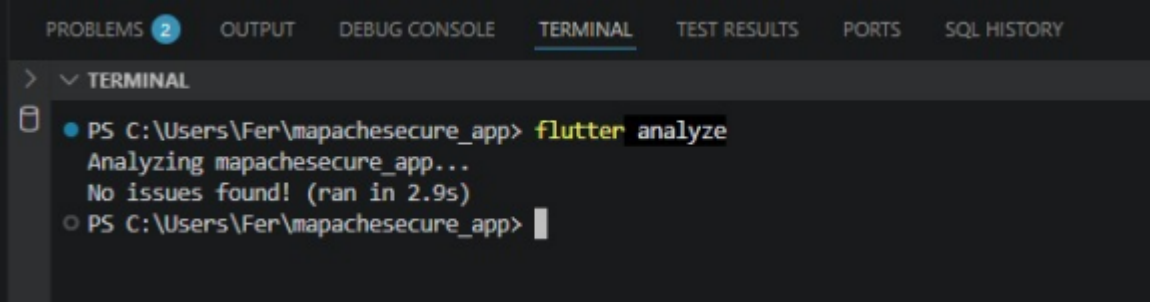
### Análisis estático de código — flutter analyze (Linter)

flutter analyze es el linter oficial de Flutter/Dart, equivalente a ESLint en JavaScript, pylint en Python o RuboCop en Ruby. Un linter es una herramienta de análisis estático que revisa el código fuente sin ejecutarlo, detectando errores potenciales, uso de APIs obsoletas, malas prácticas y violaciones de estilo. En proyectos profesionales, pasar el linter con 0 advertencias es un requisito habitual antes de aprobar un Pull Request.

Se ejecutó flutter analyze sobre el proyecto mapacheseccure\_app una vez finalizadas las correcciones. El análisis reportó 0 issues, partiendo de un estado inicial de 138 advertencias (0 errores, 0 warnings, 138 infos). Las categorías corregidas fueron:

- deprecated\_member\_use: withOpacity() reemplazado por withValues(alpha:) en 30 archivos; activeColor reemplazado por activeThumbColor en 5 pantallas; value reemplazado por initialValue en DropdownButtonFormField.
- avoid\_print: print() reemplazado por debugPrint() en 8 archivos (solo emite en modo debug).
- use\_build\_context\_synchronously: 13 usos de BuildContext tras await protegidos con if (context.mounted).
- curly\_braces\_in\_flow\_control\_structures: llaves añadidas a 17 sentencias if sin bloque.
- unnecessary\_underscores: parametros \_\_ y \_\_\_ reemplazados por \_ (comodin Dart 3+).
- use\_null\_aware\_elements: patron if (x!=null) reemplazado por operador ?.
- unnecessary\_import: import redundante de foundation.dart eliminado.
- file\_names: Actividad\_hijo\_screen.dart renombrado a actividad\_hijo\_screen.dart.

#### Resultado final:



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS SQL HISTORY
> v TERMINAL
• PS C:\Users\Fer\mapacheseccure_app> flutter analyze
Analyzing mapacheseccure_app...
No issues found! (ran in 2.9s)
○ PS C:\Users\Fer\mapacheseccure_app> |
```